BNCmatmul User's Guide

Tomonori Kouya
https://github.com/tkouya/bncmatmul

Version 0.23: March 16, 2026

Contents

Chapter 1 What's BNCmatmul? 1
1.1 Copyright and condition for distribution . . . 1
1.2 How to compile BNCmatmul . . . 1
1.3 Mathematical notation . . . 2
1.4 Why do we require a multiple-precision floating-point arithmetic library? . . . 2
1.5 Algorithms and performance of optimized matrix multiplication . . . 3
1.6 Software layer of BNCmatmul . . . 5
1.7 History of Version and Todo list . . . 6

Chapter 2 Basic datatypes and arithmetic 7
2.1 c_dd_qd.h . . . 7
2.2 Error-free transformation . . . 8
2.3 Double-double precision arithmetic . . . 8
2.4 Quadruple-double precision arithmetic . . . 9
2.5 Triple-double precision arithmetic . . . 10
2.6 rdd.h . . . 11
2.7 AVX2 . . . 13
2.8 Miscellaneous functions . . . 16
2.9 Tool functions for benchmarking . . . 17

Chapter 3 Basic linear computation 20
3.1 Detatypes of D, DD, TD, QD and MPFR vector and matrix . . . 20
3.2 Vector arithmetic . . . 21
3.3 Matrix arithmetic . . . 23
3.4 File I/O with matrix and vector . . . 26
3.5 Generating test matrices . . . 27
3.6 Simple, block, and Strassen matrix multiplication and related functions . . . 28
3.7 Getting relative errors of vector and matrix . . . 29
3.8 Ozaki scheme and related functions . . . 30

Chapter 4 LU decomposition 32
4.1 List of functions . . . 32

Chapter 5 Array, polynomial and solvers of algebraic equations 34
5.1 Data types of Array and polynomial . . . 34
5.2 List of functions . . . 35

Chapter 6 Sparse matrix and related functions 45
6.1 Structures of CRS matrix . . . 45
6.2 Krylov subspace iterative solvers . . . 52

Bibliography 61

Index 62

Chapter 1

What's BNCmatmul?

BNCmatmul is one of optimized BLAS(Basic Linear Algebra Subprograms) libaries supporting multiple precision arithmetics such as double(binary64), DD(double-double), TD(triple-double), QD(quadruple-double), and arbitrary precision floating point arithmetic of MPFR.

Now that MPLAPACK/MPBLAS, a library for extended multiple precision linear computation, provides the basis for a standard multiple precision numerical computing environment, there is a growing demand for an optimized library that provides faster basic linear computation. Standard optimization methods include the use of SIMD instructions and parallelization with OpenMP, but for multiple precision matrix multiplication, optimization with algorithms such as divide-and-conquer method and Ozaki scheme are also available. We have developed new version of BNCmatmul that can use all of these optimization methods and supports both fixed-precision computation using the multi-component way and arbitrary-precision computation using the multi-digit way. In this paper, we describe its software structure and present the results of performance evaluation of optimized matrix-vector multiplication and matrix multiplication.

We are developing BNCmatmul on the following environment: Xeon and EPYC. On other x86_64 linux environment with Intel OneAPI or GNU Compiler Collection, our library can probably be available.

EPYC AMD EPYC 7402P 24 cores, Ubuntu 18.04.6 LTS, Intel Compiler version 2021.4.0, MPLAPACK 1.0.1, MPFR 4.1.0

Xeon Intel Xeon W-2295 3.0GHz 18 cores, Ubuntu 20.04.3 LTS, Intel Compiler version 2021.5.0, MPLAPACK 1.0.1, MPFR 4.1.0

From BNCmatmul version 0.23, Arm Neon of AArch64 can be used to accelerate our library. We has tested in following Arm CPU environment,

Raspberrypi Raspberry Pi 5 Model B Rev 1.0, Broadcom BCM2712 2.4GHz Coretex-A76 4 cores, Debian GNU/Linux 12 ,GCC 12.2.0, MPLAPACK 2.0.1

Snapdragon Qualcomm Snapdragon X Plus 3.24GHz 8 cores, Ubuntu 24.04.3 LTS on Windows 11, GCC 13.3.0, MPLAPACK 2.0.1

1.1 Copyright and condition for distribution

BNCmatmul is originated by Tomonori Kouya (`https://na-inet.jp/`), and including this document is distributed under [GPL] version 2 or later.

[GPL]: `https://www.gnu.org/copyleft/gpl.html`

1.2 How to compile BNCmatmul

BNCmatmul can be compiled with Intel OneAPI Compiler (ICC) or GNU Compiler Collection (GCC) as follows:

1. Git clone at your home directory from `https://github.com/tkouya/bncmatmul`.

2. Edit bncmatmul.inc to select ICC or GCC for compilation.
3. Run "make all" to build `libbncmatmul*.a` and `python/libncmatmul*.so`.
4. Compile some iterative refinement C++ programs in "test" directory with BNCmatmul and MPLAPACK/MPBLAS.

SIMD arithmetic can accelerate BNCmatmul fixed-precision multi-component-way aritmetic such as DD

1.3 Mathematical notation

Here, we define $\mathbb{F}_{bS}$ and $\mathbb{F}_{bL}$ as sets of the S- and L-bit mantissas of floating-point numbers, respectively. For instance, $\mathbb{F}_{b24}$ and $\mathbb{F}_{b53}$ refer to sets of IEEE754-1985 binary32 and binary64 floating-point numbers, whereas $\mathbb{F}_{b106}$, $\mathbb{F}_{b159}$, and $\mathbb{F}_{b212}$ represent examples of DD, TD, and QD precision floating-point numbers, respectively. Although any mantissa length can be selected in MPFR arithmetic, the set of MPFR numbers is expressed as $\mathbb{F}_{bM}$, which is primarily defined as M-bit using the `mpfr_set_default_prec` function.

Moreover, we use $(\mathbf{x})_i (= x_i)$ as the i-th element of the n-dimensional vector $\mathbf{x} = [x_i]_{i=1,2,..,n} \in \mathbb{R}^n$, and $(A)_{ij} (= a_{ij})$ as the (i, j)-th element of $A = [a_{ij}]_{i=1,2,...,m,j=1,2,...,n} \in \mathbb{R}^{m\times n}$.

1.4 Why do we require a multiple-precision floating-point arithmetic library?

The following is a brief explanation regarding the need for multiple-precision calculations with a mantissa exceeding binary64. First, users may seek to use floating-point arithmetic to obtain a value $F(x)$ from an input value x represented by a floating-point number. Ultimately, $F(x)$ must maintain at least U significant digits. Intuitively, in numerical computations, the initial error due to rounding is assumed to be included in the input value. Suppose the number of mantissa digits for the floating-point arithmetic to be $L(> U)$. Then, the number of significant digits of $F(x)$ is U–R, where R denotes the digits lost in the process of computing $F(x)$. Furthermore, it is assumed that the algorithm cannot be modified to ensure accuracy with an L-digit computation.

Unless the calculation algorithm for $F(x)$ is modified, the error propagation minimally changes irrespective of the number of mantissa digits. Therefore, we can increase the number of mantissa digits, slightly exceeding R with some slack of α digits, to $L+R+\alpha$ digits. If the initial error can be significantly minimized, $F(x)$ with U or more significant digits can be subsequently obtained. This process illustrates how multiple-precision calculations are used to ensure accuracy.

In contrast, the multi-fold calculation method proposed by Rump and Ogita [9] [10] prevents an increase in the initial error by suppressing the rounding error generated by arithmetic operations to the lower digits using error-free transformation techniques. The computational manner of the error-free transformation process is almost the same as that of multiple-precision calculations. Moreover, in this process, floating-point arithmetic with L or more digits is not used. Accordingly, there is no need to perform renormalization procedures. This offers the advantage of reducing computational complexity compared to that of a multi-component approach that uses error-free transformation techniques.

A conceptual diagram of the two aforementioned approaches is presented in Fig. 1.1.

Currently, not all algorithms used in numerical computation can be applied using multi-fold arithmetic, especially with nonlinear calculations. Therefore, it is necessary to employ a combination of multi-fold and multiple-precision calculations, where the basic linear calculation part, such as explicit extrapolation process solving initial value problems of ordinary differential equations, uses multi-fold calculations to reduce computational time.

The Ozaki scheme, a matrix multiplication algorithm that we incorporated into our library, is a technique based on the multi-fold calculation approach. By leveraging the speed of existing binary32 and binary64 xGEMM algorithms, it significantly optimizes multiple-precision matrix multiplication under certain conditions, as revealed in our benchmark and previous studies[7][11]. We are confident that the acceleration of multiple-precision linear computation libraries will remain an important theme in guaranteeing the accuracy of broader numerical computation algorithms, following the development

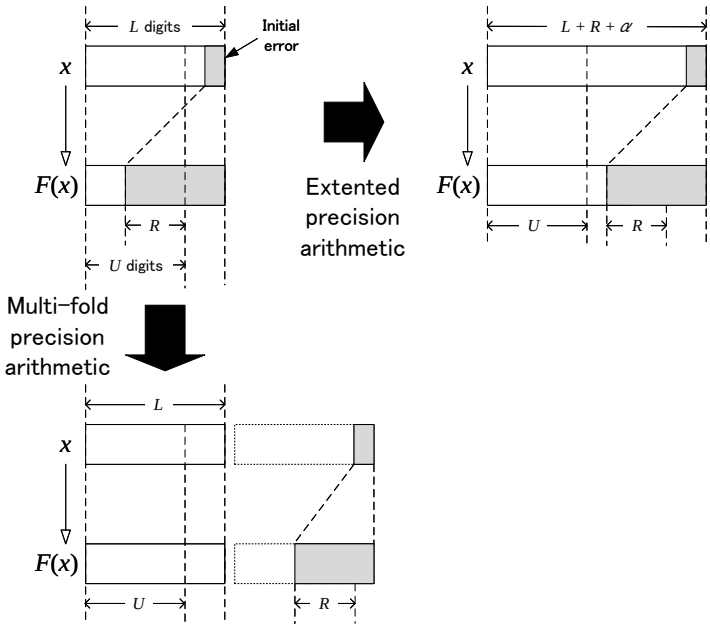

Fig. 1.1 Extended precision and multi-fold precision arithmetics

trend of multiple-precision numerical algorithms.

1.5 Algorithms and performance of optimized matrix multiplication

We focused on the optimization of multiple-precision matrix multiplication, starting with MPI support for arbitrary-precision numerical calculations[4]. Subsequently, owing to current popularity of multi-core CPUs, we accelerated multiple-precision matrix multiplication via parallelization support using OpenMP. As far as multiple-precision calculations are concerned, no other such results have been achieved by incorporating divide-and-conquer methods, such as the Strassen and Winograd algorithms. Although limited to multi-component methods, our proposed method is the only approach that supports AVX2 and has been converted to OpenMP for achieving higher speeds. However, the current code does not improve performance beyond eight threads for OpenMP acceleration, and a drastic reformulation is necessary to further improve parallelization performance. Algorithm 1 is the Strassen matrix multiplication.

The usefulness of the Ozaki scheme is also clear at multiple-precision levels owing to the success of Mukunoki et al. in accelerating the float128 precision matrix multiplication[7]. The float128 arithmetic supported by GCC features TD to QD precision performance for addition and multiplication and is likewise expected to be sufficient for this precision range.

The Ozaki scheme is an algorithm that aims to simultaneously accelerate performance and improve accuracy by dividing matrices into more matrices with elements represented by shorter digits. Similarly to the "Split" method used in the error-free transformation technique, this approach leverages on the speed of optimized short-precision matrix multiplication (xGEMM) functions. For a given matrix $A \in \mathbb{R}^{m \times l}$ and $B \in \mathbb{R}^{l \times n}$, to obtain a matrix product $C := AB \in \mathbb{R}^{m \times n}$ of long L-bit precision, A and B are divided using the Ozaki scheme, where $D \in \mathbb{N}$ is the maximum number of divisions of short S-bit precision matrices ($S << L$), as depicted in Algorithm 2. The S-bit arithmetic is used for calculations where no particular description is given, and the L-bit arithmetic is used only where high-precision operations are required.

Fig. 1.2 illustrates an example of the Ozaki scheme when A and $B \in \mathbb{R}^{3 \times 3}$ are divided into three short-digit matrices. The most important feature of the Ozaki scheme is that the matrices A and B are

Algorithm 1 Strassen algorithm for matrix multiplication

Input: $A = [A_{ij}]_{i,j=1,2} \in \mathbb{R}^{m\times l}$, $A_{ij} \in \mathbb{R}^{m/2\times l/2}$, $B = [A_{ij}]_{i,j=1,2} \in \mathbb{R}^{l\times n}$, $B_{ij} \in \mathbb{R}^{l/2\times n/2}$
Output: $C := \mathrm{Strassen}(A, B) = AB \in \mathbb{R}^{m\times n}$

if $m < m_0$ && $n < n_0$ **then**
  $C := AB$
end if
$P_1 := \mathrm{Strassen}(A_{11} + A_{22}, B_{11} + B_{22})$
$P_2 := \mathrm{Strassen}(A_{21} + A_{22}, B_{11})$
$P_3 := \mathrm{Strassen}(A_{11}, B_{12} - B_{22})$
$P_4 := \mathrm{Strassen}(A_{22}, B_{21} - B_{11})$
$P_5 := \mathrm{Strassen}(A_{11} + A_{12}, B_{22})$
$P_6 := \mathrm{Strassen}(A_{21} - A_{11}, B_{11} + B_{12})$
$P_7 := \mathrm{Strassen}(A_{12} - A_{22}, B_{21} + B_{22})$
$C_{11} := P_1 + P_4 - P_5 + P_7$; $C_{12} := P_3 + P_5$
$C_{21} := P_2 + P_4$; $C_{22} := P_1 + P_3 - P_2 + P_6$
$C := [C_{ij}]_{i,j=1,2}$

Algorithm 2 Ozaki scheme for multiple-precision matrix multiplication

Input: $A \in \mathbb{F}_{bL}^{m\times l}, B \in \mathbb{F}_{bL}^{l\times n}$
Output: $C \in \mathbb{F}_{bL}^{m\times n}$

$A^{(S)} := A$, $B^{(S)} := B$: $A^{(S)} \in \mathbb{F}_{bS}^{m\times l}$, $B^{(S)} \in \mathbb{F}_{bS}^{l\times n}$
$\mathbf{e} := [1\ 1\ ...\ 1]^T \in \mathbb{F}_{bS}^{l}$
$\alpha := 1$
while $\alpha < D$ **do**
  $\boldsymbol{\mu}_A := [\max_{1\le p\le l} |(A^{(S)})_{ip}|]_{i=1,2,\ldots,m} \in \mathbb{F}_{bS}^{m}$
  $\boldsymbol{\mu}_B := [\max_{1\le q\le l} |(B^{(S)})_{qj}|]_{j=1,2,\ldots,n} \in \mathbb{F}_{bS}^{n}$
  $\boldsymbol{\tau}_A := [2^{\lceil \log_2((\boldsymbol{\mu}_A)_i)\rceil + \lceil (S+\log_2(l))/2\rceil}]_{i=1,2,\ldots,m} \in \mathbb{F}_{bS}^{m}$
  $\boldsymbol{\tau}_B := [2^{\lceil \log_2((\boldsymbol{\mu}_B)_j)\rceil + \lceil (S+\log_2(l))/2\rceil}]_{j=1,2,\ldots,n} \in \mathbb{F}_{bS}^{n}$
  $S_A := \boldsymbol{\tau}_A \mathbf{e}^T$
  $S_B := \mathbf{e}\boldsymbol{\tau}_B^T$
  $A_\alpha := (A^{(S)} + S_A) - S_A$: $A_\alpha \in \mathbb{F}_{bS}^{m\times l}$
  $B_\alpha := (B^{(S)} + S_B) - S_B$: $B_\alpha \in \mathbb{F}_{bS}^{l\times n}$
  $A := A - A_\alpha$, $B := B - B_\alpha$: L-bit FP arithmetic
  $A^{(S)} := A$, $B^{(S)} := B$
  $\alpha := \alpha + 1$
end while
$A_D := A^{(S)}$, $B_D := B^{(S)}$
$C := O$
for $\alpha = 1, 2, ..., D$ **do**
  for $\beta = 1, 2, ..., D - \alpha + 1$ **do**
    $C_{\alpha\beta} := A_\alpha B_\beta$
  end for
  $C := C + \sum_{\beta=1}^{D-\alpha+1} C_{\alpha\beta}$: L-bit FP arithmetic
end for

divided into A_1, A_2, A_3, B_1, B_2, and B_3 to fit within a short precision, thereby avoiding rounding errors in fast, low-precision matrix multiplication. An error-free matrix product $C_{ij} := A_i B_j (i, j = 1, 2, 3)$ can be obtained as a result, and a highly accurate matrix product C can be obtained using a multiple-precision matrix addition operation for $C := \sum_{i,j} C_{ij}$. Although the number of divisions of A and B is finite, it is difficult to determine the minimum number of divisions required to guarantee a certain accuracy threshold. As a result, benchmark tests must be performed to determine if the algorithm can be executed faster than other multiplication algorithms.

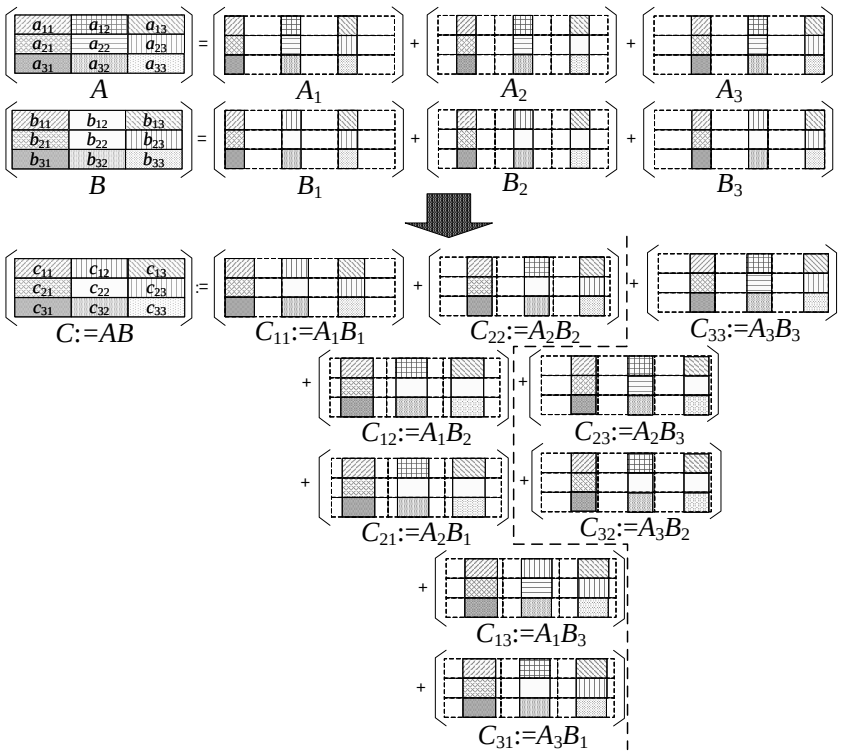

Fig. 1.2 Matrix multiplication based on Ozaki scheme when the matrices are divided into three components

1.6 Software layer of BNCmatmul

We have already used QD[1] as multi-component-type fixed precision C++ class library, CAMPARY[3] as multi-component-type arbitrary precision library, and MPFR[8] baased on GNU MP[2]'s MPN kernel. MPLAPACK/MPBLAS[6] is greeding these all reliable MP libraries and providing the coresponding APIs of LAPACK/BLAS[5]. Our BNCmatmul developed aims to get better performance than MPBLAS as ATLAS, OpenBLAS and Intel Math Kernel, and it has the following features:

- Core functions and macros are wriiten in the traditional way of ANSI C, not C++.
- Supporting native multi-component-type DD, TD, and QD precision arithmetic, and also accelelating vector and matrix computation like BLAS with AVX2. AVX-512 is partly implemented but not recommended to use.
- Direct use of MPFR functions to avoid overhead due to mpreal class library.
- Supporting partly shared-memory-type parallelization based on OpenMP.
- Implementing three algorightms of matrix multiplication: simple triple-loop, blocking (tiling), and Strassen or Winograd algorithm. Ozaki scheme can be used as trial optimization.

The software layer of BNCmatmul except parallelization is shown as Fig. 1.3.

Basic DD, TD, and QD precision arithmetic are defined as C inline functions in c_dd_qd.h and are defined as macros in rdd.h. To use the Python environment, rdd.c, which is based on rdd.h, is compiled as librdd.so. The DD, TD, and QD classes on Python use the functions defined in librdd.so.

BNCmatmul, which is also built on rdd.h and c_dd_qd.h, includes block and Strassen matrix multiplications that provide more efficient performance than simple triple-loop matrix multiplication. These have been accelerated using SIMDized functions with AVX2 defined in bncavx2.h. The LU decomposition has also been accelerated with AVX2.

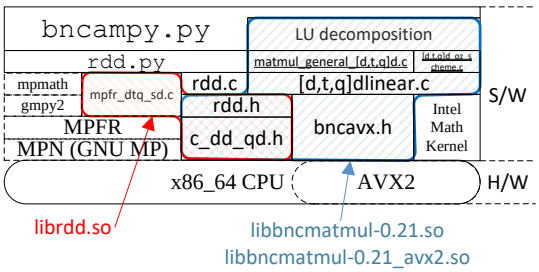

Fig. 1.3 Software layer of BNCmatmul library

These functions that are defined in BNCmatmul (Fig. 1.3) can be used from libbncmatmul-0.xx.so without any SIMDization techniques, libbncmatmul-0.xx_avx2.so with SIMDization of AVX2, and libbncmatmul-0.xx_neon.so with that of Arm Neon. As these DLLs have the same function names, so users do not need to modify their codes when selecting the BNCmatmul DLLs.

1.7 History of Version and Todo list

■History of Version

Version 0.23: 2026-03-16 BNCmatmul accelerated on Arm Neon CPUs has been released.

Version 0.22: 2024-05-31 Some functions have been appended:

1. Complex arithmetic and linear computation
2. Miscellaneous functions (accessares, elementary and random functions)
3. Sparse matrix and SpMV

Version 0.21: 2023-05-31 Secondly opening sources including DD, TD, QD and MPFR precision real BLAS functions at `https://github.com/tkouya/bncmatmul/blob/main/bncmatmul-0.21.tar.bz2`.

Version 0.20: 2016-06-08 Firstly opening sources at `https://na-inet.jp/na/bnc/bncmatmul-0.2.tar.bz2`, which provides only artibrary matrix multiplication based on MPFR.

■Todo list

1. Showing more sample sources in this manual using BNCmatmul and MPLAPACK/BLAS
2. Completing descriptions of our library usage and how to apply various situations with it.

Chapter 2

Basic datatypes and arithmetic

In this chapter, we present and describe all basic data types and functions defined in BNCmatmul, and show how to use them in order to construct your programs including multiple-precsion computing.

As shown in Chapter 1, all basic datatype and functions are written in ANSI C, not C++ way.

2.1 c_dd_qd.h

(Macro) `DFMA`(a, b, c) returns $a \times b + c$ in double precision
(Macro) `SFMA`(a, b, c) returns $a \times b + c$ in single precision
(Macro) `QD_FMA`(a, b, c) the same as DFMA(a, b, c)
(Macro) `QD_FMS`(a, b, c) returns $a \times b - c$ in double precision

(Macro) DDSIZE is 2, the number of binary64(double) variables in DD
(Macro) TDSIZE is 3, the number of binary64 variables in TD
(Macro) QDSIZE is 4, the number of binary64 variables in QD

2.1.1 DD (double-double): 106-bit (53bits $\times$ 2) precision floating-point number

(Macro) `DD_HI`(a) is a[0].
(Macro) `DD_LOW`(a) is a[1].
(Macro) DD_TRUE is 1UL.
(Macro) TD_TRUE
(Macro) QD_TRUE
(Macro) DD_FALSE is 0UL.
(Macro) TD_FALSE
(Macro) QD_FALSE
(Macro) `DD_ISNAN`(a) checks if a includes NAN.
(Macro) `TD_ISNAN`(a)
(Macro) `QD_ISNAN`(a)
(Macro) `DD_ISINF`(a) checks if a includes INF.
(Macro) `TD_ISINF`(a)
(Macro) `QD_ISINF`(a)
(Macro) `DD_ISZERO`(a) checks if a is zero.
(Macro) `TD_ISZERO`(a)
(Macro) `QD_ISZERO`(a)
(Macro) `DD_ISONE`(a) checks if a is one.
(Macro) `TD_ISONE`(a)
(Macro) `QD_ISONE`(a)
(Macro) `DD_ISNEGATIVE`(a) checks if a < 0.
(Macro) `TD_ISNEGATIVE`(a)
(Macro) `QD_ISNEGATIVE`(a)

(Macro) DD_NAN is `FP\_NAN`.
(Macro) TD_NAN
(Macro) QD_NAN

2.2 Error-free transformation

- double `quick_two_sum`(double a, double b, double * `err`) $\cdots$ Computes $\mathrm{fl}(a+b)$ and $\mathrm{err}(a+b)$. Assumes $|a| \geq |b|$.
- double `quick_two_diff`(double `a`, double `b`, double * `err`) $\cdots$ Computes $\mathrm{fl}(a-b)$ and $\mathrm{err}(a-b)$. Assumes $|a| >= |b|$.
- double `two_sum`(double `a`, double `b`, double * `err`) $\cdots$ Computes $\mathrm{fl}(a+b)$ and $\mathrm{err}(a+b)$.
- double `two_diff`(double `a`, double `b`, double * `err`) $\cdots$ Computes $\mathrm{fl}(a-b)$ and $\mathrm{err}(a-b)$.
- void `split`(double `a`, double *hi, double *lo) $\cdots$ divide a to hi and lo without error.
- double `two_prod`(double `a`, double `b`, double * `err`) $\cdots$ Computes $\mathrm{fl}(ab)$ and $\mathrm{err}(ab)$.
- double `two_sqr`(double `a`, double * `err`) $\cdots$ Computes $\mathrm{fl}(a^2)$ and $\mathrm{err}(a^2)$. Faster than the above method.

2.3 Double-double precision arithmetic

- dd_bool `dd_is_zero`(const double * `x`) $\cdots$ Check if *x is zero.
- dd_bool `dd_is_one`(const double * `x`) $\cdots$ Check if *x is one.
- void `c_dd_set`(const double * `x`, double `*ret`) $\cdots$ $ret := x$
- void `c_dd_set_dd_d`(const double `x`, double * `ret`) $\cdots$ $ret := (double)x$
- void `c_dd_set0`(double * `ret`) $\cdots$ $ret := 0$
- void `c_dd_setnan`(double * `ret`) $\cdots$ $ret := NaN$
- void `c_dd_neg`(const double * `a`, double * `b`) $\cdots$ $b := -a$
- dd_bool `dd_is_positive`(const double * `x`) $\cdots$ *x > 0 ?
- dd_bool `dd_is_negative`(const double * `x`) $\cdots$ *x < 0 ?
- void `c_dd_comp`(const double * `a`, const double * `b`, int * `result`) $\cdots$ Compare with a and b, and store 0 if a == b, +1 else if a > b, −1 else if a < b.
- void `c_dd_comp_dd_d`(const double * `a`, double `b`, int * `result`)
- void `c_dd_comp_d_dd`(double `a`, const double * `b`, int * `result`)
- void `c_dd_pi`(double * `a`)
- void `c_d_add`(double `a`, double `b`, double * `c`) $\cdots$ double-double := double + double
- void `c_dd_add`(const double * `a`, const double * `b`, double * `c`) $\cdots$ $c := a + b$
- void `c_dd_add_sloppy`(const double * `a`, const double * `b`, double * `c`)
- void `c_dd_add_dd_d`(const double * `a`, double `b`, double * `c`)
- void `c_dd_add_d_dd`(double `a`, const double * `b`, double * `c`)
- void `c_d_sub`(double `a`, double `b`, double * `c`) $\cdots$ double-double = double - double
- void `c_dd_sub`(const double * `a`, const double * `b`, double * `c`) $\cdots$ $c := a - b$
- void `c_dd_sub_sloppy`(const double * `a`, const double * `b`, double * `c`)
- void `c_dd_sub_dd_d`(const double * `a`, double `b`, double * `c`)
- void `c_dd_sub_d_dd`(double `a`, const double * `b`, double * `c`)
- void `c_d_mul`(double `a`, double `b`, double * `c`) $\cdots$ double-double := double * double
- void `c_dd_mul`(const double * `a`, const double * `b`, double * `c`)
- void `c_dd_mul_dd_d`(const double * `a`, double `b`, double * `c`)
- void `c_dd_mul_d_dd`(double `a`, const double * `b`, double * `c`)
- void `c_d_div`(double `a`, double `b`, double * `c`) $\cdots$ $c := a/b$
- void `c_dd_div`(const double * `a`, const double * `b`, double * `c`) $\cdots$ $c := a/b$
- void `c_dd_sloppy_div`(double * `a`, double * `b`, double * `c`)
- void `c_dd_div_dd_d`(const double * `a`, double `b`, double * `c`)

- void c_dd_div_d_dd(double a, const double * b, double * c)
- void c_dd_copy(const double * a, double * b) $b := a$
- void c_dd_copy_d(double a, double * b)
- void c_d_sqr(double a, double * b) $\cdots$ $b := a^2$
- void c_dd_sqr_d(double a, double * ret)
- void c_dd_sqr(const double * a, double * b)
- void c_dd_sqrt(const double * a, double * b) $\cdots$ $b := \sqrt{(a)}$
- void c_dd_abs(const double * a, double * b) $\cdots$ $b := |a|$
- void c_dd_floor(const double * a, double * b) $\cdots$ $b := \mathrm{floor}(a)$
- void c_dd_ceil(const double * a, double * b) $\cdots$ $b := \mathrm{ceil}(a)$

2.4 Quadruple-double precision arithmetic

- void c_qd_copy(const double * a, double * b) $\cdots$ $b := a$
- void c_qd_copy_dd(const double * a, double * b)
- void c_qd_copy_d(double a, double * b)
- void quick_renorm(double * c0, double * c1, double * c2, double * c3, double * c4) $\cdots$ Renormalize c0 ...
- void renorm(double * c0, double * c1, double * c2, double * c3)
- void renorm4(double * c0, double * c1, double * c2, double * c3, double * c4)
- void three_sum(double * a, double * b, double * c)
- void three_sum2(double * a, double * b, double * c)
- void c_qd_add_qd_dd(const double * a, const double * b, double * c) $\cdots$ $c := a + b$
- void c_qd_add(const double * a, const double * b, double * c)
- void c_qd_add_sloppy(const double * a, const double * b, double * c) $\cdots$ not so accurate but fast addition $c := a + b$
- void c_td_addq(const double * a, const double * b, double * c) $\cdots$ triple double addition based on quad-double way
- void c_qd_selfadd(const double * a, double * b) $\cdots$ $b := b + a$
- void c_qd_selfadd_dd(const double * a, double * b)
- void c_qd_selfadd_d(double a, double * b)
- void c_qd_neg(const double * a, double * b) $\cdots$ $b := -a$
- void c_qd_neg_dd(const double * a, double * b)
- void c_qd_neg_d(const double a, double * b)
- void c_qd_sub(const double * a, const double * b, double * c) $\cdots$ $c := a - b$
- void c_qd_sub_qd_dd(const double * a, const double * b, double * c)
- void c_qd_sub_dd_qd(const double * a, const double * b, double * c)
- void c_qd_sub_qd_d(const double * a, double b, double * c)
- void c_qd_sub_d_qd(double a, const double * b, double * c)
- void c_qd_selfsub(const double * a, double * b) $\cdots$ $b := b + (-a) = b - a$
- void c_qd_selfsub_dd(const double * a, double * b)
- void c_qd_selfsub_d(double a, double * b)
- void c_qd_mul(const double * a, const double * b, double * c) $\cdots$ $c := a \times b$
- void c_qd_mul_qd_dd(const double * a, const double * b, double * c)
- void c_qd_mul_dd_qd(const double * a, const double * b, double * c)
- void c_qd_mul_qd_d(const double * a, double b, double * c)
- void c_qd_mul_d_qd(double a, const double * b, double * c)
- void c_qd_mul_sloppy(const double * a, const double * b, double * c)
- void c_qd_sqr(const double * a, double * c) $\cdots$ $c = a^2$
- void c_qd_selfmul(const double * a, double * b) $\cdots$ $b := a * b$
- void c_qd_selfmul_dd(const double * a, double * b)
- void c_qd_selfmul_d(double a, double * b)

- void c_qd_div_accurate(const double * a, const double * b, double * c) $\cdots$ $c := a/b$
- void c_qd_div_sloppy(const double * a, const double * b, double * c) $\cdots$ not so accurate but fast division (default)

(Macro) USE_QD_DIV_ACCURATE() $\cdots$ use accurate qd division if true

- void c_qd_div_qd_dd(const double * a, const double * b, double * c)
- void c_qd_div_qd_d(const double * a, double b, double * c)
- void c_qd_div_d_qd(double a, const double * b, double * c)
- void c_qd_selfdiv(const double * a, double * b) $\cdots$ $b := b/a$
- void c_qd_selfdiv_dd(const double * a, double * b)
- void c_qd_selfdiv_d(double a, double * b)
- void c_qd_mul_pwr2(const double * a, double b, double * c) $\cdots$ $c := (a[0]*b, a[1]*b, a[2]*b, a[3]*b)$
- void c_qd_set(const double * x, double * qdval) $\cdots$ $qdval := c$
- void c_qd_set0double *qdval() $\cdots$ $qdval := 0$
- void c_qd_sqrt(const double * a, double * b) $\cdots$ $b := \sqrt{a}$
- void c_qd_abs(const double * a, double * b) $\cdots$ $b := |a|$

(Macro) nint(a) $\cdots$ round(a) as integer

- void c_qd_nint(const double * a, double * b)
- void c_qd_floor(const double * a, double * b) $\cdots$ $b :=$ floor(a)
- void c_qd_ceil(const double * a, double * b) $\cdots$ $b :=$ ceil(a)
- void c_qd_comp(const double * a, const double * b, int * result) $\cdots$ return -1 if $a < b$, 0 if $a == b$, and +1 if $a > b$.
- void c_qd_comp_qd_d(const double * a, double b, int * result)
- void c_qd_comp_d_qd(double a, const double * b, int * result)

2.5 Triple-double precision arithmetic

- void c_td_copy(const double * a, double * b) $\cdots$ $b := a$
- void c_qd_copy_td(const double * a, double * c)
- void c_td_copy_qd(const double * a, double * b)
- void c_td_copy_dd(const double * a, double * b)
- void c_td_copy_d(double a, double * b)
- void vec_sum(double *e, const double * x, int n) $\cdots$ $e[n] := vec_sum(x[n])$
- void vseb(double * y, int ny, const double * e, int ne) $\cdots$ $y[n] := vec_sum_err_branch(vseb)(k)(e[n])$
- void c_to_td(double * r, double a, double b, double c) $\cdots$ $r[3] := to_td(a, b, c)$
- void merge(double * c, double * a, int na, double * b, int nb) $\cdots$ Merge a[na] and b[nb] into c[na + nb]
- void c_td_add(double * a, double * b, double * c) $\cdots$ $c := a + b$
- void c_td_add_td_d(double * a, double b, double * c)
- void c_td_neg(const double * a, double * c) $\cdots$ $c := -a$
  //
- void c_td_sub(double * c, double * a, double * b) $\cdots$ $c := a - b$
- void c_td_sub(double * a, double * b, double * c)
- void c_td_subq(const double * a, const double * b, double * c)
- void c_td_sub_d_td(double a, double * b, double * c) //
- void c_td_sub_td_d(double * c, double * a, double b)
- void c_td_sub_td_d(double * a, double b, double * c)
- void c_td_mul_accurate(double * a, double * b, double * c) $\cdots$ $c := a \times b$
- void c_td_mul_sloppy(double * a, double * b, double * c) $\cdots$ not so accurate but fast $c := a \times b$

(Macro) USE_TD_MUL_ACCURATE() $\cdots$ Use accurate td multiplication if true

- void c_td_mul_dd_td_sloppy(double * a, double * b, double * c)
- void c_td_mul_dd_td_accurate(double * a, double * b, double * c)

(Macro) USE_TD_MUL_DD_TD_ACCURATE() $\cdots$ Use accurate td-dd multiplication if true

- void c_td_mul_d_td(const double a, const double * b, double * c)
- void c_td_mul_td_d(const double * a, const double b, double * c)
- void c_td_abs(const double * a, double * b) $\cdots$ $b := |a|$
- void c_td_comp(const double * a, const double * b, int * result) $\cdots$ return -1 if $a < b$, 0 if $a == b$, and $+1$ if $a > b$.
- void c_td_comp_td_d(const double * a, double b, int * result)
- void c_td_comp_d_td(double a, const double * b, int * result)
- void c_td_2mtw_dd_td(double a[DDSIZE], double b[TDSIZE], double c[TDSIZE])

(Macro) ONE_P_2DBL_EPS() $\cdots$ $1 + 2\times$ DBL_EPSILON $= (1.00000000000000044e + 00)$

(Macro) ONE_M_2DBL_EPS() $\cdots$ $1 - 2\times$ DBL_EPSILON $= (9.99999999999999556e - 01)$

- void c_td_reci(double * a, double * c)
- void c_td_divt(double * a, double * b, double * c)

(Macro) c_td_div() $\cdots$ the same as c_td_divtq

- void c_td_divq(const double * a, const double * b, double * c) $\cdots$ td division based on quad-double division.
- void c_td_div_td_d(double * a, double b, double * c)
- void c_td_sqrt(double * a, double * c) $\cdots$ $c := \sqrt{a}$
- void c_td_sqrt_d(double a, double * c)
- void c_td_sqr(double * a, double * c) $\cdots$ $c := a^2$

2.6 rdd.h

```
// ddfloat, tdfloat, qdfloat
typedef struct { double val[DDSIZE]; } ddfloat; // 53 * 2 = 106
typedef struct { double val[TDSIZE]; } tdfloat; // 53 * 3 = 159
typedef struct { double val[QDSIZE]; } qdfloat; // 53 * 4 = 212
```

(Macro) SET0_DD(val) $\cdots$ $val := 0$ val[0] = (double)0.0; val[1] = (double)0.0;

(Macro) SET0_TD(val)

(Macro) SET0_QD(val)

- void rdd_out_str_base(FILE * fp, int base, int length, double val[DDSIZE]) $\cdots$ DD print(no appending CR)
- int rdd_cmp(double a[DDSIZE], double b[DDSIZE]) $\cdots$ return -1 if $a < b$, 0 if $a == b$, and -1 if $a < b$.
- int rdd_cmp_d(double a[DDSIZE], double b)
- void rdd_sqrt_d(double ret[DDSIZE], double a) $\cdots$ return $\sqrt{ret}$
- void rdd_fma(double ret[DDSIZE], double a[DDSIZE], double b[DDSIZE], double c[DDSIZE]) $\cdots$ $ret := a \times b + c$
- void rdd_pow(double ret[DDSIZE], double base[DDSIZE], double power[DDSIZE]) $\cdots$ ret := $\text{base}^{\text{power}}$
- void rqd_out_str_base(FILE *fp, int base, int length, double val[QDSIZE]) $\cdots$ print(no appending CR)
- int rqd_cmp(double a[QDSIZE], double b[QDSIZE]) $\cdots$ return -1 if $a < b$, 0 if $a == b$, and -1 if $a < b$.
- int rqd_cmp_d(double a[QDSIZE], double b)
- void rqd_sqrt_d(double ret[QDSIZE], double a) $\cdots$ return $\sqrt{ret}$
- void rqd_fma(double ret[QDSIZE], double a[QDSIZE], double b[QDSIZE], double c[QDSIZE]) $\cdots$ $ret := a \times b + c$
- void rqd_pow(double ret[QDSIZE], double base[QDSIZE], double power[QDSIZE]) $\cdots$ ret := $\text{base}^{\text{power}}$
- void rtd_out_str_base(FILE *fp, int base, int length, double val[TDSIZE])
- int rtd_cmp(double a[TDSIZE], double b[TDSIZE]) $\cdots$ return -1 if $a < b$, 0 if $a == b$, and -1 if

$a < b$.

- int `rtd_cmp_d`(double `a[TDSIZE]`, double `b`)
- void `rtd_sqrt_d`(double ret[TDSIZE], double `a`) $\cdots$ return $\sqrt{ret}$
- void `rtd_fma`(double ret[TDSIZE], double `a[TDSIZE]`, double `b[TDSIZE]`, double c[TDSIZE]) $\cdots$ $ret := a \times b + c$
- void `rtd_pow`(double ret[TDSIZE], double `base[TDSIZE]`, double power[TDSIZE]) $\cdots$ ret := $\text{base}^{\text{power}}$

(Macro) `set0_dd`(val) $val := 0$
(Macro) `rdd_set0`(val)
(Macro) `rdd_add`(ret, a, b) $ret := a + b$
(Macro) `rdd_sub`(ret, a, b) $ret := a - b$
(Macro) `rdd_mul`(ret, a, b) $ret := a \times b$
(Macro) `rdd_div`(ret, a, b) $ret := a/b$
(Macro) `rdd_sqrt`(ret, a) $ret := \sqrt{a}$
(Macro) `rdd_sqrt_d`(ret, a)
(Macro) `rdd_sqrt_ui`(ret, a)
(Macro) `rdd_get_d`(a)
(Macro) `rdd_set_d`(ret, d) $ret := d$
(Macro) `rdd_set_ui`(ret, d)
(Macro) `rdd_set`(ret, d)
(Macro) `rdd_neg`(ret, a) $ret := -a$
(Macro) `rdd_abs`(ret, a) $ret := |a|$
(Macro) `rdd_cmp_ui`(a, b) return -1 if $a > b$, 0 if $a == b$, and -1 if $a < b$
(Macro) `rdd_ui_div`(ret, a, b) $ret := a/b$
(Macro) `rdd_ui_sub`(ret, a, b) $ret := a - b$
(Macro) `rdd_div_d`(ret, a, b) $ret := a/b$
(Macro) `rdd_add_d`(ret, a, b) $ret := a + b$
(Macro) `rdd_sub_d`(ret, a, b) $ret := a - b$
(Macro) `rdd_mul_d`(ret, a, b) $ret := a \times b$
(Macro) `rdd_div_ui`(ret, a, b) $ret := a/b$
(Macro) `rdd_add_ui`(ret, a, b) $ret := a + b$
(Macro) `rdd_sub_ui`(ret, a, b) $ret := a - b$
(Macro) `rdd_mul_ui`(ret, a, b) $ret := a \times b$
(Macro) `set0_td`(val) SET0_TD(val)
(Macro) `rtd_set0`(val) SET0_TD(val)
(Macro) `rtd_add`(ret, a, b) $ret := a + b$
(Macro) `rtd_addt`(ret, a, b)
(Macro) `rtd_addq`(ret, a, b)
(Macro) `rtd_sub`(ret, a, b) $ret := a - b$
(Macro) `rtd_subt`(ret, a, b) $ret := a - b$
(Macro) `rtd_subq`(ret, a, b) $ret := a - b$
(Macro) `rtd_mul`(ret, a, b) $ret := a \times b$
(Macro) `rtd_divt`(ret, a, b) $ret := a/b$
(Macro) `rtd_divtq`(ret, a, b)
(Macro) `rtd_divq`(ret, a, b)
(Macro) `rtd_div`(ret, a, b)
(Macro) `rtd_sqrt`(ret, a) $ret := \sqrt{a}$
(Macro) `rtd_sqrt_d`(ret, a)
(Macro) `rtd_sqrt_ui`(ret, a)
(Macro) `rtd_get_d`(a) return (double)a
(Macro) `rtd_set_d`(ret, d) $ret := d$
(Macro) `rtd_set_ui`(ret, d)

(Macro) `rtd_set`(ret, d)
(Macro) `rtd_neg`(ret, a) $ret := -a$
(Macro) `rtd_abs`(ret, a) $ret := |a|$
(Macro) `rtd_cmp_ui`(a, b) return -1 if $a > b$, 0 if $a == b$, and -1 if $a < b$
(Macro) `rtd_ui_div`(ret, a, b) $ret := a/b$
(Macro) `rtd_ui_sub`(ret, a, b) $ret := a - b$
(Macro) `rtd_div_d`(ret, a, b) $\cdots$ $c := a/b$
(Macro) `rtd_add_d`(ret, a, b) $\cdots$ $c := a + b$
(Macro) `rtd_sub_d`(ret, a, b) $\cdots$ $c := a - b$
(Macro) `rtd_mul_d`(ret, a, b) $\cdots$ $c := a \times b$
(Macro) `rtd_div_ui`(ret, a, b) $\cdots$ $c := a/b$
(Macro) `rtd_add_ui`(ret, a, b) $\cdots$ $c := a + b$
(Macro) `rtd_sub_ui`(ret, a, b) $\cdots$ $c := a - b$
(Macro) `rtd_mul_ui`(ret, a, b) $\cdots$ $c := a \times b$
(Macro) `set0_qd`(val) $\cdots$ $val := 0$
(Macro) `rqd_set0`(val)
(Macro) `rqd_add`(ret, a, b) $\cdots$ $c := a + b$
(Macro) `rqd_sub`(ret, a, b) $\cdots$ $c := a - b$
(Macro) `rqd_mul`(ret, a, b) $\cdots$ $c := a \times b$
(Macro) `rqd_div`(ret, a, b) $\cdots$ $c := a/b$
(Macro) `rqd_sqrt`(ret, a) $\cdots$ $ret := \sqrt{a}$
(Macro) `rqd_sqrt_d`(ret, a) $ret := \sqrt{a}$
(Macro) `rqd_sqrt_ui`(ret, a)
(Macro) `rqd_get_d`(a) return (double)a
(Macro) `rqd_set_d`(ret, d) $ret := d$
(Macro) `rqd_set_ui`(ret, d)
(Macro) `rqd_set`(ret, d)
(Macro) `rqd_neg`(ret, a) $ret := -a$
(Macro) `rqd_abs`(ret, a) $ret := |a|$
(Macro) `rqd_cmp_ui`(a, b) return -1 if $a > b$, 0 if $a == b$, and -1 if $a < b$
(Macro) `rqd_ui_div`(ret, a, b) $ret := a/b$
(Macro) `rqd_ui_sub`(ret, a, b) $ret := a - b$
(Macro) `rqd_div_d`(ret, a, b) $\cdots$ $c := a/b$
(Macro) `rqd_add_d`(ret, a, b) $\cdots$ $c := a + b$
(Macro) `rqd_sub_d`(ret, a, b) $\cdots$ $c := a - b$
(Macro) `rqd_mul_d`(ret, a, b) $\cdots$ $c := a \times b$
(Macro) `rqd_div_ui`(ret, a, b) $\cdots$ $c := a/b$
(Macro) `rqd_add_ui`(ret, a, b) $\cdots$ $c := a + b$
(Macro) `rqd_sub_ui`(ret, a, b) $\cdots$ $c := a - b$
(Macro) `rqd_mul_ui`(ret, a, b) $\cdots$ $c := a \times b$

2.7 AVX2

2.7.1 float

- `__m256 _bncavx2_fabsf(__m256 val8)` $\cdots$ $\text{val8} := \max \text{val8}, 0 - \text{val8}$
- `__m256 _bncavx2_fneg(__m256 a)` $\cdots$ return $-a$
- `void _bncavx2_ffma(float ret[], float a[], float b[], float c[], int dim)` $\cdots$ $\text{ret} := a \times b + c$
- `void _bncavx2_fmul(float ret[], float a[], float b[], int dim)` $\cdots$ $\text{ret} := a \times b$

2.7.2 double

- __m256d _bncavx2_fabs(__m256d val4) $\cdots$ val4 := $\max$ val4, $-$val4
- __m256d _bncavx2_dneg(__m256d a) $\cdots$ $-a$
- void _bncavx2_dfma(double ret[], double a[], double b[], double c[], int dim) $\cdots$ ret := $a \times b + c$
- void _bncavx2_dmul(double ret[], double a[], double b[], int dim) $\cdots$ ret := $a \times b$
- void _bncavx2_ddiv(double ret[], double a[], double b[], int dim) $\cdots$ ret := a/b
- void _bncavx2_dadd(double ret[], double a[], double b[], int dim) $\cdots$ ret := $a + b$
- void _bncavx2_dsub(double ret[], double a[], double b[], int dim) $\cdots$ ret := $a - b$
- double _bncavx2_ddotp(double a[], double b[], int dim) $\cdots$ ret := $\sum_{i=0}^{\dim-1} a[i] \times b[i]$

2.7.3 Error-free transformation

- __m256d _bncavx2_dquick_two_sum(__m256d a, __m256d b, __m256d * err) $\cdots$ Computes $\mathrm{fl}(a+b)$ and $\mathrm{err}(a+b)$. Assume $|a| >\geq |b|$.
- __m256d _bncavx2_dquick_two_diff(__m256d a, __m256d b, __m256d * err) $\cdots$ Computes $\mathrm{fl}(a-b)$ and $\mathrm{err}(a-b)$. Assume $|a| \geq |b|$.
- __m256d _bncavx2_dtwo_sum(__m256d a, __m256d b, __m256d * err) $\cdots$ Computes $\mathrm{fl}(a+b)$ and $\mathrm{err}(a+b)$.
- __m256d _bncavx2_dtwo_diff(__m256d a, __m256d b, __m256d * err) $\cdots$ Computes $\mathrm{fl}(a-b)$ and $\mathrm{err}(a-b)$.
- __m256d _bncavx2_dtwo_prod(__m256d a, __m256d b, __m256d * err) $\cdots$ Computes $\mathrm{fl}(a \times b)$ and $\mathrm{err}(a \times b)$.

2.7.4 Double-double precision arithmetic

- void _bncavx2_set0_dd(__m256d) ret[DDSIZE] $\cdots$ ret := 0

(Macro) _bncavx2_rdd_set0(ret) $\cdots$ is the same as _bncavx2_set0_dd(ret)

- void _bncavx2_get_dd_m256d_i(ddfloat *ret, __m256d ret4[DDSIZE], int avx_index) $\cdots$ ret := ret4[][avx_index]
- void _bncavx2_rdd_sum256d(double ret[DDSIZE], __m256d ret4[DDSIZE]) $\cdots$ ret := ret4[0] + ret4[1] + ret4[2] + ret4[3]
- void _bncavx2_rdd_abssum256d(double ret[DDSIZE], __m256d ret4[DDSIZE]) $\cdots$ ret := $|\mathrm{ret4}[0]| + |\mathrm{ret4}[1]| + |\mathrm{ret4}[2]| + |\mathrm{ret4}[3]|$
- void _bncavx2_rdd_absmax256d(double ret[DDSIZE], __m256d ret4[DDSIZE]) $\cdots$ ret := $\max(|\mathrm{ret4}[0]|, |\mathrm{ret4}[1]|, |\mathrm{ret4}[2]|, |\mathrm{ret4}[3]|)$
- void _bncavx2_rdd_norm256d(double ret[DDSIZE], __m256d ret4[DDSIZE]) $\cdots$ ret := $\|\mathrm{ret4}[0]^2 + \mathrm{ret4}[1]^2 + \mathrm{ret4}[2]^2 + \mathrm{ret4}[3]^2\|_2$
- void _bncavx2_rdd_add(__m256d ret[DDSIZE], __m256d a[DDSIZE], __m256d b[DDSIZE]) $\cdots$ ret := $a + b$
- void _bncavx2_rdd_sub(__m256d ret[DDSIZE], __m256d a[DDSIZE], __m256d b[DDSIZE]) $\cdots$ ret := $a - b$
- void _bncavx2_rdd_mul(__m256d ret[DDSIZE], __m256d a[DDSIZE], __m256d b[DDSIZE]) $\cdots$ ret := $a \times b$
- void _bncavx2_rdd_div(__m256d ret[DDSIZE], __m256d a[DDSIZE], __m256d b[DDSIZE]) $\cdots$ ret := a/b
- void _bncavx2_rdd_abs(__m256d ret[DDSIZE], __m256d a[DDSIZE]) $\cdots$ ret := $|a|$

2.7.5 Triple-double precision arithmetic

- void _bncavx2_set0_td(__m256d ret[TDSIZE]) $\cdots$ ret := 0
- void _bncavx2_get_td_m256d_i(tdfloat * ret, __m256d ret4[TDSIZE], int avx_index) $\cdots$ ret := ret4[][avx_index]
- void _bncavx2_rtd_sum256d(double ret[TDSIZE], __m256d ret4[TDSIZE]) $\cdots$ ret := ret4[0] + ret4[1] + ret4[2]
- void _bncavx2_rtd_abs(__m256d ret[TDSIZE], __m256d a[TDSIZE]) $\cdots$ ret := $|a|$
- void _bncavx2_rtd_abssum256d(double ret[TDSIZE], __m256d ret4[TDSIZE]) $\cdots$ ret := |ret4[0]| + |ret4[1]| + |ret4[2]| + |ret4[3]|
- void _bncavx2_rtd_absmax256d(double ret[TDSIZE], __m256d ret4[TDSIZE]) $\cdots$ ret := max(|ret4[0]|, |ret4[1]|, |ret4[2]|, |ret4[3]|)
- void _bncavx2_rtd_norm256d(double ret[TDSIZE], __m256d ret4[TDSIZE])
- void _bncavx2_vec_sum(__m256d e[], const __m256d x[], int n) $\cdots$ e := vec_sum(x)
- void _bncavx2_vseb(__m256d y[], int ny, const __m256d e[], int ne) $\cdots$ y := vec_sum_err_branch(vseb(k))(e)
- void _bncavx2_merge(__m256d c[], __m256d a[], int na, __m256d b[], int nb) $\cdots$ Merge a[na] & b[nb] into c[na + nb]

(Macro) _bncavx2_rtd_add() _bncavx2_rtd_addq

- void _bncavx2_rtd_addq(__m256d ret[TDSIZE], __m256d a[TDSIZE], __m256d b[TDSIZE]) $\cdots$ ret := $a + b$
- void _bncavx2_rtd_addt(__m256d ret[TDSIZE], __m256d a[TDSIZE], __m256d b[TDSIZE])
- void _bncavx2_rtd_mulq(__m256d ret[TDSIZE], __m256d a[TDSIZE], __m256d b[TDSIZE]) $\cdots$ ret := $a \times b$
- void _bncavx2_rtd_mul(__m256d ret[TDSIZE], __m256d a[TDSIZE], __m256d b[TDSIZE])
- void _bncavx2_rtd_neg(__m256d c[TDSIZE], __m256d a[TDSIZE]) $\cdots$ $c := -a$
- void _bncavx2_rtd_sub(__m256d c[TDSIZE], __m256d a[TDSIZE], __m256d b[TDSIZE]) $\cdots$ $c := a - b$
- void _bncavx2_rtd_subq(__m256d c[TDSIZE], __m256d a[TDSIZE], __m256d b[TDSIZE])
- void _bncavx2_rtd_divq(__m256d ret[TDSIZE], __m256d a[TDSIZE], __m256d b[TDSIZE]) $\cdots$ $c := a/b$
- void _bncavx2_to_td(__m256d r[TDSIZE], __m256d a, __m256d b, __m256d c) $\cdots$ $r :=$ TD(a, b, c)
- void _bncavx2_rtd_mul_d(__m256d c[TDSIZE], __m256d a, __m256d b[TDSIZE]) $\cdots$ $c := a \times b$
- void _bncavx2_rtd_mul_dd(__m256d c[TDSIZE], __m256d a[DDSIZE], __m256d b[TDSIZE]) $\cdots$ $c := a \times b$

(Macro) _bncavx2_rtd_div()$\cdots$ _bncavx2_rtd_divtq

- void _bncavx2_rtd_divt(__m256d ret[TDSIZE], __m256d a[TDSIZE], __m256d b[TDSIZE]) $\cdots$ $c := a/sb$
- void _bncavx2_rtd_divtq(__m256d ret[TDSIZE], __m256d a[TDSIZE], __m256d b[TDSIZE])

2.7.6 Quadruple-double precision arithmetic

- void _bncavx2_set0_qd(__m256d ret[QDSIZE]) $\cdots$ ret := 0
- void _bncavx2_get_qd_m256d_i(qdfloat * ret, __m256d ret4[QDSIZE], int avx_index) ret := ret4[][avx_index]
- void _bncavx2_rqd_sum256d(double ret[QDSIZE], __m256d ret4[QDSIZE]) $\cdots$ ret := mmval[0] + ... + mmval[7]
  void_bncavx2_rqd_abs__m256d ret[QDSIZE], __m256d a[QDSIZE] $\cdots$ ret := |a|
- void _bncavx2_rqd_abssum256d(double ret[QDSIZE], __m256d ret4[QDSIZE]) $\cdots$ ret := |ret4[0]| + |ret4[1]| + |ret4[2]| + |ret4[3]|
- void _bncavx2_rqd_absmax256d(double ret[QDSIZE], __m256d ret4[QDSIZE]) $\cdots$ ret :=

max(|ret4[0]|, |ret4[1]|, |ret4[2]|, |ret4[3]|)

- void _bncavx2_rqd_norm256d(double ret[QDSIZE], __m256d ret4[QDSIZE]) $\cdots$ ret := $\|\text{ret4}[0]^2 + \text{ret4}[1]^2 + \text{ret4}[2]^2 + \text{ret4}[3]^2\|_2$
- void _bncavx2_renorm(__m256d * c0, __m256d * c1, __m256d * c2, __m256d* c3) $\cdots$ renorm(double *c0, double *c1, double *c2, double *c3)
- void _bncavx2_renorm4(__m256d * c0, __m256d * c1, __m256d * c2, __m256d* c3, __m256d * c4) $\cdots$ renorm4(double *c0, double *c1, double *c2, double *c3, double *c4)
- void _bncavx2_three_sum(__m256d * a, __m256d * b, __m256d * c) $\cdots$ three_sum(double *a, double *b, double *c)
- void _bncavx2_three_sum2(__m256d * a, __m256d * b, __m256d * c) $\cdots$ void three_sum2(double *a, double *b, double *c)
- void _bncavx2_rqd_add(__m256d ret[QDSIZE], __m256d a[QDSIZE], __m256d b[QDSIZE]) $\cdots$ ret := $a + b$
- void _bncavx2_rtd_addq(__m256d ret[TDSIZE], __m256d a[TDSIZE], __m256d b[TDSIZE]) $\cdots$ ret := $a + b$
- void _bncavx2_rtd_mulq(__m256d ret[TDSIZE], __m256d a[TDSIZE], __m256d b[TDSIZE]) $\cdots$ ret := $a \times b$
- void _bncavx2_rqd_mul(__m256d ret[QDSIZE], __m256d a[QDSIZE], __m256d b[QDSIZE]) $\cdots$ ret := $a \times b$
- void _bncavx2_rqd_sub(__m256d c[QDSIZE], __m256d a[QDSIZE], __m256d b[QDSIZE]) $\cdots$ ret := $a - b$
- void _bncavx2_rqd_mul_d(__m256d c[QDSIZE], const __m256d a[QDSIZE], __m256d b) $\cdots$ ret := $a \times b$
- void _bncavx2_rtd_divq(__m256d c[TDSIZE], __m256d a[TDSIZE], __m256d b[TDSIZE]) $\cdots$ $c := a/b$ in the manner of quad-double division
- void _bncavx2_rqd_div(__m256d c[QDSIZE], __m256d a[QDSIZE], __m256d b[QDSIZE]) $\cdots$ $c := a/b$

2.8 Miscellaneous functions

The following functions are defined in bnc_common.h, which are coded in bnc_common_func.c and algebraic_eq.c.

2.8.1 common functions

- void bnc_print_env_all(void) $\cdots$ Print all environmental variables of bncmatmul library.
- double mylog2(double x) $\cdots$ Return $\log_2(x)$.
- double matmul_gflops(double comp_sec, int dim) $\cdots$ Return giga flops when calculating a dim-dimensional dense square matrix product from comp_sec (computational time in second).
- int byte_double_sqmat(int dim) $\cdots$ Return the number of bytes to store a dim-dimensional square matrix.
- void set_bnc_default_prec(unsigned long precision)
- void set_bnc_default_prec_decimal(unsigned long precision) $\cdots$ Set the default precsision in bit or in decimal digit.
- mp_rnd_t get_bnc_default_rounding_mode(void) $\cdots$ Return the current rounding mode.
- void set_bnc_rounding_mode(mp_rnd_t rounding_mode) $\cdots$ Set rounding_mode as default rounding mode.
- unsigned long int get_bnc_default_prec(void) $\cdots$ Return the current default precision in bit.
- void mpf_fma(mpf_t rop, mpf_t op1, mpf_t op2, mpf_t op3) $\cdots$ rop := op1 $\times$ op2 + op3
- void mpf_log10(mpf_t ret, mpf_t x) $\cdots$ $\log_{10}(x)$
- void mpf_exp(mpf_t ret, mpf_t x) $\cdots$ $\exp(x)$
- void mpf_log(mpf_t ret, mpf_t x) $\cdots$ $\log_e(x)$

- void `mpf_sin`(mpf_t `ret`, mpf_t `x`) $\cdots$ $\sin(x)$
- void `void`(mpf_cos) mpf_t `ret`, mpf_t `x` $\cdots$ $\cos(x)$
- void `mpf_tan`(mpf_t `ret`, mpf_t `x`) $\cdots$ $\tan(x)$
- void `mpf_atan`(mpf_t `ret`, mpf_t `x`) $\cdots$ $\arctan(x)$

2.8.2 Random functions

- void `rdd_srand`(unsigned long `seed`)
- void `rtd_srand`(unsigned long `seed`)
- void `rqd_srand`(unsigned long `seed`)
- void `mpf_srand`(unsigned long `seed`) $\cdots$ Set random seed and initialize the current random state for each precision.
- void `rdd_urand`(double `ret[DDSIZE]`)
- void `rtd_urand`(double `ret[TDSIZE]`)
- void `rqd_urand`(double `ret[QDSIZE]`)
- void `mpf_urand`(mpf_t `ret`) $\cdots$ Return a random number on $[0, 1]$.
- void `rdd_nrand`(double `ret[DDSIZE]`)
- void `rtd_nrand`(double `ret[TDSIZE]`)
- void `rqd_nrand`(double `ret[QDSIZE]`)
- void `mpf_nrand`(mpf_t `ret`) $\cdots$ Return a random number following standard normal distribution $[-1, 1]$.

2.8.3 Solver for algebraic equation

- int `dquadratic_eq`(double `ans_re[2]`, double `ans_im[2]`, double `coef[3]`)
- int `mpfquadratic_eq`(mpf_t `ans_re[2]`, mpf_t `ans_im[2]`, mpf_t `coef[3]`) $\cdots$ Solve coef[2]$\cdot x^2 +$ coef[1]$\cdot x+$coef[0] $= 0$ and store the roots as $x =$ ans_re[0]+ans_im[0]$\cdot$i and ans_re[1]+ans_im[1]$\cdot$i

2.9 Tool functions for benchmarking

There some useful functions for benchmarking in BNCmatmul.

2.9.1 Time estimation

- float `fget_sec`(int `flag`)
- double `get_sec`(int `flag`) $\cdots$ Return the current real, user, system time as double prec. in second if flag $\neq 0$, and the sum of the current user and system time in second if flag $= 0$.
- double `get_secv`(void)
- float `fget_secv`(void) $\cdots$ The same as `[f]get_sec(0)`.
- double `get_real_sec`(int `flag`)
- float `fget_real_sec`(int `flag`) $\cdots$ Return the current tv_sec (in second) and tv_usec (in microsecond) as double prec. in second if flag $\neq 0$, and the sum of tv_sec + tv_usec/10^6.
- double `get_real_secv`(void)
- float `fget_real_secv`(void) $\cdots$ The same as `[f]get_real_sec(0)`.

2.9.2 Relative error ofvector and matrix

■Relative error ofvector

- void `relerr3_dvector`(double * `max_relerr`, double * `min_relerr`, double * `norm_relerr`, DVector `vec`, DVector `vec_true`, int `kind_of_norm`)

- void relerr3_ddvector(double max_relerr[DDSIZE], double min_relerr[DDSIZE], double norm_relerr[DDSIZE], DDVector vec, DDVector vec_true, int kind_of_norm)
- void relerr3_tdvector(double max_relerr[TDSIZE], double min_relerr[TDSIZE], double norm_relerr[TDSIZE], TDVector vec, TDVectorvec_true, int kind_of_norm)
- void relerr3_tdvector_qdvec(double max_relerr[TDSIZE], double min_relerr[TDSIZE], double norm_relerr[TDSIZE], TDVector vec, QDVectorvec_true, int kind_of_norm)
- void relerr3_qdvector(double max_relerr[QDSIZE], double min_relerr[QDSIZE], double norm_relerr[QDSIZE], QDVector vec, QDVector vec_true, int kind_of_norm)
- void relerr3_cddvector(double max_relerr[DDSIZE], double min_relerr[DDSIZE], double max_real_relerr[DDSIZE], double min_real_relerr[DDSIZE], double max_image_relerr[DDSIZE], double min_image_relerr[DDSIZE], double norm_relerr[DDSIZE], CDDVector vec, CDDVector vec_true, int kind_of_norm)
- void relerr3_ctdvector(double max_relerr[TDSIZE], double min_relerr[TDSIZE], double max_real_relerr[TDSIZE], double min_real_relerr[TDSIZE], double max_image_relerr[TDSIZE], double min_image_relerr[TDSIZE], double norm_relerr[TDSIZE], CTDVector vec, CTDVector vec_true, int kind_of_norm)
- void relerr3_cqdvector(double max_relerr[QDSIZE], double min_relerr[QDSIZE], double max_real_relerr[QDSIZE], double min_real_relerr[QDSIZE], double max_image_relerr[QDSIZE], double min_image_relerr[QDSIZE], double norm_relerr[QDSIZE], CQDVector vec, CQDVector vec_true, int kind_of_norm)
- void relerr3_mpfvector(mpf_t max_relerr, mpf_t min_relerr, mpf_t norm_relerr, MPFVector vec, MPFVector vec_true, int kind_of_norm)
- void relerr3_cmpfvector(mpf_t max_abs_relerr, mpf_t min_abs_relerr, mpf_t max_real_relerr, mpf_t min_real_relerr, mpf_t max_image_relerr, mpf_t min_image_relerr, mpf_t norm_relerr, CMPFVector vec, CMPFVector vec_true, int kind_of_norm)

■Relative error of matrix

kind_of_norm $= 0$ $\|A\|_\infty$
kind_of_norm $= 1$ $\|A\|_1$
kind_of_norm $= 2$ $\|A\|_F$

- void relerr3_dmatrix(double * max_relerr, double * min_relerr, double * norm_relerr, DMatrix mat, DMatrix mat_true, int kind_of_norm)
- void relerr3_ddmatrix(double max_relerr[DDSIZE], double min_relerr[DDSIZE], double norm_relerr[DDSIZE], DDMatrix mat, DDMatrix mat_true, int kind_of_norm)
- void relerr3_tdmatrix(double max_relerr[TDSIZE], double min_relerr[TDSIZE], double norm_relerr[TDSIZE], TDMatrix mat, TDMatrix mat_true, int kind_of_norm)
- void relerr3_qdmatrix(double max_relerr[QDSIZE], double min_relerr[QDSIZE], double norm_relerr[QDSIZE], QDMatrix mat, QDMatrix mat_true, int kind_of_norm)
- void relerr3_mpfmatrix(mpf_t max_relerr, mpf_t min_relerr, mpf_t norm_relerr, MPFMatrix mat, MPFMatrix mat_true, int kind_of_norm)
- void relerr3_cddmatrix(double max_relerr[DDSIZE], double min_relerr[DDSIZE], double max_real_relerr[DDSIZE], double min_real_relerr[DDSIZE], double max_image_relerr[DDSIZE], double min_image_relerr[DDSIZE], double norm_relerr[DDSIZE], CDDMatrix mat, CDDMatrix mat_true, int kind_of_norm)
- void relerr3_ctdmatrix(double max_relerr[TDSIZE], double min_relerr[TDSIZE], double max_real_relerr[TDSIZE], double min_real_relerr[TDSIZE], double max_image_relerr[TDSIZE], double min_image_relerr[TDSIZE], double norm_relerr[TDSIZE], CTDMatrix mat, CTDMatrix mat_true, int kind_of_norm)
- void relerr3_cqdmatrix(double max_relerr[QDSIZE], double min_relerr[QDSIZE], double max_real_relerr[QDSIZE], double min_real_relerr[QDSIZE], double max_image_relerr[QDSIZE], double min_image_relerr[QDSIZE], double norm_relerr[QDSIZE], CQDMatrix mat, CQDMatrix mat_true, int kind_of_norm)

- void `relerr3_cmpfmatrix`(mpf_t `max_abs_relerr`, mpf_t `min_abs_relerr`, mpf_t `max_real_relerr`, mpf_t `min_real_relerr`, mpf_t `max_image_relerr`, mpf_t `min_image_relerr`, mpf_t `norm_relerr`, CMPFMatrix `mat`, CMPFMatrix `mat_true`, int `kind_of_norm`)

Chapter 3

Basic linear computation

3.1 Detatypes of D, DD, TD, QD and MPFR vector and matrix

Vector datatypes of DD, TD, QD and MPFR precision are shown as follows:

DDVector, TDVector, QDVector

```
typedef struct
{
    long int dim; // dim <= real_dim
    long int real_dim; // multiplier of _BNC_D_WIDTH
    double *element[DDSIZE]; // [TDSIZE] and [QDSIZE] used as TDVector and
        ↪ QDVector
} ddvector;

typedef *DDVector; // DDVector, TDVector, QDVector are pointers.
```

MPFVector

```
typedef struct{
    unsigned long int prec; // default precision of element
    mpf_t *element;
    long int dim;
    long int real_dim;
} mpfvector;

typedef mpfvector *MPFVector; // MPFVector is a pointer.
```

Matrix datatypes of DD, TD, QD and MPFR precision are shown as follows:

DDMatrix, TDMatrix, QDMatrix

```
// DD matrix
typedef struct{
    long int row_dim, col_dim;
    long int real_row_dim, real_col_dim; // multiplier of _BNC_D_WIDTH
    double *element[DDSIZE];
} ddmatrix;

typedef *DDMatrix;
```

MPFMatrix

```
typedef struct{
    unsigned long int prec;
    mpf_t *element;
    long int row_dim, col_dim;
    long int real_row_dim, real_col_dim;
    void *element_block; // mantissa block
} mpfmatrix;

typedef mpfmatrix *MPFMatrix;
```

3.2 Vector arithmetic

- ddfloat `get_ddvector_i_ddfloat`(DDVector `vec, long int index`) ··· Get index-th element of vec as ddfloat datatype.
- tdfloat `get_tdvector_i_tdfloat`(TDVector `vec, long int index`)
- qdfloat `get_qdvector_i_qdfloat`(QDVector `vec, long int index`)

(Macro) `GET_DDVECTOR_I`(vec, index) ··· Get index-th element as pointer to ddfloat.
(Macro) `get_ddvector_i`(vec, index)
(Macro) `GET_TDVECTOR_I`(vec, index) ··· Get index-th element as pointer to tdfloat.
(Macro) `get_tdvector_i`(vec, index)
(Macro) `GET_QDVECTOR_I`(vec, index) ··· Get index-th element as pointer to qdfloat.
(Macro) `get_qdvector_i`(vec, index)
(Macro) `GET_MPFVECTOR_I`(vec, index) ··· Get index-th element as pointer to mpf_t (mpfr_t).
(Macro) `get_mpfvector_i`(vec, index)

- void `set_ddvector_i`(DDVector `vec`, long int `index`, double * `val`) ··· Store val to index-th element of vec.

(Macro) `SET_DDVECTOR_I`(vec, index, val)

- void `set_tdvector_i`(TDVector `vec`, long int `index`, double * `val`)

(Macro) `SET_TDVECTOR_I`(vec, index, val)

- void `set_qdvector_i`(QDVector `vec`, long int `index`, double * `val`)

(Macro) `SET_QDVECTOR_I`(vec, index, val)

- void `set_qdvector_i`(QDVector `vec`, long int `index`, double * `val`)

(Macro) `SET_QDVECTOR_I`(vec, index, val)

- void `set_ddvector_i_d`(DDVector `vec`, long int `index`, double `val`) ··· Store val to index-th element of vec as double.

(Macro) `SET_DDVECTOR_I_D`(vec, index, val)

- void `set_tdvector_i_d`(TDVector `vec`, long int `index`, double `val`)

(Macro) `SET_TDVECTOR_I_D`(vec, index, val)

- void `set_qdvector_i_d`(QDVector `vec`, long int `index`, double `val`)

(Macro) `SET_QDVECTOR_I_D`(vec, index, val)

- void `set0_ddvector_i`(DDVector `vec`, long int `index`) ··· Store zero to index-th element of vec.

(Macro) `SET0_DDVECTOR_I`(vec, index)

- void `set0_tdvector_i`(TDVector `vec`, long int `index`)

(Macro) `SET0_TDVECTOR_I`(vec, index)

- void `set0_qdvector_i`(QDVector `vec`, long int `index`)

(Macro) `SET0_QDVECTOR_I`(vec, index)

- DDVector `init_ddvector`(in `dimension`) ··· Allocate and initialize a vector and set it as zero vector.
- TDVector `init_tdvector`(in `dimension`)
- QDVector `init_qdvector`(in `dimension`)
- MPFVector `init_mpfvector`(in `dimension`) ··· Allocate and initialize a MPFVector in default

precision in bits with bnc_set_default_prec and set it as zero vector.

- MPFVector init2_mpfvector(int dimension, unsigned long prec) $\cdots$ Initialize a MPFVector in prec bits and set it as zero vector.
- void free_ddvector(DDVector vec) $\cdots$ Free vec.
- void free_tdvector(TDVector vec)
- void free_qdvector(QDVector vec)
- void free_mpfvector(MPFVector vec)
- void set_ddfloat_ddvec(ddfloat ret[], int ret_dim, DDVector vec) $\cdots$ Convert vec to ddfloat array.
- void set_tdfloat_tdvec(tdfloat ret[], int ret_dim, TDVector vec) $\cdots$ Convert vec to tdfloat array.
- void set_qdfloat_qdvec(qdfloat ret[], int ret_dim, QDVector vec) $\cdots$ Convert vec to qdfloat array.
- void set_ddvector_ddfloat(DDVector ret, ddfloat array[], int array_dim) $\cdots$ Convert ddfloat array to DDVector ret.
- void set_tdvector_tdfloat(TDVector ret, tdfloat array[], int array_dim) $\cdots$ Convert tdfloat array to TDVector ret.
- void set_qdvector_qdfloat(QDVector ret, qdfloat array[], int array_dim) $\cdots$ Convert qdfloat array to QDVector ret.
- void print_ddvector(DDVector vec) $\cdots$ Print vec.
- void print_tdvector(TDVector vec)
- void print_qdvector(QDVector vec)
- void print_mpfvector(MPFVector vec)
- void set0_ddvector(DDVector vec) $\cdots$ Set vec as zero vector.
- void set0_tdvector(TDVector vec)
- void set0_qdvector(QDVector vec)
- void set0_mpfvector(MPFVector vec)
- void set_ddvector_i_str(DDVector vec, long int index, const char * str) $\cdots$ Set *str expressed in decimal to index-th element of vec.
- void set_tdvector_i_str(TDVector vec, long int index, const char * str)
- void set_qdvector_i_str(QDVector vec, long int index, const char * str)
- void set_mpfvector_i_str(MPFVector vec, long int index, const char * str)
- void add_ddvector(DDVector c, DDVector a, DDVector b) $\cdots$ $c := a + b$
- void add_tdvector(TDVector c, TDVector a, TDVector b)
- void add_qdvector(QDVector c, QDVector a, QDVector b)
- void add_mpfvector(MPFVector c, MPFVector a, MPFVector b)
- void add2_ddvector(DDVector c, DDVector a) $cdots$ $c := c + a$
- void add2_tdvector(TDVector c, TDVector a)
- void add2_qdvector(TDVector c, QDVector a)
- void add2_mpfvector(MPFVector c, MPFVector a)
- void sub_ddvector(DDVector c, DDVector a, DDVector b) $\cdots$ $c := a - b$
- void sub_tdvector(TDVector c, TDVector a, TDVector b)
- void sub_qdvector(QDVector c, QDVector a, QDVector b)
- void sub_mpfvector(MPFVector c, MPFVector a, MPFVector b)
- void sub2_ddvector(DDVector c, DDVector a) $\cdots$ $c- = a$
- void sub2_tdvector(TDVector c, TDVector a)
- void sub2_qdvector(QDVector c, QDVector a)
- void sub2_mpfvector(MPFVector c, MPFVector a)
- void cmul_ddvector(DDVector c, double val[DDSIZE], DDVector a) $\cdots$ $c := \mathrm{val} \times a$
- void cmul_tdvector(TDVector c, double val[TDSIZE], TDVector a)
- void cmul_qdvector(QDVector c, double val[QDSIZE], QDVector a)
- void cmul_mpfvector(MPFVector c, mpf_t val, MPFVector a)
- void cmul2_ddvector(DDVector c, double val[DDSIZE]) $\cdots$ $c := val \times c$

- void cmul2_ddvector(TDVector c, double val[TDSIZE])
- void cmul2_ddvector(QDVector c, double val[QDSIZE])
- void cmul2_ddvector(MPFVector c, mpf_t val)
- void add_cmul_ddvector(DDVector c, DDVector a, double val[DDSIZE], DDVector b) $\cdots$ $c := a + val * b$
- void add_cmul_tdvector(TDVector c, TDVector a, double val[TDSIZE], TDVector b)
- void add_cmul_qdvector(QDVector c, QDVector a, double val[QDSIZE], QDVector b)
- void add_cmul_mpfvector(MPFVector c, MPFVector a, mpf_t val, MPFVector b)
- void ip_ddvector(double ret[DDSIZE], DDVector a, DDVector b) $\cdots$ Calculate the dot product of a and b
- void ip_tdvector(double ret[TDSIZE], TDVector a, TDVector b)
- void ip_qdvector(double ret[QDSIZE], QDVector a, QDVector b)
- void ip_mpfvector(mpf_t ret, MPFVector a, MPFVector b)
- void neg_ddvector(DDVector c, DDVector a) $\cdots$ $c := -a$
- void neg_tdvector(TDVector c, TDVector a)
- void neg_qdvector(QDVector c, QDVector a)
- void neg_mpfvector(MPFVector c, MPFVector a)
- void norm1_ddvector(double ret[DDSIZE], DDVector a) $\cdots$ $\|a\|_1$
- void norm1_tdvector(double ret[TDSIZE], TDVector a)
- void norm1_qdvector(double ret[QDSIZE], QDVector a)
- void norm1_mpfvector(mpf_t ret, MPFVector a)
- void normi_ddvector(double ret[DDSIZE], DDVector a) $\cdots$ $\|a\|_\infty$
- void normi_tdvector(double ret[TDSIZE], TDVector a)
- void normi_qdvector(double ret[QDSIZE], QDVector a)
- void normi_mpfvector(mpf_t ret, MPFVector a)
- void norm2_ddvector(double ret[DDSIZE], DDVector vec) $\cdots$ $\|a\|_2$
- void norm2_tdvector(double ret[TDSIZE], TDVector vec)
- void norm2_qdvector(double ret[QDSIZE], QDVector vec)
- void norm2_mpfvector(mpf_t ret, MPFVector vec)

3.3 Matrix arithmetic

- ddfloat get_ddmatrix_ij_ddfloat(DDMatrix mat, long int i, long int j) $\cdots$ Get the (i, j)-th element of mat.

(Macro) GET_DDMATRIX_IJ(mat, i, j) ((get_ddmatrix_ij_ddfloat((mat), (i), (j)).val))
(Macro) get_ddmatrix_ij(mat, i, j) ((get_ddmatrix_ij_ddfloat((mat), (i), (j)).val))

- tdfloat get_tdmatrix_ij_tdfloat(TDMatrix mat, long int i, long int j)

(Macro) GET_TDMATRIX_IJ(mat, i, j) ((get_tdmatrix_ij_tdfloat((mat), (i), (j)).val))
(Macro) get_tdmatrix_ij(mat, i, j) ((get_tdmatrix_ij_tdfloat((mat), (i), (j)).val))

- qdfloat get_qdmatrix_ij_qdfloat(QDMatrix mat, long int i, long int j)

(Macro) GET_QDMATRIX_IJ(mat, i, j) ((get_qdmatrix_ij_qdfloat((mat), (i), (j)).val))
(Macro) get_qdmatrix_ij(mat, i, j) ((get_qdmatrix_ij_qdfloat((mat), (i), (j)).val))
(Macro) GET_MPFMATRIX_IJ(mat, i, j) ((get_mpfmatrix_ij((mat), (i), (j))))

- mpf_ptr get_mpfmatrix_ij(MPFMatrix mat, long int i, long int j)
- void set_ddmatrix_ij(DDMatrix mat, long int i, long int j, double * val)

(Macro) SET_DDMATRIX_IJ(mat, i, j, val) set_ddmatrix_ij((mat), (i), (j), (val))

- void set_tdmatrix_ij(TDMatrix mat, long int i, long int j, double * val)

(Macro) SET_TDMATRIX_IJ(mat, i, j, val) set_tdmatrix_ij((mat), (i), (j), (val))

- void set_qdmatrix_ij(QDMatrix mat, long int i, long int j, double * val)

(Macro) SET_QDMATRIX_IJ(mat, i, j, val) set_qdmatrix_ij((mat), (i), (j), (val))

- void set_mpfmatrix_ij(MPFMatrix mat, long int i, long int j, mpf_t val)

(Macro) SET_MPFMATRIX_IJ(mat, i, j, val) set_mpfmatrix_ij((mat), (i), (j), (val))

- void set_ddmatrix_ij_d(DDMatrix mat, long int i, long int j, double val)

(Macro) SET_DDMATRIX_IJ_D(mat, i, j, val) set_ddmatrix_ij_d((mat), (i), (j), (val))

(Macro) SET_DDMATRIX_IJ_UI(mat, i, j, val) set_ddmatrix_ij_d((mat), (i), (j), (double)(val))

(Macro) set_ddmatrix_ij_ui(mat, i, j, val) set_ddmatrix_ij_d((mat), (i), (j), (double)(val))

- void set0_ddmatrix_ij(DDMatrix mat, long int i, long int j)

(Macro) SET0_DDMATRIX_IJ(mat, i, j) set0_ddmatrix_ij((mat), (i), (j))

- void set0_ddmatrix(DDMatrix mat) ··· set mat as zero matrix
- void set0_tdmatrix(TDMatrix mat)
- void set0_qdmatrix(QDMatrix mat)
- void set0_mpfmatrix(MPFMatrix mat)
- DDMatrix init_ddmatrix(long int row_dim, long int col_dim) ··· Initialize DDMatrix.
- TDMatrix init_tdmatrix(long int row_dim, long int col_dim) ··· Initialize TDMatrix.
- QDMatrix init_qdmatrix(long int row_dim, long int col_dim) ··· Initialize QDMatrix.
- MPFMatrix init_mpfmatrix(long int row_dim, long int col_dim) ··· Initialize MPFMatrix.
- MPFMatrix init2_mpfmatrix(long int row_dim, long int col_dim, unsigned long prec) ··· Initialize MPFMatrix as prec-bit precision.
- void free_ddmatrix(DDMatrix mat) ··· Free mat
- void free_tdmatrix(TDMatrix mat)
- void free_qdmatrix(QDMatrix mat)
- void free_mpfmatrix(MPFMatrix mat)
- void print_ddmatrix(DDMatrix mat) ··· Print mat.
- void print_tdmatrix(TDMatrix mat)
- void print_qdmatrix(QDMatrix mat)
- void print_mpfmatrix(MPFMatrix mat)
- void set_ddfloat_ddmat(ddfloat ret[], int ret_dim, DDMatrix mat) ··· Convert DDMatrix mat to ddfloat array.
- void set_tdfloat_tdmat(tdfloat ret[], int ret_dim, DDMatrix mat) ··· Convert TDMatrix mat to tdfloat array.
- void set_qdfloat_qdmat(qdfloat ret[], int ret_dim, DDMatrix mat) ··· Convert QDMatrix mat to qdfloat array.
- void set_ddmatrix_ddfloat(DDMatrix ret, ddfloat array[], int array_dim) ··· Convert ddfloat array to DDMatrix.
- void set_tdmatrix_tdfloat(TDMatrix ret, tdfloat array[], int array_dim) ··· Convert tdfloat array to TDMatrix.
- void set_qdmatrix_qdfloat(QDMatrix ret, qdfloat array[], int array_dim) ··· Convert qdfloat array to QDMatrix
- void mul_ddmatrix(DDMatrix ret, DDMatrix a, DDMatrix b) ··· Simple matrix multiplication.
- void mul_tdmatrix(TDMatrix ret, TDMatrix a, TDMatrix b)
- void mul_qdmatrix(QDMatrix ret, QDMatrix a, QDMatrix b)
- void mul_mpfmatrix(MPFMatrix ret, MPFMatrix a, MPFMatrix b)
- void normf_ddmatrix(double ret[DDSIZE], DDMatrix mat) ··· Get the value of Frobenius norm of mat.
- void normf_tdmatrix(double ret[TDSIZE], TDMatrix mat)
- void normf_qdmatrix(double ret[QDSIZE], QDMatrix mat)
- void normf_mpfmatrix(mpf_t ret, MPFMatrix mat)
- void print_normf_ddmatrix(const char * str, DDMatrix mat) ··· print the Frobenius norm of mat.
- void normi_ddmatrix(double ret[DDSIZE], DDMatrix mat) ··· Infinity norm of matrix.
- void normi_tdmatrix(double ret[TDSIZE], TDMatrix mat)
- void normi_qdmatrix(double ret[QDSIZE], QDMatrix mat)
- void normi_mpfmatrix(mpf_t ret, TDMatrix mat)
- void norm1_ddmatrix(double ret[DDSIZE], DDMatrix mat) ··· 1-norm of matrix.
- void norm1_tdmatrix(double ret[TDSIZE], TDMatrix mat)

- void norm1_qdmatrix(double ret[QDSIZE], QDMatrix mat)
- void norm1_mpfmatrix(mpf_t ret, MPFMatrix mat)
- void add_ddmatrix(DDMatrix c, DDMatrix a, DDMatrix b) $\cdots$ $c := a + b$
- void add_tdmatrix(TDMatrix c, TDMatrix a, TDMatrix b)
- void add_qdmatrix(QDMatrix c, QDMatrix a, QDMatrix b)
- void add_mpfmatrix(MPFMatrix c, MPFMatrix a, MPFMatrix b)
- void sub_ddmatrix(DDMatrix c, DDMatrix a, DDMatrix b) $\cdots$ $c := a - b$
- void sub_tdmatrix(TDMatrix c, TDMatrix a, TDMatrix b)
- void sub_qdmatrix(QDMatrix c, QDMatrix a, QDMatrix b)
- void sub_mpfmatrix(MPFMatrix c, MPFMatrix a, MPFMatrix b)
- void cmul_ddmatrix(DDMatrix c, double sc[DDSIZE], DDMatrix a) $\cdots$ $c := sc \times a$
- void cmul_tdmatrix(TDMatrix c, double sc[TDSIZE], TDMatrix a)
- void cmul_ddmatrix(QDMatrix c, double sc[QDSIZE], QDMatrix a)
- void cmul_mpfmatrix(MPFMatrix c, mpf_t sc, MPFMatrix a)
- void transpose_ddmatrix(DDMatrix c, DDMatrix a) $\cdots$ $c := a^T$
- void transpose_tdmatrix(TDMatrix c, TDMatrix a)
- void transpose_qdmatrix(QDMatrix c, QDMatrix a)
- void transpose_mpfmatrix(MPFMatrix c, MPFMatrix a)
- void subst_ddmatrix(DDMatrix c, DDMatrix a) $\cdots$ $c := a$
- void subst_tdmatrix(TDMatrix c, TDMatrix a)
- void subst_qdmatrix(QDMatrix c, QDMatrix a)
- void subst_mpfmatrix(MPFMatrix c, MPFMatrix a)
- void setI_ddmatrix(DDMatrix c) $\cdots$ $c := I$
- void setI_tdmatrix(TDMatrix c)
- void setI_qdmatrix(QDMatrix c)
- void setI_mpfmatrix(MPFMatrix c)
- void mul_ddmatrix_ddvec(DDVector v, DDMatrix a, DDVector vb) $\cdots$ $v := a * vb$
- void mul_tdmatrix_tdvec(TDVector v, TDMatrix a, TDVector vb)
- void mul_qdmatrix_qdvec(QDVector v, QDMatrix a, QDVector vb)
- void mul_mpfmatrix_mpfvec(MPFVector v, MPFMatrix a, MPFVector vb)
- void mul_ddmatrixt_ddvec(DDVector v, DDMatrix a, DDVector vb) $\cdots$ $v := a^T * vb$
- void mul_tdmatrixt_tdvec(TDVector v, TDMatrix a, TDVector vb)
- void mul_qdmatrixt_qdvec(QDVector v, QDMatrix a, QDVector vb)
- void mul_mpfmatrixt_mpfvec(MPFVector v, MPFMatrix a, MPFVector vb)
- void inv_ddmatrix(DDMatrix a) $\cdots$ $a := a^{-1}$ (only for square matrix)
- void inv_tdmatrix(TDMatrix a)
- void inv_qdmatrix(QDMatrix a)
- void inv_mpfmatrix(MPFMatrix a)

- void subst_mpfvector_ddvec(MPFVector c, DDVector a) $\cdots$ $c := (mpf)a$
- void subst_ddvector_mpfvec(DDVector c, MPFVector MPFVector a) $\cdots$ $c := (dd)a$
- void subst_mpfmatrix_ddmat(MPFMatrix c, DDMatrix a) $\cdots$ $c := (mpf)a$
- void subst_ddmatrix_mpfmat(DDMatrix c, MPFMatrix a) $\cdots$ $c := (dd)a$
- void relerr_ddvector_mpfvec(double relerr[DDSIZE], DDVector approx_vec, MPFVector true_vec, int norm_type) $\cdots$ Norm relative error of vector.
- void relerr_element_ddvector_mpf(double max_relerr[DDSIZE], double min_relerr[DDSIZE], double norm_relerr[DDSIZE], DDVector approx_vec, MPFVector true_vec, int norm_type) $\cdots$ E lementwise relative errors of vector.
  /* c := (dd)a */
- void subst_ddvector_dvec(DDVector c, DVector sa) $\cdots$ $c := (dd)a$
  /* c := (d)a */
- void subst_dvector_ddvec(DVector c, DDVector a) $\cdots$ $c := (double)a$
  /* c := (dd)a */

- void `subst_ddmatrix_dmat`(DDMatrix `c`, DMatrix `a`) $\cdots$ $c := (dd)a$
  /* c := (d)a */
- void `subst_dmatrix_ddmat`(DMatrix `c`, DDMatrix `a`) $\cdots$ $c := (double)a$
- void `relerr_ddvector`(double `relerr[DDSIZE]`, DDVector `approx_vec`, DDVector `true_vec`, int `norm_type`) $\cdots$ Norm relative error of vector.
- void `relerr_element_ddvector`(double `max_relerr[DDSIZE]`, double `min_relerr[DDSIZE]`, double `norm_relerr[DDSIZE]`, DDVector `approx_vec`, DDVector `true_vec`, int `norm_type`) $\cdots$ E lementwise relative errors of vector.
- void `row_swap_ddmatrix`(DDMatrix `mat`, long int `row_index0`, long int `row_index1`, long int `col_start`, long int `col_end`) $\cdots$ Exchange a(row_index0, col_start:col_end) to a(row_index1, col_start:col_end)

3.4 File I/O with matrix and vector

- void `fread_ddmatrix`(FILE * `fp`, DDMatrix `mat`) $\cdots$ Read matrix elements from fp and store theme in mat.
- void `fread_tdmatrix`(FILE * `fp`, TDMatrix `mat`)
- void `fread_qdmatrix`(FILE * `fp`, QDMatrix `mat`)
- void `fread_mpfmatrix`(FILE * `fp`, QDMatrix `mat`)
- void `fread_ddmatrix_fname`(const char * `fname`, DDMatrix `mat`) $\cdots$ Read matrix elements from fname and store theme in mat.
- void `fread_tdmatrix_fname`(const char * `fname`, TDMatrix `mat`)
- void `fread_qdmatrix_fname`(const char * `fname`, QDMatrix `mat`)
- void `fread_mpfmatrix_fname`(const char * `fname`, MPFMatrix `mat`)
- void `fwrite_ddmatrix`(FILE * `fp`, DDMatrix `mat`) $\cdots$ Write matrix elements of mat to fp.
- void `fwrite_tdmatrix`(FILE * `fp`, TDMatrix `mat`)
- void `fwrite_qdmatrix`(FILE * `fp`, QDMatrix `mat`)
- void `fwrite_mpfmatrix`(FILE * `fp`, MPFMatrix `mat`)
- void `fwrite_ddmatrix_fname`(const char * `fname`, DDMatrix `mat`) $\cdots$ Write matrix elements of mat to fname.
- void `fwrite_tdmatrix_fname`(const char * `fname`, TDMatrix `mat`)
- void `fwrite_qdmatrix_fname`(const char * `fname`, QDMatrix `mat`)
- void `fwrite_mpfmatrix_fname`(const char * `fname`, MPFMatrix `mat`)
- void `fread_ddvector`(FILE * `fp`, DDVector `vec`) $\cdots$ Read vector elements from fp and store theme in vec.
- void `fread_tdvector`(FILE * `fp`, TDVector `vec`)
- void `fread_qdvector`(FILE * `fp`, QDVector `vec`)
- void `fread_mpfvector`(FILE * `fp`, MPFVector `vec`)
- void `fread_ddvector_fname`(const char * `fname`, DDVector `vec`) $\cdots$ Read vector elements from fname and store theme in vec.
- void `fread_tdvector_fname`(const char * `fname`, TDVector `vec`)
- void `fread_qdvector_fname`(const char * `fname`, QDVector `vec`)
- void `fread_mpfvector_fname`(const char * `fname`, MPFVector `vec`)
- void `fwrite_ddvector`(FILE * `fp`, DDVector `vec`) $\cdots$ Write vector elements of vec to fp.
- void `fwrite_tdvector`(FILE * `fp`, TDVector `vec`)
- void `fwrite_qdvector`(FILE * `fp`, QDVector `vec`)
- void `fwrite_mpfvector`(FILE * `fp`, MPFVector `vec`)
- void `fwrite_ddvector_fname`(const char * `fname`, DDVector `vec`) $\cdots$ Write vector elements of vec to fname.
- void `fwrite_tdvector_fname`(const char * `fname`, TDVector `vec`)
- void `fwrite_qdvector_fname`(const char * `fname`, QDVector `vec`)
- void `fwrite_mpfvector_fname`(const char * `fname`, MPFVector `vec`)

- void read_test_linear_eq_dd(DDMatrix A, DDVector true_x, DDVector b, long int dim , const char * fname_A, const char * fname_true_x, const char * fname_b) ··· Read the coefficient matrix A from fname_A, the true_x from fname_true_x, and the vector b from fname_b.
- void read_test_linear_eq_td(TDMatrix A, TDVector true_x, TDVector b, long int dim , const char * fname_A, const char * fname_true_x, const char * fname_b)
- void read_test_linear_eq_qd(QDMatrix A, QDVector true_x, QDVector b, long int dim , const char * fname_A, const char * fname_true_x, const char * fname_b)
- void read_test_linear_eq_mpf(MPFMatrix A, MPFVector true_x, MPFVector b, long int dim , const char * fname_A, const char * fname_true_x, const char * fname_b)

3.5 Generating test matrices

The following functions provide various test matrices for benhmark tests.

- void hilbert_ddmatrix(DDMatrix a, long int dim) ··· Hilbert matrix.
- void hilbert_tdmatrix(TDMatrix a, long int dim)
- void hilbert_qdmatrix(QDMatrix a, long int dim)
- void hilbert_mpfmatrix(MPFMatrix a, long int dim)
- void lotkin_ddmatrix(DDMatrix a, long int dim) ··· Lotkin matrix.
- void lotkin_tdmatrix(TDMatrix a, long int dim)
- void lotkin_qdmatrix(QDMatrix a, long int dim)
- void lotkin_mpfmatrix(MPFMatrix a, long int dim)
- void frank_ddmatrix(DDMatrix a, long int dim) ··· Symmetric Frank matrix.
- void frank_tdmatrix(TDMatrix a, long int dim)
- void frank_qdmatrix(QDMatrix a, long int dim)
- void frank_mpfmatrix(MPFMatrix a, long int dim)
- void tridiag_ddmatrix(DDMatrix a, DDVector low_subdiag, DDVector diag, DDVector up_subdiag, long int dim) ··· Triagonal matrix.
- void tridiag_tdmatrix(DDMatrix a, TDVector low_subdiag, TDVector diag, TDVector up_subdiag, long int dim)
- void tridiag_qdmatrix(DDMatrix a, QDVector low_subdiag, QDVector diag, QDVector up_subdiag, long int dim)
- void tridiag_mpfmatrix(DDMatrix a, MPFVector low_subdiag, MPFVector diag, MPFVector up_subdiag, long int dim)
- void int_sym_rand_ddmatrix(DDMatrix mat, long int max, long int seed, long int dim) ··· Integer Symmetrix Random Matrix.
- void int_sym_rand_tdmatrix(TDMatrix mat, long int max, long int seed, long int dim)
- void int_sym_rand_qdmatrix(QDMatrix mat, long int max, long int seed, long int dim)
- void int_sym_rand_mpfmatrix(MPFMatrix mat, long int max, long int seed, long int dim)
- void int_unsym_rand_ddmatrix(DDMatrix mat, long int max, long int seed, long int dim) ··· Integer Unsymmetrix Random Matrix.
- void int_unsym_rand_tdmatrix(TDMatrix mat, long int max, long int seed, long int dim) ··· Integer Unsymmetrix Random Matrix.
- void int_unsym_rand_qdmatrix(QDMatrix mat, long int max, long int seed, long int dim) ··· Integer Unsymmetrix Random Matrix.
- void int_unsym_rand_mpfmatrix(MPFMatrix mat, long int max, long int seed, long int dim) ··· Integer Unsymmetrix Random Matrix.
- void diag_ddmatrix(DDMatrix mat, DDVector diag, long int dim) ··· Real Diagonal Matrix.
- void diag_tdmatrix(TDMatrix mat, TDVector diag, long int dim)
- void diag_qdmatrix(QDMatrix mat, QDVector diag, long int dim)
- void diag_mpfmatrix(MPFMatrix mat, MPFVector diag, long int dim)
- void toeplitz_ddmatrix(DDMatrix mat, double gamma_param[DDSIZE], long int dim) ··· Toeplitz

matrix.

- void toeplitz_tdmatrix(TDMatrix mat, double gamma_param[TDSIZE], long int dim)
- void toeplitz_qdmatrix(QDMatrix mat, double gamma_param[QDSIZE], long int dim)
- void toeplitz_mpfmatrix(MPFMatrix mat, mpf_t gamma_param, long int dim)
- void pascal_ddmatrix(DDMatrix ret, long int dim) $\cdots$ Pascal matrix.
- void pascal_tdmatrix(TDMatrix ret, long int dim)
- void pascal_qdmatrix(QDMatrix ret, long int dim)
- void pascal_mpfmatrix(MPFMatrix ret, long int dim)
- void im_rand_ddmatrix(DDMatrix ret, unsigned long seed) $\cdots$ $I - random$
- void im_rand_tdmatrix(TDMatrix ret, unsigned long seed)
- void im_rand_qdmatrix(QDMatrix ret, unsigned long seed)
- void im_rand_mpfmatrix(MPFMatrix ret, unsigned long seed)

3.6 Simple, block, and Strassen matrix multiplication and related functions

(Macro) mul_ddmatrix_simple(c, a, b) $\cdots$ mul_ddmatrixc, a, b $\cdots$ Matrix multiplication based on simple triple-loop way

(Macro) mul_tdmatrix_simple(c, a, b) $\cdots$ mul_tdmatrixc, a, b

(Macro) mul_qdmatrix_simple(c, a, b) $\cdots$ mul_qdmatrixc, a, b

- void mul_mpfmatrix_simple(MPFMatrix c, MPFMatrix a, MPFMatrix b)
- void mul_dmatrix_strassen(DMatrix ret, DMatrix mat_a, DMatrix mat_b, long int min_dim) $\cdots$ Matrix multiplication based on Strassen or Winograd algorithms
- void mul_ddmatrix_strassen(DDMatrix ret, DDMatrix mat_a, DDMatrix mat_b, long int min_dim)
- void mul_tdmatrix_strassen(TDMatrix ret, TDMatrix mat_a, TDMatrix mat_b, long int min_dim)
- void mul_qdmatrix_strassen(QDMatrix ret, QDMatrix mat_a, QDMatrix mat_b, long int min_dim)
- void mul_mpfmatrix_strassen(MPFMatrix ret, MPFMatrix mat_a, MPFMatrix mat_b, long int min_dim)
- void mul_dmatrix_block(DMatrix ret, DMatrix mat_a, DMatrix mat_b, long int min_dim) $\cdots$ Block matrix multiplicaiton
- void mul_ddmatrix_block(DDMatrix ret, DDMatrix mat_a, DDMatrix mat_b, long int min_dim)
- void mul_tdmatrix_block(TDMatrix ret, TDMatrix mat_a, TDMatrix mat_b, long int min_dim)
- void mul_qdmatrix_block(QDMatrix ret, QDMatrix mat_a, QDMatrix mat_b, long int min_dim)
- void mul_mpfmatrix_block(MPFMatrix ret, MPFMatrix mat_a, MPFMatrix mat_b, long int min_dim)
- void mul_dmatrix_strassen_even(DMatrix ret, DMatrix mat_a, DMatrix mat_b, long int min_dim) $\cdots$ Strassen's Algorithm (even dimension)
- void mul_ddmatrix_strassen_even(DDMatrix ret, DDMatrix mat_a, DDMatrix mat_b, long int min_dim)
- void mul_tdmatrix_strassen_even(TDMatrix ret, TDMatrix mat_a, TDMatrix mat_b, long int min_dim)
- void mul_qdmatrix_strassen_even(QDMatrix ret, QDMatrix mat_a, QDMatrix mat_b, long int min_dim)
- void mul_mpfmatrix_strassen_even(MPFMatrix ret, MPFMatrix mat_a, MPFMatrix mat_b, long int min_dim)
- void mul_dmatrix_winograd_even(DMatrix ret, DMatrix mat_a, DMatrix mat_b, long int min_dim) $\cdots$ Winograd Variant of Strassen's Algorithm
- void mul_ddmatrix_winograd_even(DDMatrix ret, DDMatrix mat_a, DDMatrix mat_b, long int min_dim)
- void mul_tdmatrix_winograd_even(TDMatrix ret, TDMatrix mat_a, TDMatrix mat_b, long int

min_dim)

- void mul_qdmatrix_winograd_even(QDMatrix ret, QDMatrix mat_a, QDMatrix mat_b, long int min_dim)
- void mul_mpfmatrix_winograd_even(MPFMatrix ret, MPFMatrix mat_a, MPFMatrix mat_b, long int min_dim)
- void inv_ddmatrix_strassen_even(DDMatrix ret, DDMatrix mat_a, long int min_dim) ··· Computattion of Inverse Matrix by using Strassen's Algorithm
- void inv_tdmatrix_strassen_even(TDMatrix ret, TDMatrix mat_a, long int min_dim)
- void inv_qdmatrix_strassen_even(QDMatrix ret, QDMatrix mat_a, long int min_dim)

3.6.1 OpenMP

(Macro) _bncomp_mul_ddmatrix_simple(c, a, b) _bncomp_mul_ddmatrix(c, a, b) ··· Parallelized simple matrix multiplication,

(Macro) _bncomp_mul_tdmatrix_simple(c, a, b) _bncomp_mul_tdmatrix(c, a, b)

(Macro) _bncomp_mul_qdmatrix_simple(c, a, b) _bncomp_mul_qdmatrix(c, a, b)

(Macro) _bncomp_mul_mpfdmatrix_simple(c, a, b) _bncomp_mul_mpfdmatrix(c, a, b)

- void _bncomp_mul_ddmatrix_block(DDMatrix ret, DDMatrix mat_a, DDMatrix mat_b, long int min_dim) ··· Parallelized block matrix multiplicaiton.
- void _bncomp_mul_tdmatrix_block(TDMatrix ret, TDMatrix mat_a, TDMatrix mat_b, long int min_dim)
- void _bncomp_mul_qdmatrix_block(QDMatrix ret, QDMatrix mat_a, QDMatrix mat_b, long int min_dim)
- void _bncomp_mul_mpfmatrix_block(MPFMatrix ret, MPFMatrix mat_a, MPFMatrix mat_b, long int min_dim)
- void _bncomp_mul_ddmatrix_strassen(DDMatrix ret, DDMatrix mat_a, DDMatrix mat_b, long int min_dim) ··· Parallelized Strassen and Winograd matrix multiplication (limited performance due to poor coding)
- void _bncomp_mul_tdmatrix_strassen(TDMatrix ret, TDMatrix mat_a, TDMatrix mat_b, long int min_dim)
- void _bncomp_mul_qdmatrix_strassen(QDMatrix ret, QDMatrix mat_a, QDMatrix mat_b, long int min_dim)
- void _bncomp_mul_mpfmatrix_strassen(MPFMatrix ret, MPFMatrix mat_a, MPFMatrix mat_b, long int min_dim)

3.7 Getting relative errors of vector and matrix

- void relerr3_dvector(double * max_relerr, double * min_relerr, double * norm_relerr, DVector vec, DVector vec_true, int kind_of_norm) ··· Relative errors of vec.
- void relerr3_ddvector(double max_relerr[DDSIZE], double min_relerr[DDSIZE], double norm_relerr[DDSIZE], DDVector vec, DDVector vec_true, int kind_of_norm)
- void relerr3_tdvector(double max_relerr[TDSIZE]) , double min_relerr[TDSIZE], double norm_relerr[TDSIZE], TDVector vec, TDVector vec_true, int kind_of_norm
- void relerr3_qdvector(double max_relerr[QDSIZE]) , double min_relerr[QDSIZE], double norm_relerr[QDSIZE], QDVector vec, QDVector vec_true, int kind_of_norm
- void relerr3_mpfvector(mpf_t max_relerr, mpf_t min_relerr, mpf_t norm_relerr, MPFVector vec, MPFVector vec_true, int kind_of_norm)
- void relerr3_dmatrix(double * max_relerr, double * min_relerr, double * norm_relerr, DMatrix mat, DMatrix mat_true, int kind_of_norm) ··· Relative errors of mat.
- void relerr3_ddmatrix(double max_relerr[DDSIZE], double min_relerr[DDSIZE], double norm_relerr[DDSIZE], DDMatrix mat, DDMatrix mat_true, int kind_of_norm)

- void relerr3_tdmatrix(double max_relerr[TDSIZE], double min_relerr[TDSIZE], double norm_relerr[TDSIZE], TDMatrix mat, TDMatrix mat_true, int kind_of_norm)
- void relerr3_qdmatrix(double max_relerr[QDSIZE], double min_relerr[QDSIZE], double norm_relerr[QDSIZE], QDMatrix mat, QDMatrix mat_true, int kind_of_norm)
- void relerr3_mpfmatrix(mpf_t max_relerr, mpf_t min_relerr, mpf_t norm_relerr, MPFMatrix mat, MPFMatrix mat_true, int kind_of_norm)

3.8 Ozaki scheme and related functions

These following functions are necessary for Ozaki scheme and multi-fold precision arithmetic.

- void extract_dvector(DVector ret_high_vec, DVector ret_low_vec, DVector org_vec, double num_bits) $\cdots$ ret_high + ret_low = org_vec
- void split_dmatrix(DMatrix ret_high_mat , DMatrix ret_low_mat, DMatrix org_mat) $\cdots$ split org_mat to ret_high_mat and ret_low_mat in row direction
- void split_dmatrix_t(DMatrix ret_high_mat ,DMatrix ret_low_mat, DMatrix org_mat) $\cdots$ split org_mat to ret_high_mat and ret_low_mat in column direction
- int split_ddvector_dvec(DVector ret_vec[], int num_div, DDVector org_vec) $\cdots$ Split org_vec to ret_vec[0] + ret_vec[1] +...+ ret_vec[num_div - 1]
- int split_tdvector_dvec(DVector ret_vec[], int num_div, TDVector org_vec)
- void absmax_ddvector(double ret[DDSIZE], long int * max_index, DDVector vec) $\cdots$ get absolutely maximum of elements in vec.
- void absmax_tdvector(double ret[TDSIZE], long int * max_index, TDVector vec)
- void absmax_qdvector(double ret[QDSIZE], long int * max_index, QDVector vec)
- void absmax_mpfvector(mpf_t ret, long int * max_index, MPFVector vec)
- void add_ddvector_dvec(DDVector c, DDVector a, DVector b) $\cdots$ $c := a + (double)b$
- void add_tdvector_dvec(TDVector c, TDVector a, DVector b)
- void add_qdvector_dvec(QDVector c, QDVector a, DVector b)
- void add_mpfvector_dvec(MPFVector c, MPFVector a, DVector b) ;
- void sub_ddvector_dvec(DDVector c, DDVector a, DVector b) $\cdots$ $c := a - (double)b$
- void sub_tdvector_dvec(TDVector c, TDVector a, DVector b)
- void sub_qdvector_dvec(QDVector c, QDVector a, DVector b)
- void sub_mpfvector_dvec(MPFVector c, MPFVector a, DVector b)
- void subst_dvector_qdvec(DVector c, QDVector a) $\cdots$ $c := (DDVector)a$
- void absmax_row_ddmatrix(double mu[DDSIZE], long int * max_j, long int row_index, DDMatrix mat) $\cdots$ Return the absolutely maximum element in the max_j-th row.
- void absmax_row_tdmatrix(double mu[TDSIZE], long int * max_j, long int row_index, TDMatrix mat)
- void absmax_row_qdmatrix(double mu[QDSIZE], long int * max_j, long int row_index, QDMatrix mat)
- void absmax_row_mpfmatrix(mpf_t mu, long int * max_j, long int row_index, MPFMatrix mat)
- void add_ddmatrix_dmat(DDMatrix c, DDMatrix a, DMatrix b) $\cdots$ $c := a + (DMatrix)b$
- void add_tdmatrix_dmat(TDMatrix c, TDMatrix a, DMatrix b)
- void add_qdmatrix_dmat(QDMatrix c, QDMatrix a, DMatrix b)
- void add_mpfmatrix_dmat(MPFMatrix c, MPFMatrix a, DMatrix b)
- void sub_ddmatrix_dmat(DDMatrix c, DDMatrix a, DMatrix b) $\cdots$ $c := a - (DMatrix)b$
- void sub_tdmatrix_dmat(TDMatrix c, TDMatrix a, DMatrix b)
- void sub_qdmatrix_dmat(QDMatrix c, QDMatrix a, DMatrix b)
- void sub_mpfmatrix_dmat(MPFMatrix c, MPFMatrix a, DMatrix b)
- int split_ddmatrix_dmat(DMatrix ret_mat[], int num_div, DDMatrix org_mat) *cdots* Split org_mat to ret_mat[] in row direction and return real number of divisions
- int split_tdmatrix_dmat(DMatrix ret_mat[], int num_div, TDMatrix org_mat)

- int split_qdmatrix_dmat(DMatrix ret_mat[], int num_div, QDMatrix org_mat)
- int split_mpfmatrix_dmat(DMatrix ret_mat[], int num_div, MPFMatrix org_mat)
- void absmax_col_ddmatrix(double mu[DDSIZE], long int * max_i, long int col_index, DDMatrix mat) $\cdots$ return absolutely maximum element and its (i, j)-index in each column of mat.
- void absmax_col_tdmatrix(double mu[TDSIZE], long int * max_i, long int col_index, TDMatrix mat)
- void absmax_col_qdmatrix(double mu[QDSIZE], long int * max_i, long int col_index, QDMatrix-mat)
- void absmax_col_mpfmatrix(mpf_t mu, long int * max_i, long int col_index, MPFMatrix mat)
- int split_ddmatrix_t_dmat(DMatrix ret_mat[], int num_div, DDMatrix org_mat) $\cdots$ Split org_mat to ret_mat[] in column direction and return real number of divisions.
- int split_tdmatrix_t_dmat(DMatrix ret_mat[], int num_div, TDMatrix org_mat)
- int split_qdvector_dvec(DVector ret_vec[], int num_div, QDVector org_vec)
- int split_qdmatrix_t_dmat(DMatrix ret_mat[], int num_div, QDMatrix org_mat)
- int split_mpfmatrix_t_dmat(DMatrix ret_mat[], int num_div , MPFMatrix org_mat)
- void subst_qdmatrix_dmat(QDMatrix c, DMatrix a)
- void subst_dmatrix_qdmat(DMatrix c, QDMatrix a)
- void mul_ddmatrix_oz(DDMatrix ret, DDMatrix a, int max_num_div_a, DDMatrix b, int max_num_div_b) *cdots* Matrix multiplication based on Ozaki scheme
- void mul_tdmatrix_oz(TDMatrix ret, TDMatrix a, int max_num_div_a, TDMatrix b, int max_num_div_b)
- void mul_qdmatrix_oz(QDMatrix ret, QDMatrix a, int max_num_div_a, QDMatrix b, int max_num_div_b)
- void mul_mpfmatrix_oz(MPFMatrix ret, MPFMatrix a, int max_num_div_a, MPFMatrix b, int max_num_div_b)
- void mul_ddmatrix_ddvec_oz(DDVector ret, DDMatrix a, int max_num_div_a, DDVector vb, int max_num_div_vb) $\cdots$ Matrix-Vector multiplication based on Ozaki scheme
- void mul_tdmatrix_tdvec_oz(TDVector ret, TDMatrix a, int max_num_div_a, TDVector vb, int max_num_div_vb)
- void mul_qdmatrix_qdvec_oz(QDVector ret, QDMatrix a, int max_num_div_a, QDVector vb, int max_num_div_vb)
- void mul_mpfmatrix_dvec_oz(MPFVector ret, MPFMatrix a, int max_num_div_a, MPFVector vb, int max_num_div_vb)

Chapter 4

LU decomposition

BNCmatmul has various LU decompositions and solvers for the corresponding decomposed linear system of equations.

4.1 List of functions

- int `DLUdecomp`(DMatrix A) $\cdots$ LU decompose the matrix A without any types of pivoting.
- int `DDLUdecomp`(DDMatrix A)
- int `TDLUdecomp`(TDMatrix A)
- int `QDLUdecomp`(QDMatrix A)
- int `MPFLUdecomp`(MFPMatrix A)
- int `SolveDLS`(DVector $\mathbf{x}$, DMatrix LU, DVector $\mathbf{b}$) $\cdots$ Solve $(LU)\mathbf{x} = \mathbf{b}$ for $\mathbf{x}$.
- int `SolveDDLS`(DDVector $\mathbf{x}$, DDMatrix LU, DDVector $\mathbf{b}$)
- int `SolveTDLS`(TDVector $\mathbf{x}$, TDMatrix LU, TDVector $\mathbf{b}$)
- int `SolveQDLS`(QDVector $\mathbf{x}$, QDMatrix `l`LU, QDVector $\mathbf{b}$)
- int `SolveMPFLS`(MPFVector $\mathbf{x}$, MPFMatrix LU, MPFVector $\mathbf{b}$)
- int `DLUdecompP`(DMatrix A, long int `ch[]`) $\cdots$ LU decompose the matrix A with partial pivoting, order of rows is stored in `ch[]`.
- int `DDLUdecompP`(DDMatrix A, long int `ch[]`)
- int `TDLUdecompP`(TDMatrix A, long int `ch[]`)
- int `QDLUdecompP`(DDMatrix A, long int `ch[]`)
- int `MPFLUdecompP`(MPFMatrix A, long int `ch[]`)
- int `SolveDLSP`(DVector $\mathbf{x}$, DMatrix LU, DVector $\mathbf{b}$, long int `ch[]`) $\cdots$ Solve $(LU)\mathbf{x} = \mathbf{b}$ for $\mathbf{x}$ using `ch[]` as row numbering.
- int `SolveDDLSP`(DDVector $\mathbf{x}$, DDMatrix LU, DDVector $\mathbf{b}$, long int `ch[]`)
- int `SolveTDLSP`(TDVector $\mathbf{x}$, TDMatrix LU, TDVector $\mathbf{b}$, long int `ch[]`)
- int `SolveQDLSP`(DDVector $\mathbf{x}$, QDMatrix LU, QDVector $\mathbf{b}$, long int `ch[]`)
- int `SolveMPFLSP`(MPFVector $\mathbf{x}$, MPFMatrix LU, MPFVector $\mathbf{b}$, long int `ch[]`)
- int `DLUdecompC`(DMatrix A, long int `row_ch[]`, long int `col_ch[]`) $\cdots$ LU decompose the matrix A with complete pivoting, order of rows is stored in `row_ch[]`, and order of columns in `col_ch[]`.
- int `DDLUdecompC`(DDMatrix A, long int `row_ch[]`, long int `col_ch[]`)
- int `TDLUdecompC`(TDMatrix A, long int `row_ch[]`, long int `col_ch[]`)
- int `QDLUdecompC`(QDMatrix A, long int `row_ch[]`, long int `col_ch[]`)
- int `MPFLUdecompC`(MPFMatrix A, long int `row_ch[]`, long int `col_ch[]`)
- int `SolveDLSC`(DVector $\mathbf{x}$, DMatrix LU, DVector $\mathbf{b}$, long int `row_ch[]`, long int `col_ch[]`) $\cdots$ Solve $(LU)\mathbf{x} = \mathbf{b}$ for $\mathbf{x}$ using `row_ch[]` and `col_ch[]` as row and column numbering, respectively.
- int `SolveDDLSC`(DDVector $\mathbf{x}$, DDMatrix LU, DDVector $\mathbf{b}$, long int `row_ch[]`, long int `col_ch[]`)
- int `SolveTDLSC`(TDVector $\mathbf{x}$, TDMatrix LU, TDVector $\mathbf{b}$, long int `row_ch[]`, long int `col_ch[]`)
- int `SolveQDLSC`(QDVector $\mathbf{x}$, QDMatrix LU, QDVector $\mathbf{b}$, long int `row_ch[]`, long int `col_ch[]`)
- int `SolveMPFLSC`(MPFVector $\mathbf{x}$, MPFMatrix LU, MPFVector $\mathbf{b}$, long int `row_ch[]`, long int `col_ch[]`)

- int `DLUdecomp_strassen`(DMatrix A, long int `min_dim`) $\cdots$ LU decompose, using Strassen matrix multiplication.
- int `DDLUdecomp_strassen`(DDMatrix A, long int `min_dim`)
- int `TDLUdecomp_strassen`(TDMatrix A, long int `min_dim`)
- int `QDLUdecomp_strassen`(QDMatrix A, long int `min_dim`)
- int `MPFLUdecomp_strassen`(MPFMatrix A, long int `min_dim`)
- int `DLUdecomp_strassenPM`(DMatrix A, long int `ch[]`, long int `min_dim`) $\cdots$ LU decompose, using Strassen matrix multiplication, the matrix A with partial pivoting, order of rows is stored in `ch[]`.
- int `DDLUdecomp_strassenPM`(DDMatrix A, long int `ch[]`, long int `min_dim`)
- int `TDLUdecomp_strassenPM`(TDMatrix A, long int `ch[]`, long int `min_dim`)
- int `QDLUdecomp_strassenPM`(QDMatrix A, long int `ch[]`, long int `min_dim`)
- int `MPFLUdecomp_strassenPM`(MPFMatrix A, long int `ch[]`, long int `min_dim`)
- int `DLUdecomp_oz`(DMatrix A, long int `min_dim`, int `max_num_div`) $\cdots$ LU decompose, using Ozaki scheme setting `max_num_div` as maximum number of divisions.
- int `DDLUdecomp_oz`(DDMatrix A, long int `min_dim`, int `max_num_div`)
- int `TDLUdecomp_oz`(DMatrix A, long int `min_dim`, int `max_num_div`)
- int `QDLUdecomp_oz`(DMatrix A, long int `min_dim`, int `max_num_div`)
- int `MPFLUdecomp_oz`(MPFMatrix A, long int `min_dim`, int `max_num_div`)
- int `DLUdecomp_ozPM`(DMatrix A, long int `ch[]`, long int `min_dim`, int `max_num_div`) $\cdots$ LU decompose, using partial pivoting and Ozaki scheme setting `max_num_div` as maximum number of divisions.
- int `DDLUdecomp_ozPM`(DMatrix A, long int `ch[]`, long int `min_dim`, int `max_num_div`)
- int `TDLUdecomp_ozPM`(DMatrix A, long int `ch[]`, long int `min_dim`, int `max_num_div`)
- int `QDLUdecomp_ozPM`(DMatrix A, long int `ch[]`, long int `min_dim`, int `max_num_div`)
- int `MPFLUdecomp_ozPM`(MPFMatrix A, long int `ch[]`, long int `min_dim`, int `max_num_div`)

Chapter 5

Array, polynomial and solvers of algebraic equations

As application of multiple-precision arithmetic and linear computing described previously, we have introduced accelerated solvers of algebraic equations as presenting in this chapter.

5.1 Data types of Array and polynomial

Polynomials are represented as in equation (5.1):

$$p_n(x) = \sum_{k=0}^{n} a_k x^k \tag{5.1}$$

The library provides the following polynomial and array types, all declared in `poly.h` (copyright © 2002–2025 Tomonori Kouya; distributed under the GNU LGPL v3 or later).

5.1.1 Polynomial types

Float (`FPoly`) and float complex (`CFPoly`)

- `fpoly` — single-precision polynomial; pointer type `FPoly`. Members: `float *coef`, `long int deg`, `long int max_len`.
- `cfpoly` — single-precision complex polynomial; pointer type `CFPoly`. Members: `fcmplx *coef`, `long int deg`, `long int max_len`.

Double (`DPoly`) and double complex (`CDPoly`)

- `dpoly` — double-precision polynomial; pointer type `DPoly`. Members: `double *coef`, `long int deg`, `long int max_len`.
- `cdpoly` — double-precision complex polynomial; pointer type `CDPoly`. Members: `double _Complex *coef`, `long int deg`, `long int max_len`.

Double-double (`DDPoly`) and complex double-double (`CDDPoly`)
Available when `USE_DDLINEAR` is defined.

- `ddpoly` — double-double polynomial; pointer type `DDPoly`. Members: `ddfloat *coef`, `ddfloat zero`, `long int deg`, `long int max_len`.
- `cddpoly` — complex double-double polynomial; pointer type `CDDPoly`. Members: `cddfloat *coef`, `cddfloat zero`, `long int deg`, `long int max_len`.

Triple-double (`TDPoly`) and complex triple-double (`CTDPoly`)
Available when `USE_TDLINEAR` is defined.

- tdpoly — triple-double polynomial; pointer type TDPoly. Members: tdfloat *coef, tdfloat zero, long int deg, long int max_len.
- ctdpoly — complex triple-double polynomial; pointer type CTDPoly. Members: ctdfloat *coef, ctdfloat zero, long int deg, long int max_len.

Quadruple-double (QDPoly) and complex quadruple-double (CQDPoly)
Available when USE_QDLINEAR is defined.

- qdpoly — quadruple-double polynomial; pointer type QDPoly. Members: qdfloat *coef, qdfloat zero, long int deg, long int max_len.
- cqdpoly — complex quadruple-double polynomial; pointer type CQDPoly. Members: cqdfloat *coef, cqdfloat zero, long int deg, long int max_len.

Multiple-precision (MPFPoly) and complex multiple-precision (CMPFPoly)
Available when USE_GMP is defined.

- mpfpoly — GMP multiple-precision polynomial; pointer type MPFPoly. Members: unsigned long int prec, mpf_t *coef, mpf_t zero, long int deg, long int max_len.
- cmpfpoly — MPC complex multiple-precision polynomial; pointer type CMPFPoly. Members: unsigned long int prec, mpc_t *coef, mpc_t zero, long int deg, long int max_len.
- _bncold_cmpfpoly — legacy complex multiple-precision polynomial (old implementation); pointer type _bncold_CMPFPoly. Members: unsigned long int prec, mpfcmplx *coef, long int deg, long int max_len.

5.1.2 Array types

- farray — float array; pointer type FArray. Members: float *array, long int size.
- cfarray — float complex array; pointer type CFArray. Members: fcmplx *array, long int size.
- darray — double array; pointer type DArray. Members: double *array, long int size.
- cdarray — double complex array; pointer type CDArray. Members: dcmplx *array, long int size.
- mpfarray — GMP multiple-precision array (requires USE_GMP); pointer type MPFArray. Members: unsigned long prec, mpf_t *array, long int size.
- cmpfarray — MPC complex multiple-precision array (requires USE_GMP); pointer type CMPFArray. Members: unsigned long prec, mpc_t *array, long int size.
- _bncold_cmpfarray — legacy complex multiple-precision array (requires USE_GMP); pointer type _bncold_CMPFArray. Members: unsigned long prec, mpfcmplx *array, long int size.

5.2 List of functions

5.2.1 Float polynomial functions (poly.c — FPoly)

- FPoly init_fpoly(long int max_len)
- void free_fpoly(FPoly pol)
- float get_fpoly_i(FPoly pol, long int i) (alias: gfpi)
- void set_fpoly_i(FPoly pol, long int i, float val) (alias: sfpi)
- long int setdegree_fpoly(FPoly pol)
- void print_fpoly(FPoly pol)
- void add_fpoly(FPoly ret, FPoly a, FPoly b)
- void sub_fpoly(FPoly ret, FPoly a, FPoly b)
- void cmul_fpoly(FPoly ret, float c, FPoly a)

- void subst_fpoly(FPoly ret, FPoly a)
- void set0_fpoly(FPoly pol)
- long int max_abscoef_fpoly(FPoly pol)
- long int num_nonzero_fpoly(FPoly pol)
- void diff_fpoly(FPoly pol)
- float eval_fpoly(FPoly pol, float x)
- float eval_diff_fpoly(FPoly pol, float x)
- void ceval_fpoly(FCmplx ret, FPoly pol, FCmplx x)
- void ceval_diff_fpoly(FCmplx ret, FPoly pol, FCmplx x)

5.2.2 Double polynomial functions (poly.c — DPoly)

- DPoly init_dpoly(long int max_len)
- void free_dpoly(DPoly pol)
- DPoly init_set_dpoly(DPoly org)
- double get_dpoly_i(DPoly pol, long int i) (alias: gdpi)
- void set_dpoly_i(DPoly pol, long int i, double val) (alias: sdpi)
- long int setdegree_dpoly(DPoly pol)
- void print_dpoly(DPoly pol)
- void add_dpoly(DPoly ret, DPoly a, DPoly b)
- void sub_dpoly(DPoly ret, DPoly a, DPoly b)
- void add2_dpoly(DPoly ret, DPoly a)
- void sub2_dpoly(DPoly ret, DPoly a)
- void mul_dpoly(DPoly ret, DPoly a, DPoly b)
- void cmul_dpoly(DPoly ret, double c, DPoly a)
- void subst_dpoly(DPoly ret, DPoly a)
- void set0_dpoly(DPoly pol)
- long int max_abscoef_dpoly(DPoly pol)
- long int num_nonzero_dpoly(DPoly pol)
- void diff_dpoly(DPoly pol)
- double eval_dpoly_horner(DPoly pol, double x) (also aliased as eval_dpoly)
- double eval_dpoly_estrin(DPoly pol, double x)
- double eval_diff_dpoly(DPoly pol, double x)
- void div_dpoly(DPoly quo, DPoly rem, DPoly a, DPoly b)
- void xpow_mul_dpoly(DPoly ret, long int n, DPoly a)
- void ceval_dpoly_horner(DCmplx ret, DPoly pol, DCmplx x) (also aliased as ceval_dpoly)
- void ceval_dpoly_estrin(DCmplx ret, DPoly pol, DCmplx x)
- void ceval_diff_dpoly(DCmplx ret, DPoly pol, DCmplx x)
- double _bncavx2_eval_dpoly_estrin(DPoly a, double x) (AVX2)
- void _bncavx2_ceval_dpoly_estrin(DCmplx ret, DPoly a, DCmplx x) (AVX2)

5.2.3 Double complex polynomial functions (poly.c — CDPoly)

- CDPoly init_cdpoly(long int max_len)
- void free_cdpoly(CDPoly pol)
- CDPoly init_set_cdpoly(CDPoly org)
- CDPoly init_set_cdpoly_dpoly(DPoly org)
- void set_cdpoly_dpoly(CDPoly ret, DPoly src)
- double _Complex get_cdpoly_i(CDPoly pol, long int i) (alias: gcdpi)
- void set_cdpoly_i(CDPoly pol, long int i, double _Complex val) (alias: scdpi)
- void set_cdpoly_i_d(CDPoly pol, long int i, double val)
- long int setdegree_cdpoly(CDPoly pol)

- void print_cdpoly(CDPoly pol)
- void add_cdpoly(CDPoly ret, CDPoly a, CDPoly b)
- void sub_cdpoly(CDPoly ret, CDPoly a, CDPoly b)
- void add2_cdpoly(CDPoly ret, CDPoly a)
- void sub2_cdpoly(CDPoly ret, CDPoly a)
- void mul_cdpoly(CDPoly ret, CDPoly a, CDPoly b)
- void mul2_cdpoly(CDPoly ret, CDPoly a)
- void cmul_cdpoly(CDPoly ret, double _Complex c, CDPoly a)
- void subst_cdpoly(CDPoly ret, CDPoly a)
- void set0_cdpoly(CDPoly pol)
- long int max_abscoef_cdpoly(CDPoly pol)
- long int num_nonzero_cdpoly(CDPoly pol)
- void diff_cdpoly(CDPoly pol)
- double _Complex eval_cdpoly_horner(CDPoly a, double _Complex x) (also aliased as eval_cdpoly)
- double _Complex eval_cdpoly_estrin(CDPoly a, double _Complex x)
- double _Complex eval_diff_cdpoly(CDPoly a, double _Complex x)
- void div_cdpoly(CDPoly quo, CDPoly rem, CDPoly a, CDPoly b)
- void xpow_mul_cdpoly(CDPoly ret, long int n, CDPoly a)

5.2.4 Double-double polynomial functions (poly.c — DDPoly, CDDPoly; requires USE_DDLINEAR)

- DDPoly init_ddpoly(long int max_len)
- void free_ddpoly(DDPoly pol)
- double * get_ddpoly_i(DDPoly pol, long int i) (alias: gddpi)
- void set_ddpoly_i(DDPoly pol, long int i, double * val) (alias: sddpi)
- void set_ddpoly_i_si(DDPoly pol, long int i, long val)
- void set_ddpoly_i_ui(DDPoly pol, long int i, unsigned long val)
- void set_ddpoly_i_d(DDPoly pol, long int i, double val)
- void set_ddpoly_i_str(DDPoly pol, long int i, const char * str, int base)
- long int setdegree_ddpoly(DDPoly pol)
- void print_ddpoly(DDPoly pol)
- void add_ddpoly(DDPoly ret, DDPoly a, DDPoly b)
- void sub_ddpoly(DDPoly ret, DDPoly a, DDPoly b)
- void mul_ddpoly(DDPoly ret, DDPoly a, DDPoly b)
- void cmul_ddpoly(DDPoly ret, double * c, DDPoly a)
- void subst_ddpoly(DDPoly ret, DDPoly a)
- void set0_ddpoly(DDPoly pol)
- long int max_abscoef_ddpoly(DDPoly pol)
- long int num_nonzero_ddpoly(DDPoly pol)
- void diff_ddpoly(DDPoly pol)
- void eval_ddpoly_horner(double * ret, DDPoly a, double * x) (also aliased as eval_ddpoly)
- void eval_ddpoly_estrin(double * ret, DDPoly a, double * x)
- void eval_diff_ddpoly(double * ret, DDPoly a, double * x)
- void ceval_ddpoly_horner(cddfloat * ret, DDPoly a, cddfloat * x) (also aliased as ceval_ddpoly)
- void ceval_ddpoly_estrin(cddfloat * ret, DDPoly a, cddfloat * x)
- void ceval_diff_ddpoly(cddfloat * ret, DDPoly a, cddfloat * x)
- void _bncavx2_eval_ddpoly_estrin(double ret[DDSIZE], DDPoly a, double x[DDSIZE]) (AVX2)
- void _bncavx2_ceval_ddpoly_estrin(cddfloat * ret, DDPoly a, cddfloat * x) (AVX2)

Complex double-double polynomial (CDDPoly):

- CDDPoly `init_cddpoly`(long int `max_len`)
- void `free_cddpoly`(CDDPoly `pol`)
- CDDPoly `init_set_cddpoly`(CDDPoly `org`)
- CDDPoly `init_set_cddpoly_ddpoly`(DDPoly `org`)
- cddfloat * `get_cddpoly_i`(CDDPoly `pol`, long int `i`) (alias: `gcddpi`)
- void `set_cddpoly_i`(CDDPoly `pol`, long int `i`, cddfloat * `val`) (alias: `scddpi`)
- long int `setdegree_cddpoly`(CDDPoly `pol`)
- void `print_cddpoly`(CDDPoly `pol`)
- void `add_cddpoly`(CDDPoly `ret`, CDDPoly `a`, CDDPoly `b`)
- void `sub_cddpoly`(CDDPoly `ret`, CDDPoly `a`, CDDPoly `b`)
- void `mul_cddpoly`(CDDPoly `ret`, CDDPoly `a`, CDDPoly `b`)
- void `subst_cddpoly`(CDDPoly `ret`, CDDPoly `a`)
- void `set0_cddpoly`(CDDPoly `pol`)
- long int `max_abscoef_cddpoly`(CDDPoly `pol`)
- long int `num_nonzero_cddpoly`(CDDPoly `pol`)
- void `diff_cddpoly`(CDDPoly `pol`)
- void `eval_cddpoly_horner`(cddfloat * `ret`, CDDPoly `a`, cddfloat * `x`) (also aliased as `eval_cddpoly`)
- void `eval_cddpoly_estrin`(cddfloat * `ret`, CDDPoly `a`, cddfloat * `x`)
- void `eval_diff_cddpoly`(cddfloat * `ret`, CDDPoly `a`, cddfloat * `x`)
- void `_bncavx2_eval_cddpoly_estrin`(cddfloat * `ret`, CDDPoly `a`, cddfloat * `x`) (AVX2)

5.2.5 Triple-double polynomial functions (`poly.c` — `TDPoly`; requires `USE_TDLINEAR`)

- TDPoly `init_tdpoly`(long int `max_len`)
- void `free_tdpoly`(TDPoly `pol`)
- tdfloat `get_tdpoly_i_float`(TDPoly `pol`, long int `i`)
- double * `get_tdpoly_i`(TDPoly `pol`, long int `i`) (alias: `gtdpi`)
- void `set_tdpoly_i`(TDPoly `pol`, long int `i`, double `val[TDSIZE]`) (alias: `stdpi`)
- void `set_tdpoly_i_si`(TDPoly `pol`, long int `i`, long `val`)
- void `set_tdpoly_i_ui`(TDPoly `pol`, long int `i`, unsigned long `val`)
- void `set_tdpoly_i_d`(TDPoly `pol`, long int `i`, double `val`)
- void `set_tdpoly_i_str`(TDPoly `pol`, long int `i`, const char * `str`, int `base`)
- long int `setdegree_tdpoly`(TDPoly `pol`)
- void `print_tdpoly`(TDPoly `pol`)
- void `add_tdpoly`(TDPoly `c`, TDPoly `a`, TDPoly `b`) $\cdots$ $c = a + b$
- void `add2_tdpoly`(TDPoly `c`, TDPoly `a`) $\cdots$ $c \mathrel{+}= a$
- void `sub_tdpoly`(TDPoly `c`, TDPoly `a`, TDPoly `b`) $\cdots$ $c = a - b$
- void `sub2_tdpoly`(TDPoly `c`, TDPoly `a`) $\cdots$ $c \mathrel{-}= a$
- void `mul_tdpoly`(TDPoly `c`, TDPoly `a`, TDPoly `b`) $\cdots$ $c = a \times b$
- void `cmul_tdpoly`(TDPoly `c`, double `val[TDSIZE]`, TDPoly `a`) $\cdots$ $c = val \times a$
- void `cmul2_tdpoly`(TDPoly `c`, double `val[TDSIZE]`) $\cdots$ $c \mathrel{*}= val$
- void `subst_tdpoly`(TDPoly `c`, TDPoly `a`) $\cdots$ $c := a$
- void `set0_tdpoly`(TDPoly `c`) $\cdots$ $c := 0$
- long int `num_nonzero_tdpoly`(TDPoly `c`)
- long int `max_abscoef_tdpoly`(TDPoly `c`)
- void `diff_tdpoly`(TDPoly `a`) $\cdots$ $a := a'(x)$
- void `eval_tdpoly_horner`(double `ret[TDSIZE]`, TDPoly `a`, double `x[TDSIZE]`) (also aliased as `eval_tdpoly`)
- void `eval_tdpoly_estrin`(double `ret[TDSIZE]`, TDPoly `a`, double `x[TDSIZE]`)
- void `eval_diff_tdpoly`(double `ret[TDSIZE]`, TDPoly `a`, double `x[TDSIZE]`)
- void `ceval_tdpoly_horner`(ctdfloat * `ret`, TDPoly `a`, ctdfloat * `x`) (also aliased as `ceval_tdpoly`)

- void ceval_tdpoly_estrin(ctdfloat * ret, TDPoly a, ctdfloat * x)
- void ceval_diff_tdpoly(ctdfloat * ret, TDPoly a, ctdfloat * x)
- void _bncavx2_eval_tdpoly_estrin(double ret[TDSIZE], TDPoly a, double x[TDSIZE]) (AVX2)
- void _bncavx2_ceval_tdpoly_estrin(ctdfloat * ret, TDPoly a, ctdfloat * x) (AVX2)

5.2.6 Quadruple-double polynomial functions (poly.c — QDPoly; requires USE_QDLINEAR)

- QDPoly init_qdpoly(long int max_len)
- void free_qdpoly(QDPoly pol)
- qdfloat get_qdpoly_i_float(QDPoly pol, long int i)
- double * get_qdpoly_i(QDPoly pol, long int i) (alias: gqdpi)
- void set_qdpoly_i(QDPoly pol, long int i, double val[QDSIZE]) (alias: sqdpi)
- void set_qdpoly_i_si(QDPoly pol, long int i, long val)
- void set_qdpoly_i_ui(QDPoly pol, long int i, unsigned long val)
- void set_qdpoly_i_d(QDPoly pol, long int i, double val)
- void set_qdpoly_i_str(QDPoly pol, long int i, const char * str, int base)
- long int setdegree_qdpoly(QDPoly pol)
- void print_qdpoly(QDPoly pol)
- void add_qdpoly(QDPoly c, QDPoly a, QDPoly b) $\cdots$ $c = a + b$
- void add2_qdpoly(QDPoly c, QDPoly a) $\cdots$ $c \mathrel{+}= a$
- void sub_qdpoly(QDPoly c, QDPoly a, QDPoly b) $\cdots$ $c = a - b$
- void sub2_qdpoly(QDPoly c, QDPoly a) $\cdots$ $c \mathrel{-}= a$
- void mul_qdpoly(QDPoly c, QDPoly a, QDPoly b) $\cdots$ $c = a \times b$
- void cmul_qdpoly(QDPoly c, double val[QDSIZE], QDPoly a) $\cdots$ $c = val \times a$
- void cmul2_qdpoly(QDPoly c, double val[QDSIZE]) $\cdots$ $c \mathrel{*}= val$
- void subst_qdpoly(QDPoly c, QDPoly a) $\cdots$ $c := a$
- void set0_qdpoly(QDPoly c) $\cdots$ $c := 0$
- long int num_nonzero_qdpoly(QDPoly c)
- long int max_abscoef_qdpoly(QDPoly c)
- void diff_qdpoly(QDPoly a) $\cdots$ $a := a'(x)$
- void eval_qdpoly_horner(double ret[QDSIZE], QDPoly a, double x[QDSIZE]) (also aliased as eval_qdpoly)
- void eval_qdpoly_estrin(double ret[QDSIZE], QDPoly a, double x[QDSIZE])
- void eval_diff_qdpoly(double ret[QDSIZE], QDPoly a, double x[QDSIZE])
- void ceval_qdpoly_horner(cqdfloat * ret, QDPoly a, cqdfloat * x) (also aliased as ceval_qdpoly)
- void ceval_qdpoly_estrin(cqdfloat * ret, QDPoly a, cqdfloat * x)
- void ceval_diff_qdpoly(cqdfloat * ret, QDPoly a, cqdfloat * x)
- void ceval_hes_qdmatrix(cqdfloat * ret, QDMatrix hes, cqdfloat * x)
- void _bncavx2_eval_qdpoly_estrin(double ret[QDSIZE], QDPoly a, double x[QDSIZE]) (AVX2)
- void _bncavx2_ceval_qdpoly_estrin(cqdfloat * ret, QDPoly a, cqdfloat * x) (AVX2)

Conversion functions between polynomial types (requires USE_GMP):

- void set_ddpoly_i_mpf(DDPoly pol, long int i, mpf_t val)
- void set_tdpoly_i_mpf(TDPoly pol, long int i, mpf_t val)
- void set_qdpoly_i_mpf(QDPoly pol, long int i, mpf_t val)
- void set_cddpoly_i_mpf(CDDPoly pol, long int i, mpc_t val)
- DDPoly init_set_ddpoly_mpfpoly(MPFPoly org_pol)
- TDPoly init_set_tdpoly_mpfpoly(MPFPoly org_pol)
- QDPoly init_set_qdpoly_mpfpoly(MPFPoly org_pol)
- CDDPoly init_set_cddpoly_cmpfpoly(CMPFPoly org_pol)

5.2.7 Multiple-precision polynomial functions (poly.c — MPFPoly; requires USE_GMP)

- MPFPoly init_mpfpoly(long int max_len)
- MPFPoly init2_mpfpoly(long int max_len, unsigned long prec)
- void free_mpfpoly(MPFPoly pol)
- mpf_ptr get_mpfpoly_i(MPFPoly pol, long int i) (alias: gmpfpi)
- void set_mpfpoly_i(MPFPoly pol, long int i, mpf_t val) (alias: smpfpi)
- void set_mpfpoly_i_d(MPFPoly pol, long int i, double val)
- void set_mpfpoly_i_si(MPFPoly pol, long int i, long val)
- void set_mpfpoly_i_ui(MPFPoly pol, long int i, unsigned long val)
- void set_mpfpoly_i_str(MPFPoly pol, long int i, const char * str, int base)
- void print_mpfpoly(MPFPoly pol)
- void print_fdmpfpoly(FPoly fp, DPoly dp, MPFPoly mp)
- void add_mpfpoly(MPFPoly ret, MPFPoly a, MPFPoly b)
- void sub_mpfpoly(MPFPoly ret, MPFPoly a, MPFPoly b)
- void add2_mpfpoly(MPFPoly ret, MPFPoly a)
- void sub2_mpfpoly(MPFPoly ret, MPFPoly a)
- void mul_mpfpoly(MPFPoly ret, MPFPoly a, MPFPoly b)
- void cmul_mpfpoly(MPFPoly ret, mpf_t c, MPFPoly a)
- void subst_mpfpoly(MPFPoly ret, MPFPoly a)
- void set0_mpfpoly(MPFPoly pol)
- long int max_abscoef_mpfpoly(MPFPoly pol)
- long int num_nonzero_mpfpoly(MPFPoly pol)
- long int setdegree_mpfpoly(MPFPoly pol)
- void diff_mpfpoly(MPFPoly pol)
- void eval_mpfpoly_horner(mpf_t ret, MPFPoly pol, mpf_t x) (also aliased as eval_mpfpoly)
- void eval_mpfpoly_estrin(mpf_t ret, MPFPoly pol, mpf_t x)
- void eval_diff_mpfpoly(mpf_t ret, MPFPoly pol, mpf_t x)
- void ceval_mpfpoly_horner(mpc_t ret, MPFPoly pol, mpc_t x) (also aliased as ceval_mpfpoly)
- void ceval_mpfpoly_estrin(mpc_t ret, MPFPoly pol, mpc_t x)
- void ceval_diff_mpfpoly(mpc_t ret, MPFPoly pol, mpc_t x)
- void ceval_hes_mpfmatrix(mpc_t ret, MPFMatrix hes, mpc_t x)

Complex multiple-precision polynomial (CMPFPoly; requires USE_GMP):

- CMPFPoly init_cmpfpoly(long int max_len)
- CMPFPoly init2_cmpfpoly(long int max_len, unsigned long prec)
- void free_cmpfpoly(CMPFPoly pol)
- mpc_ptr get_cmpfpoly_i(CMPFPoly pol, long int i) (alias: gcmpfpi)
- void set_cmpfpoly_i(CMPFPoly pol, long int i, mpc_t val) (alias: scmpfpi)
- void set_cmpfpoly_i_cd(CMPFPoly pol, long int i, double _Complex val)
- void set_cmpfpoly_i_d(CMPFPoly pol, long int i, double val)
- void set_cmpfpoly_i_si_si(CMPFPoly pol, long int i, long re, long im)
- void set_cmpfpoly_i_si(CMPFPoly pol, long int i, long val)
- void set_cmpfpoly_i_ui(CMPFPoly pol, long int i, unsigned long val)
- void set_cmpfpoly_i_str(CMPFPoly pol, long int i, const char * str, int base)
- long int setdegree_cmpfpoly(CMPFPoly pol)
- void print_cmpfpoly(CMPFPoly pol)
- void subst_cmpfpoly_mpfpoly(CMPFPoly ret, MPFPoly pol)
- CMPFPoly init_set_cmpfpoly_mpfpoly(MPFPoly org)
- CMPFPoly init_set_cmpfpoly(CMPFPoly org)
- void add_cmpfpoly(CMPFPoly ret, CMPFPoly a, CMPFPoly b)
- void sub_cmpfpoly(CMPFPoly ret, CMPFPoly a, CMPFPoly b)

- void add2_cmpfpoly(CMPFPoly ret, CMPFPoly a)
- void sub2_cmpfpoly(CMPFPoly ret, CMPFPoly a)
- void mul_cmpfpoly(CMPFPoly ret, CMPFPoly a, CMPFPoly b)
- void cmul_cmpfpoly(CMPFPoly ret, mpc_t c, CMPFPoly a)
- void subst_cmpfpoly(CMPFPoly ret, CMPFPoly a)
- void set0_cmpfpoly(CMPFPoly pol)
- long int max_abscoef_cmpfpoly(CMPFPoly pol)
- long int num_nonzero_cmpfpoly(CMPFPoly pol)
- void diff_cmpfpoly(CMPFPoly pol)
- void eval_cmpfpoly_horner(mpc_t ret, CMPFPoly pol, mpc_t x) (also aliased as eval_cmpfpoly)
- void eval_cmpfpoly_estrin(mpc_t ret, CMPFPoly pol, mpc_t x)
- void eval_diff_cmpfpoly(mpc_t ret, CMPFPoly pol, mpc_t x)
- void ceval_cmpfpoly_horner(mpc_t ret, CMPFPoly pol, mpc_t x)
- void ceval_cmpfpoly_estrin(mpc_t ret, CMPFPoly pol, mpc_t x)
- void ceval_diff_cmpfpoly(mpc_t ret, CMPFPoly pol, mpc_t x)
- void ceval_hes_cmpfmatrix(mpc_t ret, CMPFMatrix hes, mpc_t x)

5.2.8 Array functions (array.c)

- FArray init_farray(long int size)
- void free_farray(FArray a)
- float get_farray_i(FArray a, long int i) (alias: gfai)
- void set_farray_i(FArray a, long int i, float val) (alias: sfai)
- void subst_farray(FArray ret, FArray src)
- void print_farray(FArray a)
- CFArray init_cfarray(long int size)
- void free_cfarray(CFArray a)
- FCmplx get_cfarray_i(CFArray a, long int i) (alias: gcfai)
- void set_cfarray_i(CFArray a, long int i, FCmplx val) (alias: scfai)
- void subst_cfarray(CFArray ret, CFArray src)
- void print_cfarray(CFArray a)
- DArray init_darray(long int size)
- void free_darray(DArray a)
- double get_darray_i(DArray a, long int i) (alias: gdai)
- void set_darray_i(DArray a, long int i, double val) (alias: sdai)
- void subst_darray(DArray ret, DArray src)
- void print_darray(DArray a)
- CDArray init_cdarray(long int size)
- void free_cdarray(CDArray a)
- DCmplx get_cdarray_i(CDArray a, long int i) (alias: gcdai)
- void set_cdarray_i(CDArray a, long int i, DCmplx val) (alias: scdai)
- void subst_cdarray(CDArray ret, CDArray src)
- void print_cdarray(CDArray a)

Multiple-precision array functions (requires USE_GMP):

- MPFArray init_mpfarray(long int size)
- MPFArray init2_mpfarray(long int size, unsigned long prec)
- void free_mpfarray(MPFArray a)
- mpf_ptr get_mpfarray_i(MPFArray a, long int i) (alias: gmpfai)
- void set_mpfarray_i(MPFArray a, long int i, mpf_t val) (alias: smpfai)
- void set_mpfarray_i_d(MPFArray a, long int i, double val)
- void set_mpfarray_i_ui(MPFArray a, long int i, unsigned long val)

- void subst_mpfarray(MPFArray ret, MPFArray src)
- void print_mpfarray(MPFArray a)
- CMPFArray init_cmpfarray(long int size)
- CMPFArray init2_cmpfarray(long int size, unsigned long prec)
- void free_cmpfarray(CMPFArray a)
- mpc_ptr get_cmpfarray_i(CMPFArray a, long int i) (alias: gcmpfai)
- void set_cmpfarray_i(CMPFArray a, long int i, mpc_t val) (alias: scmpfai)
- void set_cmpfarray_i_d(CMPFArray a, long int i, double val)
- void set_cmpfarray_i_ui(CMPFArray a, long int i, unsigned long val)
- void set_cmpfarray_i_real(CMPFArray a, long int i, mpf_t val)
- void subst_cmpfarray(CMPFArray ret, CMPFArray src)
- void print_cmpfarray(CMPFArray a)

5.2.9 DKA root-finding methods (dka.c)

The Durand–Kerner–Aberth (DKA) method simultaneously finds all roots of a polynomial.

- float fdka_center(FPoly pol)
- float fdka_radius(FPoly pol)
- void fdka_init(CFArray x, FPoly pol)
- long int fdka(CFArray ans, CFArray x_init, FPoly pol, long int maxtimes, float aeps, float reps)
- double ddka_center(DPoly pol)
- double _Complex cddka_center(CDPoly pol)
- double ddka_radius(DPoly pol)
- double cddka_radius(CDPoly pol)
- void ddka_init(CDArray x, DPoly pol)
- void cddka_init(double _Complex[] x, CDPoly pol)
- void cddka_init_cdarray(CDArray x, CDPoly pol)
- long int ddka(CDArray ans, CDArray x_init, DPoly pol, long int maxtimes, double aeps, double reps)
- long int cddka(double _Complex[] ans, double _Complex[] x_init, CDPoly pol, long int maxtimes, double aeps, double reps)

Double-double DKA (dd_dka.c):

- void dd_dka_center(double ret[DDSIZE], DDPoly pol)
- void dd_dka_radius(double ret[DDSIZE], DDPoly pol)
- void dd_dka_init(CDDVector x, DDPoly pol)
- long int dd_dka(CDDVector ans, CDDVector x_init, DDPoly pol, long int maxtimes, double * aeps, double * reps)

Triple-double DKA (td_dka.c):

- void td_dka_center(double ret[TDSIZE], TDPoly pol)
- void td_dka_radius(double ret[TDSIZE], TDPoly pol)
- void td_dka_init(CTDVector x, TDPoly pol)
- long int td_dka(CTDVector ans, CTDVector x_init, TDPoly pol, long int maxtimes, double * aeps, double * reps)

Quadruple-double DKA (qd_dka.c):

- void qd_dka_center(double ret[QDSIZE], QDPoly pol)
- void qd_dka_radius(double ret[QDSIZE], QDPoly pol)
- void qd_dka_init(CQDVector x, QDPoly pol)
- long int qd_dka(CQDVector ans, CQDVector x_init, QDPoly pol, long int maxtimes, double * aeps, double * reps)

- long int qd_dka_mod(CQDVector ans, CQDVector x_init, QDPoly pol, long int maxtimes, double aeps[QDSIZE], double reps[QDSIZE])
- long int qd_petckovic(CQDVector ans, CQDVector x_init, QDPoly pol, long int maxtimes, double aeps[QDSIZE], double reps[QDSIZE])
- long int qd_aberth(CQDVector ans, CQDVector x_init, QDPoly pol, long int maxtimes, double aeps[QDSIZE], double reps[QDSIZE])

Multiple-precision DKA (mpf_dka.c; requires USE_GMP):

- void mpf_dka_center(mpf_t ret, MPFPoly pol)
- void mpf_dka_radius(mpf_t ret, MPFPoly pol)
- void mpf_dka_ozawa_radius(mpf_t ret, MPFPoly pol) $\cdots$ Ozawa initial radius: $r := (p(\text{center})/a_n)^{1/n}$
- void mpf_dka_init(CMPFArray x, MPFPoly pol)
- void mpf_dka_init2(CMPFArray x_init, MPFPoly pol, void (*) get_radius, void (*) get_center)

- long int mpf_dka(CMPFArray ans, CMPFArray x_init, MPFPoly pol, long int maxtimes, mpf_t aeps, mpf_t reps)
- long int mpf_dka_mod(CMPFArray ans, CMPFArray x_init, MPFPoly pol, long int maxtimes, mpf_t aeps, mpf_t reps) $\cdots$ DKA method, 2nd order
- long int mpf_petckovic(CMPFArray ans, CMPFArray x_init, MPFPoly pol, long int maxtimes, mpf_t aeps, mpf_t reps) $\cdots$ Petkovic method, 3rd order
- long int mpf_aberth(CMPFArray ans, CMPFArray x_init, MPFPoly pol, long int maxtimes, mpf_t aeps, mpf_t reps) $\cdots$ Aberth method, 3rd order

MPC complex multiple-precision DKA (mpc_dka.c; requires USE_GMP):

- void mpc_dka_center(mpc_t ret, CMPFPoly pol)
- void mpc_dka_radius(mpf_t ret, CMPFPoly pol)
- void mpc_dka_ozawa_radius(mpf_t ret, CMPFPoly pol)
- void mpc_dka_init(CMPFArray x, CMPFPoly pol)
- void mpc_dka_init2(CMPFArray x_init, CMPFPoly pol, void (*) get_radius, void (*) get_center)
- long int mpc_dka(CMPFArray ans, CMPFArray x_init, CMPFPoly pol, long int maxtimes, mpf_t aeps, mpf_t reps)
- long int mpc_dka_mod(CMPFArray ans, CMPFArray x_init, CMPFPoly pol, long int maxtimes, mpf_t aeps, mpf_t reps)
- long int mpc_petckovic(CMPFArray ans, CMPFArray x_init, CMPFPoly pol, long int maxtimes, mpf_t aeps, mpf_t reps)
- long int mpc_aberth(CMPFArray ans, CMPFArray x_init, CMPFPoly pol, long int maxtimes, mpf_t aeps, mpf_t reps)

5.2.10 Hirano method (hirano.c, dd_hirano.c)

The Hirano method finds a root x satisfying $|p(x)| < \max_j |a_j x^j| \cdot r_\varepsilon + a_\varepsilon$.

- double _Complex dhorner(double _Complex x, DPoly pol) $\cdots$ Horner evaluation of $p(x)$
- double _Complex cdhorner(double _Complex x, CDPoly pol)
- void dcoef_horner(double _Complex[] ret, double _Complex x, DPoly pol) $\cdots$ coefficients of $p(x+d)$
- void cdcoef_horner(double _Complex[] ret, double _Complex x, CDPoly pol)
- long int absmax_dpoly(double * absmax, double _Complex x, DPoly pol)
- long int absmax_cdpoly(double * absmax, double _Complex x, CDPoly pol)
- double dget_plus_arg(double _Complex x) $\cdots$ $\arg(x) \in [0, 2\pi]$
- long int dget_nearest_int(double x)

- double _Complex dget_min_branch(double _Complex x, double mu, double _Complex[] coef, long int i_num, long int i_den)
- long int dhirano(double _Complex * ret, double _Complex init_x, DPoly pol, double reps, double aeps, long int maxtimes)
- long int cdhirano(double _Complex * ret, double _Complex init_x, CDPoly pol, double reps, double aeps, long int maxtimes)

Multiple-precision Hirano (hirano.c; requires USE_GMP):

- void mpf_coef_horner(CMPFArray ret, mpc_t x, MPFPoly pol)
- void mpc_coef_horner(CMPFArray ret, mpc_t x, CMPFPoly pol)
- long int absmax_mpfpoly(mpf_t absmax, mpc_t x, MPFPoly pol)
- long int absmax_cmpfpoly(mpf_t absmax, mpc_t x, CMPFPoly pol)
- void mpf_get_plus_arg(mpf_t ret, mpc_t x)
- void mpf_get_nearest_int(mpf_t ret, mpf_t x)
- void mpf_get_min_branch(mpc_t ret, mpc_t x, mpf_t mu, CMPFArray coef, long int i_num, long int i_den)
- long int mpf_hirano(mpc_t ret, mpc_t init_x, MPFPoly pol, mpf_t reps, mpf_t aeps, long int maxtimes)
- long int mpc_hirano(mpc_t ret, mpc_t init_x, CMPFPoly pol, mpf_t reps, mpf_t aeps, long int maxtimes)
- void ono_poly(MPFPoly pol, long int deg) $\cdots$ Ono's problem polynomial
- void wilkinson_poly(MPFPoly ret, long int n) $\cdots$ Wilkinson polynomial $(x-1)(x-2)\cdots(x-n)$
- void deflation_cmpfpoly(CMPFPoly ret, CMPFPoly pol, mpc_t root) $\cdots$ deflation $p(x)/(x-r)$

Double-double Hirano (dd_hirano.c; requires USE_DDLINEAR):

- void dd_horner(cddfloat * ret, cddfloat * x, DDPoly pol)
- void dd_coef_horner(CDDVector ret, cddfloat * x, DDPoly pol)
- void cdd_horner(cddfloat * ret, cddfloat * x, CDDPoly pol)
- void cdd_coef_horner(CDDVector ret, cddfloat * x, CDDPoly pol)
- long int absmax_ddpoly(double absmax[DDSIZE], cddfloat * x, DDPoly pol)
- long int absmax_cddpoly(double absmax[DDSIZE], cddfloat * x, CDDPoly pol)
- void dd_get_plus_arg(double ret[DDSIZE], cddfloat * x)
- void dd_get_nearest_int(double ret[DDSIZE], double x[DDSIZE])
- void dd_get_min_branch(cddfloat * ret, cddfloat * x, double mu[DDSIZE], CDDVector coef, long int i_num, long int i_den)
- long int dd_hirano(cddfloat * ret, cddfloat * init_x, DDPoly pol, double reps[DDSIZE], double aeps[DDSIZE], long int maxtimes)
- long int cdd_hirano(cddfloat * ret, cddfloat * init_x, CDDPoly pol, double reps[DDSIZE], double aeps[DDSIZE], long int maxtimes)
- void deflation_cddpoly(CDDPoly ret, CDDPoly pol, cddfloat * root)

5.2.11 File I/O functions (fread_write_complex.c; requires USE_GMP)

- void fread_cmpfarray(FILE * fp, CMPFArray array)
- void fread_cmpfarray_fname(const char * fname, CMPFArray array)
- void fwrite_cmpfarray(FILE * fp, CMPFArray array)
- void fwrite_cmpfarray_fname(const char * fname, CMPFArray array)
- void fread_mpfpolycoef(FILE * fp, MPFPoly pol, long int maxdeg)

Chapter 6

Sparse matrix and related functions

This chapter describes the sparse matrix types and associated functions provided by the library. Sparse matrices are stored in Compressed Row Storage (CRS) format. For each matrix type, the library provides real and complex variants at double, double-double (DD), triple-double (TD), quadruple-double (QD), and multiple-precision (MPF, via GMP) precisions.

6.1 Structures of CRS matrix

6.1.1 `DRSMatrix` — double-precision sparse matrix

The `drsmatrix` structure stores a real double-precision sparse matrix in CRS format. Its pointer type is `DRSMatrix`. Key members are:

- `double *element` — array of non-zero elements
- `long int row_dim, col_dim` — row and column dimensions
- `long int **nzero_index` — column indices of non-zero elements
- `long int *nzero_col_dim` — number of non-zero elements in each row
- `long int *nzero_row_dim` — number of non-zero elements in each column
- `long int nzero_total_num` — total number of non-zero elements
- `long int real_nzero_total_num` — SIMD-aligned total; equals $\sum_i$ `real_nzero_col_dim`$[i]$
- `long int *real_nzero_col_dim` — SIMD-aligned row non-zero counts (padded to `_BNC_D_WIDTH`)
- `double zero_element` — zero sentinel value

Functions for `DRSMatrix`:

- DRSMatrix `init_drsmatrix`(long int `row_dim`, long int * `nzero_col_dim`, long int `nzero_total_num`)

- void `free_drsmatrix`(DRSMatrix `mat`)
- void `set_nzero_row_dim`(DRSMatrix `mat`)
- void `print_drsmatrix`(DRSMatrix `mat`)
- void `set_dmatrix_drsmatrix`(DMatrix `ret`, DRSMatrix `spmat`) $\cdots$ Dense := Sparse
- DRSMatrix `init_set_drsmatrix_dmatrix`(DMatrix `org`)
- double `get_drsmatrix_ij`(DRSMatrix `mat`, long int `i`, long int `j`)
- void `set_drsmatrix_ij`(DRSMatrix `mat`, long int `i`, long int `j`, double `val`)
- DRSMatrix `init_set_drsmatrix`(DRSMatrix `org`)
- void `set0_drsmatrix`(DRSMatrix `spmat`)
- int `get_vars_drsmatrix_fname`(long int * `row_dim`, long int ** `nzero_col_dim`, long int * `nzero_total_num`, const char * `fname`)
- int `fread_urilinkdat_fname`(DRSMatrix `mat`, const char * `fname`)
- int `mul_drsmatrix_dvec`(DVector `ret`, DRSMatrix `mat`, DVector `vec`)
- int `mul_drsmatrixt_dvec`(DVector `ret`, DRSMatrix `mat`, DVector `vec`) $\cdots$ transposed
- void `mul_drsmatrix_dvec_oz`(DVector `ret`, DRSMatrix `a`, int `max_num_div_a`, DVector `vb`, int `max_num_div_vb`) $\cdots$ Ozaki scheme

- void `mul_drsmatrixt_dvec_oz`(DVector `ret`, DRSMatrix `a`, int `max_num_div_a`, DVector `vb`, int `max_num_div_vb`) $\cdots$ transposed, Ozaki scheme
- double `dpower_rsmatrix`(DVector `evec`, DRSMatrix `mat`, double `reps`, double `aeps`, long int `maxtimes`) $\cdots$ power method
- int `smul_dvector`(DVector `ret`, double `c`, DVector `vec`)
- long int `absmax_index_dvector`(double * `absmax`, DVector `vec`)
- void `subst_drsmatrix`(DRSMatrix `ret`, DRSMatrix `a`) $\cdots$ $ret := a$
- void `add_drsmatrix`(DRSMatrix `ret`, DRSMatrix `a`, DRSMatrix `b`) $\cdots$ $ret := a + b$
- void `sub_drsmatrix`(DRSMatrix `ret`, DRSMatrix `a`, DRSMatrix `b`) $\cdots$ $ret := a - b$
- void `cmul_drsmatrix`(DRSMatrix `ret`, double `c`, DRSMatrix `a`) $\cdots$ $ret := c \cdot a$
- double `absmax_row_drsmatrix`(long int * `max_j`, long int `row`, DRSMatrix `mat`)
- double `absmax_col_drsmatrix`(long int * `max_i`, long int `col`, DRSMatrix `mat`)
- double `normf_drsmatrix`(DRSMatrix `mat`) $\cdots$ Frobenius norm
- void `split_drsmatrix`(DRSMatrix `high`, DRSMatrix `low`, DRSMatrix `org`) $\cdots$ SplitMat_A
- int `split_drsmatrix_drsmat`(DRSMatrix[] `ret`, int `num_div`, DRSMatrix `org`) $\cdots$ SplitMat_A (multiple parts)
- int `split_drsmatrix_t_drsmat`(DRSMatrix[] `ret`, int `num_div`, DRSMatrix `org`) $\cdots$ SplitMat_B
- void `iLU0_drsmatrix`(DRSMatrix `mat`) $\cdots$ incomplete LU decomposition (iLU0)
- void `solve_iLU0_drsmatrix`(DVector `ret`, DRSMatrix `ilu`, DVector `b`) $\cdots$ solve (iLU)$\mathbf{x} = \mathbf{b}$
- void `solve_iLU0t_drsmatrix`(DVector `ret`, DRSMatrix `ilu`, DVector `b`) $\cdots$ solve $\mathbf{x}^T(\text{iLU}) = \mathbf{b}^T$

6.1.2 `CDRSMatrix` — complex double-precision sparse matrix

The `cdrsmatrix` structure stores a complex sparse matrix as a pair of real `DRSMatrix` parts. Its pointer type is `CDRSMatrix`. Members: `DRSMatrix re` (real part), `DRSMatrix im` (imaginary part).

- CDRSMatrix `init_cdrsmatrix`(long int `row_dim`, long int * `nzero_col_dim`, long int `nzero_total_num`)

- void `free_cdrsmatrix`(CDRSMatrix `mat`)
- void `print_cdrsmatrix`(CDRSMatrix `mat`)
- void `set_cdmatrix_cdrsmatrix`(CDMatrix `ret`, CDRSMatrix `spmat`)
- double `normf_cdrsmatrix`(CDRSMatrix `mat`) $\cdots$ Frobenius norm
- CDRSMatrix `init_set_cdrsmatrix_cdmatrix`(CDMatrix `org`)
- CDRSMatrix `init_set_cdrsmatrix`(CDRSMatrix `org`)
- double _Complex `get_cdrsmatrix_ij`(CDRSMatrix `mat`, long int `i`, long int `j`)
- void `set_cdrsmatrix_ij`(CDRSMatrix `mat`, long int `i`, long int `j`, double _Complex `val`)
- int `mul_cdrsmatrix_cdvec`(CDVector `ret`, CDRSMatrix `mat`, CDVector `vec`)
- int `mul_cdrsmatrixt_cdvec`(CDVector `ret`, CDRSMatrix `mat`, CDVector `vec`) $\cdots$ transposed
- int `mul_cdrsmatrixs_cdvec`(CDVector `ret`, CDRSMatrix `mat`, CDVector `vec`) $\cdots$ conjugate-transposed
- void `iLU0_cdrsmatrix`(CDRSMatrix `mat`)
- void `solve_iLU0_cdrsmatrix`(CDVector `ret`, CDRSMatrix `ilu`, CDVector `b`)
- void `solve_iLU0t_cdrsmatrix`(CDVector `ret`, CDRSMatrix `ilu`, CDVector `b`)
- void `solve_iLU0s_cdrsmatrix`(CDVector `ret`, CDRSMatrix `ilu`, CDVector `b`)

6.1.3 `DDRSMatrix` and `CDDRSMatrix` — double-double sparse matrices (requires `USE_DDLINEAR`)

The `ddrsmatrix` structure stores a double-double sparse matrix; pointer type `DDRSMatrix`. Its elements are stored as `double *element[DDSIZE]`. The complex variant `CDDRSMatrix` holds a pair of `DDRSMatrix` parts.

- DDRSMatrix `init_ddrsmatrix`(long int `row_dim`, long int * `nzero_col_dim`, long int `nzero_total_num`)

- void `free_ddrsmatrix`(DDRSMatrix `mat`)
- void `set_nzero_row_dim_dd`(DDRSMatrix `mat`)
- void `print_ddrsmatrix`(DDRSMatrix `mat`)
- void `normf_ddrsmatrix`(double `ret[DDSIZE]`, DDRSMatrix `mat`)
- DDRSMatrix `init_set_ddrsmatrix_ddmatrix`(DDMatrix `org`)
- void `get_ddrsmatrix_ij`(double `ret[DDSIZE]`, DDRSMatrix `mat`, long int `i`, long int `j`)
- void `set_ddrsmatrix_ij`(DDRSMatrix `mat`, long int `i`, long int `j`, double `val[DDSIZE]`)
- DDRSMatrix `init_set_ddrsmatrix`(DDRSMatrix `org`)
- void `set0_ddrsmatrix`(DDRSMatrix `spmat`)
- void `subst_ddrsmatrix`(DDRSMatrix `ret`, DDRSMatrix `a`)
- DRSMatrix `init_set_drsmatrix_ddrsmat`(DDRSMatrix `org`)
- int `get_vars_ddrsmatrix_fname`(long int * `row_dim`, long int ** `nzero_col_dim`, long int * `nzero_total_num`, const char * `fname`)
- int `mul_ddrsmatrix_ddvec`(DDVector `ret`, DDRSMatrix `mat`, DDVector `vec`)
- int `mul_ddrsmatrixt_ddvec`(DDVector `ret`, DDRSMatrix `mat`, DDVector `vec`)
- int `mul_drsmatrix_ddvec`(DDVector `ret`, DRSMatrix `mat`, DDVector `vec`)
- int `mul_drsmatrixt_ddvec`(DDVector `ret`, DRSMatrix `mat`, DDVector `vec`)
- void `ddpower_sp`(double `max_eig[DDSIZE]`, DDVector `evec`, DDRSMatrix `mat`, double `reps[DDSIZE]`, double `aeps[DDSIZE]`, long int `maxtimes`)
- long int `absmax_index_ddvector`(double `ret[DDSIZE]`, DDVector `vec`)
- void `subst_ddrsmatrix_drsmat`(DDRSMatrix `c`, DRSMatrix `a`) $\cdots$ $c := (\mathrm{DD})a$
- void `subst_drsmatrix_ddrsmat`(DRSMatrix `c`, DDRSMatrix `a`) $\cdots$ $c := (\mathrm{D})a$
- void `absmax_row_ddrsmatrix`(double `mu[DDSIZE]`, long int * `max_j`, long int `row`, DDRSMatrix `mat`)

- void `absmax_col_ddrsmatrix`(double `mu[DDSIZE]`, long int * `max_i`, long int `col`, DDRSMatrix `mat`)

- void `add_ddrsmatrix_drsmat`(DDRSMatrix `c`, DDRSMatrix `a`, DRSMatrix `b`)
- void `sub_ddrsmatrix_drsmat`(DDRSMatrix `c`, DDRSMatrix `a`, DRSMatrix `b`)
- int `split_ddrsmatrix_drsmat`(DRSMatrix[] `ret`, int `num_div`, DDRSMatrix `org`)
- int `split_ddrsmatrix_t_drsmat`(DRSMatrix[] `ret`, int `num_div`, DDRSMatrix `org`)
- void `mul_ddrsmatrix_ddvec_oz`(DDVector `ret`, DDRSMatrix `a`, int `max_num_div_a`, DDVector `vb`, int `max_num_div_vb`)
- void `mul_ddrsmatrixt_ddvec_oz`(DDVector `ret`, DDRSMatrix `a`, int `max_num_div_a`, DDVector `vb`, int `max_num_div_vb`)
- void `iLU0_ddrsmatrix`(DDRSMatrix `mat`)
- void `solve_iLU0_ddrsmatrix`(DDVector `ret`, DDRSMatrix `ilu`, DDVector `b`)
- void `solve_iLU0t_ddrsmatrix`(DDVector `ret`, DDRSMatrix `ilu`, DDVector `b`)
- void `solve_iLU0_drsmatrix_ddvec`(DDVector `ret`, DRSMatrix `ilu`, DDVector `b`)
- void `solve_iLU0t_drsmatrix_ddvec`(DDVector `ret`, DRSMatrix `ilu`, DDVector `b`)

Complex DD sparse matrix (`CDDRSMatrix`):

- CDDRSMatrix `init_cddrsmatrix`(long int `row_dim`, long int * `nzero_col_dim`, long int `nzero_total_num`)
- void `free_cddrsmatrix`(CDDRSMatrix `mat`)
- void `print_cddrsmatrix`(CDDRSMatrix `mat`)
- void `normf_cddrsmatrix`(double `ret[DDSIZE]`, CDDRSMatrix `mat`)
- CDDRSMatrix `init_set_cddrsmatrix`(CDDRSMatrix `org`)
- int `mul_cddrsmatrix_cddvec`(CDDVector `ret`, CDDRSMatrix `mat`, CDDVector `vec`)
- int `mul_cddrsmatrixt_cddvec`(CDDVector `ret`, CDDRSMatrix `mat`, CDDVector `vec`)
- int `mul_cddrsmatrixs_cddvec`(CDDVector `ret`, CDDRSMatrix `mat`, CDDVector `vec`)
- int `mul_cdrsmatrix_cddvec`(CDDVector `ret`, CDRSMatrix `mat`, CDDVector `vec`)

- int mul_cdrsmatrixt_cddvec(CDDVector ret, CDRSMatrix mat, CDDVector vec)
- int mul_cdrsmatrixs_cddvec(CDDVector ret, CDRSMatrix mat, CDDVector vec)
- void iLU0_cddrsmatrix(CDDRSMatrix mat)
- void solve_iLU0_cddrsmatrix(CDDVector ret, CDDRSMatrix ilu, CDDVector b)
- void solve_iLU0t_cddrsmatrix(CDDVector ret, CDDRSMatrix ilu, CDDVector b)
- void solve_iLU0s_cddrsmatrix(CDDVector ret, CDDRSMatrix ilu, CDDVector b)
- void solve_iLU0_cdrsmatrix_cddvec(CDDVector ret, CDRSMatrix ilu, CDDVector b)
- void solve_iLU0t_cdrsmatrix_cddvec(CDDVector ret, CDRSMatrix ilu, CDDVector b)
- void solve_iLU0s_cdrsmatrix_cddvec(CDDVector ret, CDRSMatrix ilu, CDDVector b)

6.1.4 TDRSMatrix and CTDRSMatrix — triple-double sparse matrices (requires USE_TDLINEAR)

The tdrsmatrix structure stores a triple-double sparse matrix; pointer type TDRSMatrix. Elements are stored as double *element[TDSIZE].

- TDRSMatrix init_tdrsmatrix(long int row_dim, long int * nzero_col_dim, long int nzero_total_num)

- void free_tdrsmatrix(TDRSMatrix mat)
- void set_nzero_row_dim_td(TDRSMatrix mat)
- void print_tdrsmatrix(TDRSMatrix mat)
- void get_tdrsmatrix_ij(double ret[TDSIZE], TDRSMatrix mat, long int i, long int j)
- void set_tdrsmatrix_ij(TDRSMatrix mat, long int i, long int j, double val[TDSIZE])
- TDRSMatrix init_set_tdrsmatrix(TDRSMatrix org)
- void set0_tdrsmatrix(TDRSMatrix spmat)
- void subst_tdrsmatrix(TDRSMatrix ret, TDRSMatrix a)
- DRSMatrix init_set_drsmatrix_tdrsmat(TDRSMatrix org)
- TDRSMatrix init_set_tdrsmatrix_tdmatrix(TDMatrix org)
- int get_vars_tdrsmatrix_fname(long int * row_dim, long int ** nzero_col_dim, long int * nzero_total_num, const char * fname)
- int mul_tdrsmatrix_tdvec(TDVector ret, TDRSMatrix mat, TDVector vec)
- int mul_tdrsmatrixt_tdvec(TDVector ret, TDRSMatrix mat, TDVector vec)
- int mul_drsmatrix_tdvec(TDVector ret, DRSMatrix mat, TDVector vec)
- int mul_drsmatrixt_tdvec(TDVector ret, DRSMatrix mat, TDVector vec)
- void tdpower_sp(double max_eig[TDSIZE], TDVector evec, TDRSMatrix mat, double reps[TDSIZE], double aeps[TDSIZE], long int maxtimes)
- long int absmax_index_tdvector(double ret[TDSIZE], TDVector vec)
- void subst_tdrsmatrix_drsmat(TDRSMatrix c, DRSMatrix a)
- void subst_drsmatrix_tdrsmat(DRSMatrix c, TDRSMatrix a)
- void absmax_row_tdrsmatrix(double mu[TDSIZE], long int * max_j, long int row, TDRSMatrix mat)

- void absmax_col_tdrsmatrix(double mu[TDSIZE], long int * max_i, long int col, TDRSMatrix mat)

- void add_tdrsmatrix_drsmat(TDRSMatrix c, TDRSMatrix a, DRSMatrix b)
- void sub_tdrsmatrix_drsmat(TDRSMatrix c, TDRSMatrix a, DRSMatrix b)
- int split_tdrsmatrix_drsmat(DRSMatrix[] ret, int num_div, TDRSMatrix org)
- int split_tdrsmatrix_t_drsmat(DRSMatrix[] ret, int num_div, TDRSMatrix org)
- void mul_tdrsmatrix_tdvec_oz(TDVector ret, TDRSMatrix a, int max_num_div_a, TDVector vb, int max_num_div_vb)
- void mul_tdrsmatrixt_tdvec_oz(TDVector ret, TDRSMatrix a, int max_num_div_a, TDVector vb, int max_num_div_vb)
- void iLU0_tdrsmatrix(TDRSMatrix mat)
- void solve_iLU0_tdrsmatrix(TDVector ret, TDRSMatrix ilu, TDVector b)
- void solve_iLU0t_tdrsmatrix(TDVector ret, TDRSMatrix ilu, TDVector b)

- void solve_iLU0_drsmatrix_tdvec(TDVector ret, DRSMatrix ilu, TDVector b)
- void solve_iLU0t_drsmatrix_tdvec(TDVector ret, DRSMatrix ilu, TDVector b)

Complex TD sparse matrix (CTDRSMatrix):

- CTDRSMatrix init_ctdrsmatrix(long int row_dim, long int * nzero_col_dim, long int nzero_total_num)
- void free_ctdrsmatrix(CTDRSMatrix mat)
- void print_ctdrsmatrix(CTDRSMatrix mat)
- CTDRSMatrix init_set_ctdrsmatrix(CTDRSMatrix org)
- int mul_ctdrsmatrix_ctdvec(CTDVector ret, CTDRSMatrix mat, CTDVector vec)
- int mul_ctdrsmatrixt_ctdvec(CTDVector ret, CTDRSMatrix mat, CTDVector vec)
- int mul_ctdrsmatrixs_ctdvec(CTDVector ret, CTDRSMatrix mat, CTDVector vec)
- int mul_cdrsmatrix_ctdvec(CTDVector ret, CDRSMatrix mat, CTDVector vec)
- int mul_cdrsmatrixt_ctdvec(CTDVector ret, CDRSMatrix mat, CTDVector vec)
- int mul_cdrsmatrixs_ctdvec(CTDVector ret, CDRSMatrix mat, CTDVector vec)
- void iLU0_ctdrsmatrix(CTDRSMatrix mat)
- void solve_iLU0_ctdrsmatrix(CTDVector ret, CTDRSMatrix ilu, CTDVector b)
- void solve_iLU0t_ctdrsmatrix(CTDVector ret, CTDRSMatrix ilu, CTDVector b)
- void solve_iLU0s_ctdrsmatrix(CTDVector ret, CTDRSMatrix ilu, CTDVector b)
- void solve_iLU0_cdrsmatrix_ctdvec(CTDVector ret, CDRSMatrix ilu, CTDVector b)
- void solve_iLU0t_cdrsmatrix_ctdvec(CTDVector ret, CDRSMatrix ilu, CTDVector b)
- void solve_iLU0s_cdrsmatrix_ctdvec(CTDVector ret, CDRSMatrix ilu, CTDVector b)

6.1.5 QDRSMatrix and CQDRSMatrix — quadruple-double sparse matrices (requires USE_QDLINEAR)

The qdrsmatrix structure stores a quadruple-double sparse matrix; pointer type QDRSMatrix. Elements are stored as double *element[QDSIZE].

- QDRSMatrix init_qdrsmatrix(long int row_dim, long int * nzero_col_dim, long int nzero_total_num)

- void free_qdrsmatrix(QDRSMatrix mat)
- void set_nzero_row_dim_qd(QDRSMatrix mat)
- void print_qdrsmatrix(QDRSMatrix mat)
- QDRSMatrix init_set_qdrsmatrix_qdmatrix(QDMatrix org)
- void get_qdrsmatrix_ij(double ret[QDSIZE], QDRSMatrix mat, long int i, long int j)
- void set_qdrsmatrix_ij(QDRSMatrix mat, long int i, long int j, double val[QDSIZE])
- QDRSMatrix init_set_qdrsmatrix(QDRSMatrix org)
- void set0_qdrsmatrix(QDRSMatrix spmat)
- void subst_qdrsmatrix(QDRSMatrix ret, QDRSMatrix a)
- DRSMatrix init_set_drsmatrix_qdrsmat(QDRSMatrix org)
- int get_vars_qdrsmatrix_fname(long int * row_dim, long int ** nzero_col_dim, long int * nzero_total_num, const char * fname)
- int mul_qdrsmatrix_qdvec(QDVector ret, QDRSMatrix mat, QDVector vec)
- int mul_qdrsmatrixt_qdvec(QDVector ret, QDRSMatrix mat, QDVector vec)
- int mul_drsmatrix_qdvec(QDVector ret, DRSMatrix mat, QDVector vec)
- int mul_drsmatrixt_qdvec(QDVector ret, DRSMatrix mat, QDVector vec)
- void qdpower_sp(double max_eig[QDSIZE], QDVector evec, QDRSMatrix mat, double reps[QDSIZE], double aeps[QDSIZE], long int maxtimes)
- long int absmax_index_qdvector(double ret[QDSIZE], QDVector vec)
- void subst_qdrsmatrix_drsmat(QDRSMatrix c, DRSMatrix a)
- void subst_drsmatrix_qdrsmat(DRSMatrix c, QDRSMatrix a)

- void absmax_row_qdrsmatrix(double mu[QDSIZE], long int * max_j, long int row, QDRSMatrix mat)

- void absmax_col_qdrsmatrix(double mu[QDSIZE], long int * max_i, long int col, QDRSMatrix mat)

- void add_qdrsmatrix_drsmat(QDRSMatrix c, QDRSMatrix a, DRSMatrix b)
- void sub_qdrsmatrix_drsmat(QDRSMatrix c, QDRSMatrix a, DRSMatrix b)
- int split_qdrsmatrix_drsmat(DRSMatrix[] ret, int num_div, QDRSMatrix org)
- int split_qdrsmatrix_t_drsmat(DRSMatrix[] ret, int num_div, QDRSMatrix org)
- void mul_qdrsmatrix_qdvec_oz(QDVector ret, QDRSMatrix a, int max_num_div_a, QDVector vb, int max_num_div_vb)
- void mul_qdrsmatrixt_qdvec_oz(QDVector ret, QDRSMatrix a, int max_num_div_a, QDVector vb, int max_num_div_vb)
- void iLU0_qdrsmatrix(QDRSMatrix mat)
- void solve_iLU0_qdrsmatrix(QDVector ret, QDRSMatrix ilu, QDVector b)
- void solve_iLU0t_qdrsmatrix(QDVector ret, QDRSMatrix ilu, QDVector b)
- void solve_iLU0_drsmatrix_qdvec(QDVector ret, DRSMatrix ilu, QDVector b)
- void solve_iLU0t_drsmatrix_qdvec(QDVector ret, DRSMatrix ilu, QDVector b)

Complex QD sparse matrix (CQDRSMatrix):

- CQDRSMatrix init_cqdrsmatrix(long int row_dim, long int * nzero_col_dim, long int nzero_total_num)
- void free_cqdrsmatrix(CQDRSMatrix mat)
- void print_cqdrsmatrix(CQDRSMatrix mat)
- CQDRSMatrix init_set_cqdrsmatrix(CQDRSMatrix org)
- int mul_cqdrsmatrix_cqdvec(CQDVector ret, CQDRSMatrix mat, CQDVector vec)
- int mul_cqdrsmatrixt_cqdvec(CQDVector ret, CQDRSMatrix mat, CQDVector vec)
- int mul_cqdrsmatrixs_cqdvec(CQDVector ret, CQDRSMatrix mat, CQDVector vec)
- int mul_cdrsmatrix_cqdvec(CQDVector ret, CDRSMatrix mat, CQDVector vec)
- int mul_cdrsmatrixt_cqdvec(CQDVector ret, CDRSMatrix mat, CQDVector vec)
- int mul_cdrsmatrixs_cqdvec(CQDVector ret, CDRSMatrix mat, CQDVector vec)
- void iLU0_cqdrsmatrix(CQDRSMatrix mat)
- void solve_iLU0_cqdrsmatrix(CQDVector ret, CQDRSMatrix ilu, CQDVector b)
- void solve_iLU0t_cqdrsmatrix(CQDVector ret, CQDRSMatrix ilu, CQDVector b)
- void solve_iLU0s_cqdrsmatrix(CQDVector ret, CQDRSMatrix ilu, CQDVector b)
- void solve_iLU0_cdrsmatrix_cqdvec(CQDVector ret, CDRSMatrix ilu, CQDVector b)
- void solve_iLU0t_cdrsmatrix_cqdvec(CQDVector ret, CDRSMatrix ilu, CQDVector b)
- void solve_iLU0s_cdrsmatrix_cqdvec(CQDVector ret, CDRSMatrix ilu, CQDVector b)

6.1.6 MPFRSMatrix and CMPFRSMatrix — multiple-precision sparse matrices (requires USE_GMP)

The mpfrsmatrix structure stores a GMP multiple-precision sparse matrix; pointer type MPFRSMatrix. Its elements are stored as mpf_t *element. The complex variant CMPFRSMatrix uses mpc_t *element.

- MPFRSMatrix init_mpfrsmatrix(long int row_dim, long int * nzero_col_dim, long int nzero_total_num)
- MPFRSMatrix init2_mpfrsmatrix(long int row_dim, long int * nzero_col_dim, long int nzero_total_num, unsigned long prec)
- void free_mpfrsmatrix(MPFRSMatrix mat)
- void set_nzero_row_dim_mpf(MPFRSMatrix mat)
- void print_mpfrsmatrix(MPFRSMatrix mat)
- void get_mpfrsmatrix_ij(mpf_t ret, MPFRSMatrix mat, long int i, long int j)

- void set_mpfrsmatrix_ij(MPFRSMatrix mat, long int i, long int j, mpf_t val)
- MPFRSMatrix init_set_mpfrsmatrix(MPFRSMatrix org)
- void set0_mpfrsmatrix(MPFRSMatrix spmat)
- void set_mpfmatrix_mpfrsmat(MPFMatrix ret, MPFRSMatrix spmat)
- MPFRSMatrix init_set_mpfrsmatrix_mpfmatrix(MPFMatrix org)
- int get_vars_mpfrsmatrix_fname(long int * row_dim, long int ** nzero_col_dim, long int * nzero_total_num, const char * fname)
- int fread_urilinkdat_fname_mpf(MPFRSMatrix mat, const char * fname)
- long int absmax_index_mpfvector(mpf_t absmax, MPFVector vec)
- int mul_mpfrsmatrix_mpfvec(MPFVector ret, MPFRSMatrix mat, MPFVector vec)
- int mul_mpfrsmatrixt_mpfvec(MPFVector ret, MPFRSMatrix mat, MPFVector vec)
- int mul_drsmatrix_mpfvec(MPFVector ret, DRSMatrix mat, MPFVector vec)
- int mul_drsmatrixt_mpfvec(MPFVector ret, DRSMatrix mat, MPFVector vec)
- void mpfpower_rsmatrix(mpf_t max_eig, MPFVector evec, MPFRSMatrix mat, mpf_t reps, mpf_t aeps, long int maxtimes)
- int smul_mpfvector(MPFVector ret, mpf_t c, MPFVector vec)
- void iLU0_mpfrsmatrix(MPFRSMatrix mat)
- void solve_iLU0_mpfrsmatrix(MPFVector ret, MPFRSMatrix ilu, MPFVector b)
- void solve_iLU0t_mpfrsmatrix(MPFVector ret, MPFRSMatrix ilu, MPFVector b)
- void solve_iLU0_drsmatrix_mpfvec(MPFVector ret, DRSMatrix ilu, MPFVector b)
- void solve_iLU0t_drsmatrix_mpfvec(MPFVector ret, DRSMatrix ilu, MPFVector b)

Complex MPF sparse matrix (CMPFRSMatrix):

- CMPFRSMatrix init_cmpfrsmatrix(long int row_dim, long int * nzero_col_dim, long int nzero_total_num)
- CMPFRSMatrix init2_cmpfrsmatrix(long int row_dim, long int * nzero_col_dim, long int nzero_total_num, unsigned long prec)
- void free_cmpfrsmatrix(CMPFRSMatrix mat)
- void set_nzero_row_dim_cmpf(CMPFRSMatrix mat)
- void print_cmpfrsmatrix(CMPFRSMatrix mat)
- void normf_cmpfrsmatrix(mpf_t ret, CMPFRSMatrix mat)
- mpc_ptr get_cmpfrsmatrix_ij(CMPFRSMatrix mat, long int i, long int j)
- void set_cmpfrsmatrix_ij(CMPFRSMatrix mat, long int i, long int j, mpc_t val)
- CMPFRSMatrix init_set_cmpfrsmatrix(CMPFRSMatrix org)
- void set_cmpfmatrix_cmpfrsmat(CMPFMatrix ret, CMPFRSMatrix spmat)
- CMPFRSMatrix init_set_cmpfrsmatrix_cmpfmatrix(CMPFMatrix org)
- int get_vars_cmpfrsmatrix_fname(long int * row_dim, long int ** nzero_col_dim, long int * nzero_total_num, const char * fname)
- int mul_cmpfrsmatrix_cmpfvec(CMPFVector ret, CMPFRSMatrix mat, CMPFVector vec)
- int mul_cmpfrsmatrixt_cmpfvec(CMPFVector ret, CMPFRSMatrix mat, CMPFVector vec)
- int mul_cmpfrsmatrixs_cmpfvec(CMPFVector ret, CMPFRSMatrix mat, CMPFVector vec)
- int mul_cdrsmatrix_cmpfvec(CMPFVector ret, CDRSMatrix mat, CMPFVector vec)
- int mul_cdrsmatrixt_cmpfvec(CMPFVector ret, CDRSMatrix mat, CMPFVector vec)
- int mul_cdrsmatrixs_cmpfvec(CMPFVector ret, CDRSMatrix mat, CMPFVector vec)
- void iLU0_cmpfrsmatrix(CMPFRSMatrix mat)
- void solve_iLU0_cmpfrsmatrix(CMPFVector ret, CMPFRSMatrix ilu, CMPFVector b)
- void solve_iLU0t_cmpfrsmatrix(CMPFVector ret, CMPFRSMatrix ilu, CMPFVector b)
- void solve_iLU0s_cmpfrsmatrix(CMPFVector ret, CMPFRSMatrix ilu, CMPFVector b)
- void solve_iLU0_cdrsmatrix_cmpfvec(CMPFVector ret, CDRSMatrix ilu, CMPFVector b)
- void solve_iLU0t_cdrsmatrix_cmpfvec(CMPFVector ret, CDRSMatrix ilu, CMPFVector b)
- void solve_iLU0s_cdrsmatrix_cmpfvec(CMPFVector ret, CDRSMatrix ilu, CMPFVector b)

Conversion functions from MPFRSMatrix to other types (requires USE_GMP):

- DRSMatrix `init_set_drsmatrix_mpfrsmatrix`(MPFRSMatrix `org`)
- CDRSMatrix `init_set_cdrsmatrix_cmpfrsmatrix`(CMPFRSMatrix `org`)
- DDRSMatrix `init_set_ddrsmatrix_mpfrsmatrix`(MPFRSMatrix `org`) (requires `USE_DDLINEAR`)
- CDDRSMatrix `init_set_cddrsmatrix_cmpfrsmatrix`(CMPFRSMatrix `org`) (requires `USE_DDLINEAR`)
- TDRSMatrix `init_set_tdrsmatrix_mpfrsmatrix`(MPFRSMatrix `org`) (requires `USE_TDLINEAR`)
- CTDRSMatrix `init_set_ctdrsmatrix_cmpfrsmatrix`(CMPFRSMatrix `org`) (requires `USE_TDLINEAR`)
- QDRSMatrix `init_set_qdrsmatrix_mpfrsmatrix`(MPFRSMatrix `org`) (requires `USE_QDLINEAR`)
- CQDRSMatrix `init_set_cqdrsmatrix_cmpfrsmatrix`(CMPFRSMatrix `org`) (requires `USE_QDLINEAR`)

6.2 Krylov subspace iterative solvers

6.2.1 Conjugate Gradient method (`FCG`, `DCG`, `DDCG`, `TDCG`, `QDCG`, `MPFCG`)

- long int `FCG`(FVector `ans`, FMatrix `a`, FVector `b`, float `reps`, float `aeps`, long int `maxtimes`)
- long int `DCG_sp`(DVector `ans`, DRSMatrix `a`, DVector `b`, double `reps`, double `aeps`, long int `maxtimes`)

- long int `bnc_DCG`(DVector `ans`, DMatrix `a`, DVector `b`, double `reps`, double `aeps`, long int `maxtimes`)

- long int `CDCOCG_sp`(CDVector `ans`, CDRSMatrix `a`, CDVector `b`, double `reps`, double `aeps`, long int `maxtimes`)
- long int `CDCOCG`(CDVector `ans`, CDMatrix `a`, CDVector `b`, double `reps`, double `aeps`, long int `maxtimes`)

Double-double CG (requires `USE_DDLINEAR`):

- long int `DDCG_sp`(DDVector `ans`, DDRSMatrix `a`, DDVector `b`, double `reps[DDSIZE]`, double `aeps[DDSIZE]`, long int `maxtimes`)
- long int `DDCG_sp_d`(DDVector `ans`, DRSMatrix `a`, DDVector `b`, double `reps[DDSIZE]`, double `aeps[DDSIZE]`, long int `maxtimes`)
- long int `DDCG`(DDVector `ans`, DDMatrix `a`, DDVector `b`, double `reps[DDSIZE]`, double `aeps[DDSIZE]`, long int `maxtimes`)
- long int `CDDCOCG_sp`(CDDVector `ans`, CDDRSMatrix `a`, CDDVector `b`, double `reps[DDSIZE]`, double `aeps[DDSIZE]`, long int `maxtimes`)
- long int `CDDCOCG_sp_d`(CDDVector `ans`, CDRSMatrix `a`, CDDVector `b`, double `reps[DDSIZE]`, double `aeps[DDSIZE]`, long int `maxtimes`)
- long int `CDDCOCG`(CDDVector `ans`, CDDMatrix `a`, CDDVector `b`, double `reps[DDSIZE]`, double `aeps[DDSIZE]`, long int `maxtimes`)

Triple-double CG (requires `USE_TDLINEAR`):

- long int `TDCG_sp`(TDVector `ans`, TDRSMatrix `a`, TDVector `b`, double `reps[TDSIZE]`, double `aeps[TDSIZE]`, long int `maxtimes`)
- long int `TDCG_sp_d`(TDVector `ans`, DRSMatrix `a`, TDVector `b`, double `reps[TDSIZE]`, double `aeps[TDSIZE]`, long int `maxtimes`)
- long int `TDCG`(TDVector `ans`, TDMatrix `a`, TDVector `b`, double `reps[TDSIZE]`, double `aeps[TDSIZE]`, long int `maxtimes`)
- long int `CTDCOCG_sp`(CTDVector `ans`, CTDRSMatrix `a`, CTDVector `b`, double `reps[TDSIZE]`, double `aeps[TDSIZE]`, long int `maxtimes`)
- long int `CTDCOCG_sp_d`(CTDVector `ans`, CDRSMatrix `a`, CTDVector `b`, double `reps[TDSIZE]`, double `aeps[TDSIZE]`, long int `maxtimes`)
- long int `CTDCOCG`(CTDVector `ans`, CTDMatrix `a`, CTDVector `b`, double `reps[TDSIZE]`, double `aeps[TDSIZE]`, long int `maxtimes`)

Quadruple-double CG (requires `USE_QDLINEAR`):

- long int QDCG_sp(QDVector ans, QDRSMatrix a, QDVector b, double reps[QDSIZE], double aeps[QDSIZE], long int maxtimes)
- long int QDCG_sp_d(QDVector ans, DRSMatrix a, QDVector b, double reps[QDSIZE], double aeps[QDSIZE], long int maxtimes)
- long int QDCG(QDVector ans, QDMatrix a, QDVector b, double reps[QDSIZE], double aeps[QDSIZE], long int maxtimes)
- long int CQDCOCG_sp(CQDVector ans, CQDRSMatrix a, CQDVector b, double reps[QDSIZE], double aeps[QDSIZE], long int maxtimes)
- long int CQDCOCG_sp_d(CQDVector ans, CDRSMatrix a, CQDVector b, double reps[QDSIZE], double aeps[QDSIZE], long int maxtimes)
- long int CQDCOCG(CQDVector ans, CQDMatrix a, CQDVector b, double reps[QDSIZE], double aeps[QDSIZE], long int maxtimes)

Multiple-precision CG (requires USE_GMP):

- long int MPFCG_sp(MPFVector ans, MPFRSMatrix a, MPFVector b, mpf_t reps, mpf_t aeps, long int maxtimes)
- long int MPFCG_sp_d(MPFVector ans, DRSMatrix a, MPFVector b, mpf_t reps, mpf_t aeps, long int maxtimes)
- long int MPFCG(MPFVector ans, MPFMatrix a, MPFVector b, mpf_t reps, mpf_t aeps, long int maxtimes)
- long int CMPFCOCG_sp(CMPFVector ans, CMPFRSMatrix a, CMPFVector b, mpf_t reps, mpf_t aeps, long int maxtimes)
- long int CMPFCOCG_sp_d(CMPFVector ans, CDRSMatrix a, CMPFVector b, mpf_t reps, mpf_t aeps, long int maxtimes)
- long int CMPFCOCG(CMPFVector ans, CMPFMatrix a, CMPFVector b, mpf_t reps, mpf_t aeps, long int maxtimes)

6.2.2 BiCG, CGS, BiCGSTAB, GPBiCG methods (double)

Each solver is provided in sparse (_sp), sparse with preconditioning (_sp_iLU0), and dense variants.

- long int DBiCG_sp(DVector ans, DRSMatrix a, DVector b, double reps, double aeps, long int maxtimes)
- long int DCGS_sp(DVector ans, DRSMatrix a, DVector b, double reps, double aeps, long int maxtimes)
- long int DBiCGSTAB_sp(DVector ans, DRSMatrix a, DVector b, double reps, double aeps, long int maxtimes)
- long int DGPBiCG_sp(DVector ans, DRSMatrix a, DVector b, double reps, double aeps, long int maxtimes)
- long int DBiCG_sp_iLU0(DVector ans, DRSMatrix a, DVector b, double reps, double aeps, long int maxtimes, DRSMatrix ilu, double * norm2_res_history)
- long int DCGS_sp_iLU0(DVector ans, DRSMatrix a, DVector b, double reps, double aeps, long int maxtimes, DRSMatrix ilu, double * norm2_res_history)
- long int DBiCGSTAB_sp_iLU0(DVector ans, DRSMatrix a, DVector b, double reps, double aeps, long int maxtimes, DRSMatrix ilu, double * norm2_res_history)
- long int DGPBiCG_sp_iLU0(DVector ans, DRSMatrix a, DVector b, double reps, double aeps, long int maxtimes, DRSMatrix ilu, double * norm2_res_history)
- long int DBiCG(DVector ans, DMatrix a, DVector b, double reps, double aeps, long int maxtimes)
- long int DCGS(DVector ans, DMatrix a, DVector b, double reps, double aeps, long int maxtimes)
- long int DBiCGSTAB(DVector ans, DMatrix a, DVector b, double reps, double aeps, long int maxtimes)
- long int DGPBiCG(DVector ans, DMatrix a, DVector b, double reps, double aeps, long int maxtimes)

- long int CDBiCG_sp(CDVector ans, CDRSMatrix a, CDVector b, double reps, double aeps, long int maxtimes)
- long int CDCGS_sp(CDVector ans, CDRSMatrix a, CDVector b, double reps, double aeps, long int maxtimes)
- long int CDBiCGSTAB_sp(CDVector ans, CDRSMatrix a, CDVector b, double reps, double aeps, long int maxtimes)
- long int CDGPBiCG_sp(CDVector ans, CDRSMatrix a, CDVector b, double reps, double aeps, long int maxtimes)
- long int CDBiCG_sp_iLU0(CDVector ans, CDRSMatrix a, CDVector b, double reps, double aeps, long int maxtimes, CDRSMatrix ilu, double * norm2_res_history)
- long int CDCGS_sp_iLU0(CDVector ans, CDRSMatrix a, CDVector b, double reps, double aeps, long int maxtimes, CDRSMatrix ilu, double * norm2_res_history)
- long int CDBiCGSTAB_sp_iLU0(CDVector ans, CDRSMatrix a, CDVector b, double reps, double aeps, long int maxtimes, CDRSMatrix ilu, double * norm2_res_history)
- long int CDGPBiCG_sp_iLU0(CDVector ans, CDRSMatrix a, CDVector b, double reps, double aeps, long int maxtimes, CDRSMatrix ilu, double * norm2_res_history)
- long int CDBiCG(CDVector ans, CDMatrix a, CDVector b, double reps, double aeps, long int maxtimes)
- long int CDCGS(CDVector ans, CDMatrix a, CDVector b, double reps, double aeps, long int maxtimes)

- long int CDBiCGSTAB(CDVector ans, CDMatrix a, CDVector b, double reps, double aeps, long int maxtimes)
- long int CDGPBiCG(CDVector ans, CDMatrix a, CDVector b, double reps, double aeps, long int maxtimes)

6.2.3 BiCG, CGS, BiCGSTAB, GPBiCG methods (double-double; requires USE_DDLINEAR)

- long int DDBiCG_sp(DDVector ans, DDRSMatrix a, DDVector b, double reps[DDSIZE], double aeps[DDSIZE], long int maxtimes)
- long int DDCGS_sp(DDVector ans, DDRSMatrix a, DDVector b, double reps[DDSIZE], double aeps[DDSIZE], long int maxtimes)
- long int DDBiCGSTAB_sp(DDVector ans, DDRSMatrix a, DDVector b, double reps[DDSIZE], double aeps[DDSIZE], long int maxtimes)
- long int DDGPBiCG_sp(DDVector ans, DDRSMatrix a, DDVector b, double reps[DDSIZE], double aeps[DDSIZE], long int maxtimes)
- long int DDBiCG_sp_iLU0(DDVector ans, DDRSMatrix a, DDVector b, double reps[DDSIZE], double aeps[DDSIZE], long int maxtimes, DDRSMatrix ilu, double * norm2_res_history)
- long int DDCGS_sp_iLU0(DDVector ans, DDRSMatrix a, DDVector b, double reps[DDSIZE], double aeps[DDSIZE], long int maxtimes, DDRSMatrix ilu, double * norm2_res_history)
- long int DDBiCGSTAB_sp_iLU0(DDVector ans, DDRSMatrix a, DDVector b, double reps[DDSIZE], double aeps[DDSIZE], long int maxtimes, DDRSMatrix ilu, double * norm2_res_history)
- long int DDGPBiCG_sp_iLU0(DDVector ans, DDRSMatrix a, DDVector b, double reps[DDSIZE], double aeps[DDSIZE], long int maxtimes, DDRSMatrix ilu, double * norm2_res_history)
- long int DDBiCG_sp_d(DDVector ans, DRSMatrix a, DDVector b, double reps[DDSIZE], double aeps[DDSIZE], long int maxtimes)
- long int DDCGS_sp_d(DDVector ans, DRSMatrix a, DDVector b, double reps[DDSIZE], double aeps[DDSIZE], long int maxtimes)
- long int DDBiCGSTAB_sp_d(DDVector ans, DRSMatrix a, DDVector b, double reps[DDSIZE], double aeps[DDSIZE], long int maxtimes)
- long int DDGPBiCG_sp_d(DDVector ans, DRSMatrix a, DDVector b, double reps[DDSIZE], double aeps[DDSIZE], long int maxtimes)
- long int DDBiCG_sp_d_iLU0(DDVector ans, DRSMatrix a, DDVector b, double reps[DDSIZE], double

aeps[DDSIZE], long int maxtimes, DRSMatrix ilu, double * norm2_res_history)

- long int DDCGS_sp_d_iLU0(DDVector ans, DRSMatrix a, DDVector b, double reps[DDSIZE], double aeps[DDSIZE], long int maxtimes, DRSMatrix ilu, double * norm2_res_history)
- long int DDBiCGSTAB_sp_d_iLU0(DDVector ans, DRSMatrix a, DDVector b, double reps[DDSIZE], double aeps[DDSIZE], long int maxtimes, DRSMatrix ilu, double * norm2_res_history)
- long int DDGPBiCG_sp_d_iLU0(DDVector ans, DRSMatrix a, DDVector b, double reps[DDSIZE], double aeps[DDSIZE], long int maxtimes, DRSMatrix ilu, double * norm2_res_history)
- long int DDBiCG(DDVector ans, DDMatrix a, DDVector b, double reps[DDSIZE], double aeps[DDSIZE], long int maxtimes)
- long int DDCGS(DDVector ans, DDMatrix a, DDVector b, double reps[DDSIZE], double aeps[DDSIZE], long int maxtimes)
- long int DDBiCGSTAB(DDVector ans, DDMatrix a, DDVector b, double reps[DDSIZE], double aeps[DDSIZE], long int maxtimes)
- long int DDGPBiCG(DDVector ans, DDMatrix a, DDVector b, double reps[DDSIZE], double aeps[DDSIZE], long int maxtimes)
- long int CDDBiCG_sp(CDDVector ans, CDDRSMatrix a, CDDVector b, double reps[DDSIZE], double aeps[DDSIZE], long int maxtimes)
- long int CDDCGS_sp(CDDVector ans, CDDRSMatrix a, CDDVector b, double reps[DDSIZE], double aeps[DDSIZE], long int maxtimes)
- long int CDDBiCGSTAB_sp(CDDVector ans, CDDRSMatrix a, CDDVector b, double reps[DDSIZE], double aeps[DDSIZE], long int maxtimes)
- long int CDDGPBiCG_sp(CDDVector ans, CDDRSMatrix a, CDDVector b, double reps[DDSIZE], double aeps[DDSIZE], long int maxtimes)
- long int CDDBiCG_sp_iLU0(CDDVector ans, CDDRSMatrix a, CDDVector b, double reps[DDSIZE], double aeps[DDSIZE], long int maxtimes, CDDRSMatrix ilu, double * norm2_res_history)
- long int CDDCGS_sp_iLU0(CDDVector ans, CDDRSMatrix a, CDDVector b, double reps[DDSIZE], double aeps[DDSIZE], long int maxtimes, CDDRSMatrix ilu, double * norm2_res_history)
- long int CDDBiCGSTAB_sp_iLU0(CDDVector ans, CDDRSMatrix a, CDDVector b, double reps[DDSIZE], double aeps[DDSIZE], long int maxtimes, CDDRSMatrix ilu, double * norm2_res_history)
- long int CDDGPBiCG_sp_iLU0(CDDVector ans, CDDRSMatrix a, CDDVector b, double reps[DDSIZE], double aeps[DDSIZE], long int maxtimes, CDDRSMatrix ilu, double * norm2_res_history)
- long int CDDBiCG_sp_d(CDDVector ans, CDRSMatrix a, CDDVector b, double reps[DDSIZE], double aeps[DDSIZE], long int maxtimes)
- long int CDDCGS_sp_d(CDDVector ans, CDRSMatrix a, CDDVector b, double reps[DDSIZE], double aeps[DDSIZE], long int maxtimes)
- long int CDDBiCGSTAB_sp_d(CDDVector ans, CDRSMatrix a, CDDVector b, double reps[DDSIZE], double aeps[DDSIZE], long int maxtimes)
- long int CDDGPBiCG_sp_d(CDDVector ans, CDRSMatrix a, CDDVector b, double reps[DDSIZE], double aeps[DDSIZE], long int maxtimes)
- long int CDDBiCG_sp_d_iLU0(CDDVector ans, CDRSMatrix a, CDDVector b, double reps[DDSIZE], double aeps[DDSIZE], long int maxtimes, CDRSMatrix ilu, double * norm2_res_history)
- long int CDDCGS_sp_d_iLU0(CDDVector ans, CDRSMatrix a, CDDVector b, double reps[DDSIZE], double aeps[DDSIZE], long int maxtimes, CDRSMatrix ilu, double * norm2_res_history)
- long int CDDBiCGSTAB_sp_d_iLU0(CDDVector ans, CDRSMatrix a, CDDVector b, double reps[DDSIZE], double aeps[DDSIZE], long int maxtimes, CDRSMatrix ilu, double * norm2_res_history)
- long int CDDGPBiCG_sp_d_iLU0(CDDVector ans, CDRSMatrix a, CDDVector b, double reps[DDSIZE], double aeps[DDSIZE], long int maxtimes, CDRSMatrix ilu, double * norm2_res_history)
- long int CDDBiCG(CDDVector ans, CDDMatrix a, CDDVector b, double reps[DDSIZE], double aeps[DDSIZE], long int maxtimes)
- long int CDDCGS(CDDVector ans, CDDMatrix a, CDDVector b, double reps[DDSIZE], double aeps[DDSIZE], long int maxtimes)

- long int CDDBiCGSTAB(CDDVector ans, CDDMatrix a, CDDVector b, double reps[DDSIZE], double aeps[DDSIZE], long int maxtimes)
- long int CDDGPBiCG(CDDVector ans, CDDMatrix a, CDDVector b, double reps[DDSIZE], double aeps[DDSIZE], long int maxtimes)

6.2.4 BiCG, CGS, BiCGSTAB, GPBiCG methods (triple-double; requires USE_TDLINEAR)

- long int TDBiCG_sp(TDVector ans, TDRSMatrix a, TDVector b, double reps[TDSIZE], double aeps[TDSIZE], long int maxtimes)
- long int TDCGS_sp(TDVector ans, TDRSMatrix a, TDVector b, double reps[TDSIZE], double aeps[TDSIZE], long int maxtimes)
- long int TDBiCGSTAB_sp(TDVector ans, TDRSMatrix a, TDVector b, double reps[TDSIZE], double aeps[TDSIZE], long int maxtimes)
- long int TDGPBiCG_sp(TDVector ans, TDRSMatrix a, TDVector b, double reps[TDSIZE], double aeps[TDSIZE], long int maxtimes)
- long int TDBiCG_sp_iLU0(TDVector ans, TDRSMatrix a, TDVector b, double reps[TDSIZE], double aeps[TDSIZE], long int maxtimes, TDRSMatrix ilu, double * norm2_res_history)
- long int TDCGS_sp_iLU0(TDVector ans, TDRSMatrix a, TDVector b, double reps[TDSIZE], double aeps[TDSIZE], long int maxtimes, TDRSMatrix ilu, double * norm2_res_history)
- long int TDBiCGSTAB_sp_iLU0(TDVector ans, TDRSMatrix a, TDVector b, double reps[TDSIZE], double aeps[TDSIZE], long int maxtimes, TDRSMatrix ilu, double * norm2_res_history)
- long int TDGPBiCG_sp_iLU0(TDVector ans, TDRSMatrix a, TDVector b, double reps[TDSIZE], double aeps[TDSIZE], long int maxtimes, TDRSMatrix ilu, double * norm2_res_history)
- long int TDBiCG_sp_d(TDVector ans, DRSMatrix a, TDVector b, double reps[TDSIZE], double aeps[TDSIZE], long int maxtimes)
- long int TDCGS_sp_d(TDVector ans, DRSMatrix a, TDVector b, double reps[TDSIZE], double aeps[TDSIZE], long int maxtimes)
- long int TDBiCGSTAB_sp_d(TDVector ans, DRSMatrix a, TDVector b, double reps[TDSIZE], double aeps[TDSIZE], long int maxtimes)
- long int TDGPBiCG_sp_d(TDVector ans, DRSMatrix a, TDVector b, double reps[TDSIZE], double aeps[TDSIZE], long int maxtimes)
- long int TDBiCG_sp_d_iLU0(TDVector ans, DRSMatrix a, TDVector b, double reps[TDSIZE], double aeps[TDSIZE], long int maxtimes, DRSMatrix ilu, double * norm2_res_history)
- long int TDCGS_sp_d_iLU0(TDVector ans, DRSMatrix a, TDVector b, double reps[TDSIZE], double aeps[TDSIZE], long int maxtimes, DRSMatrix ilu, double * norm2_res_history)
- long int TDBiCGSTAB_sp_d_iLU0(TDVector ans, DRSMatrix a, TDVector b, double reps[TDSIZE], double aeps[TDSIZE], long int maxtimes, DRSMatrix ilu, double * norm2_res_history)
- long int TDGPBiCG_sp_d_iLU0(TDVector ans, DRSMatrix a, TDVector b, double reps[TDSIZE], double aeps[TDSIZE], long int maxtimes, DRSMatrix ilu, double * norm2_res_history)
- long int TDBiCG(TDVector ans, TDMatrix a, TDVector b, double reps[TDSIZE], double aeps[TDSIZE], long int maxtimes)
- long int TDCGS(TDVector ans, TDMatrix a, TDVector b, double reps[TDSIZE], double aeps[TDSIZE], long int maxtimes)
- long int TDBiCGSTAB(TDVector ans, TDMatrix a, TDVector b, double reps[TDSIZE], double aeps[TDSIZE], long int maxtimes)
- long int TDGPBiCG(TDVector ans, TDMatrix a, TDVector b, double reps[TDSIZE], double aeps[TDSIZE], long int maxtimes)
- long int CTDBiCG_sp(CTDVector ans, CTDRSMatrix a, CTDVector b, double reps[TDSIZE], double aeps[TDSIZE], long int maxtimes)
- long int CTDCGS_sp(CTDVector ans, CTDRSMatrix a, CTDVector b, double reps[TDSIZE], double aeps[TDSIZE], long int maxtimes)
- long int CTDBiCGSTAB_sp(CTDVector ans, CTDRSMatrix a, CTDVector b, double reps[TDSIZE],

double aeps[TDSIZE], long int maxtimes)
- long int CTDGPBiCG_sp(CTDVector ans, CTDRSMatrix a, CTDVector b, double reps[TDSIZE], double aeps[TDSIZE], long int maxtimes)
- long int CTDBiCG_sp_iLU0(CTDVector ans, CTDRSMatrix a, CTDVector b, double reps[TDSIZE], double aeps[TDSIZE], long int maxtimes, CTDRSMatrix ilu, double * norm2_res_history)
- long int CTDCGS_sp_iLU0(CTDVector ans, CTDRSMatrix a, CTDVector b, double reps[TDSIZE], double aeps[TDSIZE], long int maxtimes, CTDRSMatrix ilu, double * norm2_res_history)
- long int CTDBiCGSTAB_sp_iLU0(CTDVector ans, CTDRSMatrix a, CTDVector b, double reps[TDSIZE], double aeps[TDSIZE], long int maxtimes, CTDRSMatrix ilu, double * norm2_res_history)
- long int CTDGPBiCG_sp_iLU0(CTDVector ans, CTDRSMatrix a, CTDVector b, double reps[TDSIZE], double aeps[TDSIZE], long int maxtimes, CTDRSMatrix ilu, double * norm2_res_history)
- long int CTDBiCG_sp_d(CTDVector ans, CDRSMatrix a, CTDVector b, double reps[TDSIZE], double aeps[TDSIZE], long int maxtimes)
- long int CTDCGS_sp_d(CTDVector ans, CDRSMatrix a, CTDVector b, double reps[TDSIZE], double aeps[TDSIZE], long int maxtimes)
- long int CTDBiCGSTAB_sp_d(CTDVector ans, CDRSMatrix a, CTDVector b, double reps[TDSIZE], double aeps[TDSIZE], long int maxtimes)
- long int CTDGPBiCG_sp_d(CTDVector ans, CDRSMatrix a, CTDVector b, double reps[TDSIZE], double aeps[TDSIZE], long int maxtimes)
- long int CTDBiCG_sp_d_iLU0(CTDVector ans, CDRSMatrix a, CTDVector b, double reps[TDSIZE], double aeps[TDSIZE], long int maxtimes, CDRSMatrix ilu, double * norm2_res_history)
- long int CTDCGS_sp_d_iLU0(CTDVector ans, CDRSMatrix a, CTDVector b, double reps[TDSIZE], double aeps[TDSIZE], long int maxtimes, CDRSMatrix ilu, double * norm2_res_history)
- long int CTDBiCGSTAB_sp_d_iLU0(CTDVector ans, CDRSMatrix a, CTDVector b, double reps[TDSIZE], double aeps[TDSIZE], long int maxtimes, CDRSMatrix ilu, double * norm2_res_history)
- long int CTDGPBiCG_sp_d_iLU0(CTDVector ans, CDRSMatrix a, CTDVector b, double reps[TDSIZE], double aeps[TDSIZE], long int maxtimes, CDRSMatrix ilu, double * norm2_res_history)
- long int CTDBiCG(CTDVector ans, CTDMatrix a, CTDVector b, double reps[TDSIZE], double aeps[TDSIZE], long int maxtimes)
- long int CTDCGS(CTDVector ans, CTDMatrix a, CTDVector b, double reps[TDSIZE], double aeps[TDSIZE], long int maxtimes)
- long int CTDBiCGSTAB(CTDVector ans, CTDMatrix a, CTDVector b, double reps[TDSIZE], double aeps[TDSIZE], long int maxtimes)
- long int CTDGPBiCG(CTDVector ans, CTDMatrix a, CTDVector b, double reps[TDSIZE], double aeps[TDSIZE], long int maxtimes)

6.2.5 BiCG, CGS, BiCGSTAB, GPBiCG methods (quadruple-double; requires USE_QDLINEAR)

- long int QDBiCG_sp(QDVector ans, QDRSMatrix a, QDVector b, double reps[QDSIZE], double aeps[QDSIZE], long int maxtimes)
- long int QDCGS_sp(QDVector ans, QDRSMatrix a, QDVector b, double reps[QDSIZE], double aeps[QDSIZE], long int maxtimes)
- long int QDBiCGSTAB_sp(QDVector ans, QDRSMatrix a, QDVector b, double reps[QDSIZE], double aeps[QDSIZE], long int maxtimes)
- long int QDGPBiCG_sp(QDVector ans, QDRSMatrix a, QDVector b, double reps[QDSIZE], double aeps[QDSIZE], long int maxtimes)
- long int QDBiCG_sp_iLU0(QDVector ans, QDRSMatrix a, QDVector b, double reps[QDSIZE], double aeps[QDSIZE], long int maxtimes, QDRSMatrix ilu, double * norm2_res_history)
- long int QDCGS_sp_iLU0(QDVector ans, QDRSMatrix a, QDVector b, double reps[QDSIZE], double aeps[QDSIZE], long int maxtimes, QDRSMatrix ilu, double * norm2_res_history)

- long int QDBiCGSTAB_sp_iLU0(QDVector ans, QDRSMatrix a, QDVector b, double reps[QDSIZE], double aeps[QDSIZE], long int maxtimes, QDRSMatrix ilu, double * norm2_res_history)
- long int QDGPBiCG_sp_iLU0(QDVector ans, QDRSMatrix a, QDVector b, double reps[QDSIZE], double aeps[QDSIZE], long int maxtimes, QDRSMatrix ilu, double * norm2_res_history)
- long int QDBiCG_sp_d(QDVector ans, DRSMatrix a, QDVector b, double reps[QDSIZE], double aeps[QDSIZE], long int maxtimes)
- long int QDCGS_sp_d(QDVector ans, DRSMatrix a, QDVector b, double reps[QDSIZE], double aeps[QDSIZE], long int maxtimes)
- long int QDBiCGSTAB_sp_d(QDVector ans, DRSMatrix a, QDVector b, double reps[QDSIZE], double aeps[QDSIZE], long int maxtimes)
- long int QDGPBiCG_sp_d(QDVector ans, DRSMatrix a, QDVector b, double reps[QDSIZE], double aeps[QDSIZE], long int maxtimes)
- long int QDBiCG_sp_d_iLU0(QDVector ans, DRSMatrix a, QDVector b, double reps[QDSIZE], double aeps[QDSIZE], long int maxtimes, DRSMatrix ilu, double * norm2_res_history)
- long int QDCGS_sp_d_iLU0(QDVector ans, DRSMatrix a, QDVector b, double reps[QDSIZE], double aeps[QDSIZE], long int maxtimes, DRSMatrix ilu, double * norm2_res_history)
- long int QDBiCGSTAB_sp_d_iLU0(QDVector ans, DRSMatrix a, QDVector b, double reps[QDSIZE], double aeps[QDSIZE], long int maxtimes, DRSMatrix ilu, double * norm2_res_history)
- long int QDGPBiCG_sp_d_iLU0(QDVector ans, DRSMatrix a, QDVector b, double reps[QDSIZE], double aeps[QDSIZE], long int maxtimes, DRSMatrix ilu, double * norm2_res_history)
- long int QDBiCG(QDVector ans, QDMatrix a, QDVector b, double reps[QDSIZE], double aeps[QDSIZE], long int maxtimes)
- long int QDCGS(QDVector ans, QDMatrix a, QDVector b, double reps[QDSIZE], double aeps[QDSIZE], long int maxtimes)
- long int QDBiCGSTAB(QDVector ans, QDMatrix a, QDVector b, double reps[QDSIZE], double aeps[QDSIZE], long int maxtimes)
- long int QDGPBiCG(QDVector ans, QDMatrix a, QDVector b, double reps[QDSIZE], double aeps[QDSIZE], long int maxtimes)
- long int CQDBiCG_sp(CQDVector ans, CQDRSMatrix a, CQDVector b, double reps[QDSIZE], double aeps[QDSIZE], long int maxtimes)
- long int CQDCGS_sp(CQDVector ans, CQDRSMatrix a, CQDVector b, double reps[QDSIZE], double aeps[QDSIZE], long int maxtimes)
- long int CQDBiCGSTAB_sp(CQDVector ans, CQDRSMatrix a, CQDVector b, double reps[QDSIZE], double aeps[QDSIZE], long int maxtimes)
- long int CQDGPBiCG_sp(CQDVector ans, CQDRSMatrix a, CQDVector b, double reps[QDSIZE], double aeps[QDSIZE], long int maxtimes)
- long int CQDBiCG_sp_iLU0(CQDVector ans, CQDRSMatrix a, CQDVector b, double reps[QDSIZE], double aeps[QDSIZE], long int maxtimes, CQDRSMatrix ilu, double * norm2_res_history)
- long int CQDCGS_sp_iLU0(CQDVector ans, CQDRSMatrix a, CQDVector b, double reps[QDSIZE], double aeps[QDSIZE], long int maxtimes, CQDRSMatrix ilu, double * norm2_res_history)
- long int CQDBiCGSTAB_sp_iLU0(CQDVector ans, CQDRSMatrix a, CQDVector b, double reps[QDSIZE], double aeps[QDSIZE], long int maxtimes, CQDRSMatrix ilu, double * norm2_res_history)
- long int CQDGPBiCG_sp_iLU0(CQDVector ans, CQDRSMatrix a, CQDVector b, double reps[QDSIZE], double aeps[QDSIZE], long int maxtimes, CQDRSMatrix ilu, double * norm2_res_history)
- long int CQDBiCG_sp_d(CQDVector ans, CDRSMatrix a, CQDVector b, double reps[QDSIZE], double aeps[QDSIZE], long int maxtimes)
- long int CQDCGS_sp_d(CQDVector ans, CDRSMatrix a, CQDVector b, double reps[QDSIZE], double aeps[QDSIZE], long int maxtimes)
- long int CQDBiCGSTAB_sp_d(CQDVector ans, CDRSMatrix a, CQDVector b, double reps[QDSIZE], double aeps[QDSIZE], long int maxtimes)
- long int CQDGPBiCG_sp_d(CQDVector ans, CDRSMatrix a, CQDVector b, double reps[QDSIZE], double aeps[QDSIZE], long int maxtimes)

- long int CQDBiCG_sp_d_iLU0(CQDVector ans, CDRSMatrix a, CQDVector b, double reps[QDSIZE], double aeps[QDSIZE], long int maxtimes, CDRSMatrix ilu, double * norm2_res_history)
- long int CQDCGS_sp_d_iLU0(CQDVector ans, CDRSMatrix a, CQDVector b, double reps[QDSIZE], double aeps[QDSIZE], long int maxtimes, CDRSMatrix ilu, double * norm2_res_history)
- long int CQDBiCGSTAB_sp_d_iLU0(CQDVector ans, CDRSMatrix a, CQDVector b, double reps[QDSIZE], double aeps[QDSIZE], long int maxtimes, CDRSMatrix ilu, double * norm2_res_history)
- long int CQDGPBiCG_sp_d_iLU0(CQDVector ans, CDRSMatrix a, CQDVector b, double reps[QDSIZE], double aeps[QDSIZE], long int maxtimes, CDRSMatrix ilu, double * norm2_res_history)
- long int CQDBiCG(CQDVector ans, CQDMatrix a, CQDVector b, double reps[QDSIZE], double aeps[QDSIZE], long int maxtimes)
- long int CQDCGS(CQDVector ans, CQDMatrix a, CQDVector b, double reps[QDSIZE], double aeps[QDSIZE], long int maxtimes)
- long int CQDBiCGSTAB(CQDVector ans, CQDMatrix a, CQDVector b, double reps[QDSIZE], double aeps[QDSIZE], long int maxtimes)
- long int CQDGPBiCG(CQDVector ans, CQDMatrix a, CQDVector b, double reps[QDSIZE], double aeps[QDSIZE], long int maxtimes)

6.2.6 BiCG, CGS, BiCGSTAB, GPBiCG methods (multiple-precision; requires USE_GMP)

- long int MPFBiCG_sp(MPFVector ans, MPFRSMatrix a, MPFVector b, mpf_t reps, mpf_t aeps, long int maxtimes)
- long int MPFCGS_sp(MPFVector ans, MPFRSMatrix a, MPFVector b, mpf_t reps, mpf_t aeps, long int maxtimes)
- long int MPFBiCGSTAB_sp(MPFVector ans, MPFRSMatrix a, MPFVector b, mpf_t reps, mpf_t aeps, long int maxtimes)
- long int MPFGPBiCG_sp(MPFVector ans, MPFRSMatrix a, MPFVector b, mpf_t reps, mpf_t aeps, long int maxtimes)
- long int MPFBiCG_sp_iLU0(MPFVector ans, MPFRSMatrix a, MPFVector b, mpf_t reps, mpf_t aeps, long int maxtimes, MPFRSMatrix ilu, MPFVector norm2_res_history)
- long int MPFCGS_sp_iLU0(MPFVector ans, MPFRSMatrix a, MPFVector b, mpf_t reps, mpf_t aeps, long int maxtimes, MPFRSMatrix ilu, MPFVector norm2_res_history)
- long int MPFBiCGSTAB_sp_iLU0(MPFVector ans, MPFRSMatrix a, MPFVector b, mpf_t reps, mpf_t aeps, long int maxtimes, MPFRSMatrix ilu, MPFVector norm2_res_history)
- long int MPFGPBiCG_sp_iLU0(MPFVector ans, MPFRSMatrix a, MPFVector b, mpf_t reps, mpf_t aeps, long int maxtimes, MPFRSMatrix ilu, MPFVector norm2_res_history)
- long int MPFBiCG_sp_d(MPFVector ans, DRSMatrix a, MPFVector b, mpf_t reps, mpf_t aeps, long int maxtimes)
- long int MPFCGS_sp_d(MPFVector ans, DRSMatrix a, MPFVector b, mpf_t reps, mpf_t aeps, long int maxtimes)
- long int MPFBiCGSTAB_sp_d(MPFVector ans, DRSMatrix a, MPFVector b, mpf_t reps, mpf_t aeps, long int maxtimes)
- long int MPFGPBiCG_sp_d(MPFVector ans, DRSMatrix a, MPFVector b, mpf_t reps, mpf_t aeps, long int maxtimes)
- long int MPFBiCG_sp_d_iLU0(MPFVector ans, DRSMatrix a, MPFVector b, mpf_t reps, mpf_t aeps, long int maxtimes, DRSMatrix ilu, MPFVector norm2_res_history)
- long int MPFCGS_sp_d_iLU0(MPFVector ans, DRSMatrix a, MPFVector b, mpf_t reps, mpf_t aeps, long int maxtimes, DRSMatrix ilu, MPFVector norm2_res_history)
- long int MPFBiCGSTAB_sp_d_iLU0(MPFVector ans, DRSMatrix a, MPFVector b, mpf_t reps, mpf_t aeps, long int maxtimes, DRSMatrix ilu, MPFVector norm2_res_history)
- long int MPFGPBiCG_sp_d_iLU0(MPFVector ans, DRSMatrix a, MPFVector b, mpf_t reps, mpf_t aeps, long int maxtimes, DRSMatrix ilu, MPFVector norm2_res_history)

- long int MPFBiCG(MPFVector ans, MPFMatrix a, MPFVector b, mpf_t reps, mpf_t aeps, long int maxtimes)
- long int MPFCGS(MPFVector ans, MPFMatrix a, MPFVector b, mpf_t reps, mpf_t aeps, long int maxtimes)
- long int MPFBiCGSTAB(MPFVector ans, MPFMatrix a, MPFVector b, mpf_t reps, mpf_t aeps, long int maxtimes)
- long int MPFGPBiCG(MPFVector ans, MPFMatrix a, MPFVector b, mpf_t reps, mpf_t aeps, long int maxtimes)
- long int CMPFBiCG_sp(CMPFVector ans, CMPFRSMatrix a, CMPFVector b, mpf_t reps, mpf_t aeps, long int maxtimes)
- long int CMPFCGS_sp(CMPFVector ans, CMPFRSMatrix a, CMPFVector b, mpf_t reps, mpf_t aeps, long int maxtimes)
- long int CMPFBiCGSTAB_sp(CMPFVector ans, CMPFRSMatrix a, CMPFVector b, mpf_t reps, mpf_t aeps, long int maxtimes)
- long int CMPFGPBiCG_sp(CMPFVector ans, CMPFRSMatrix a, CMPFVector b, mpf_t reps, mpf_t aeps, long int maxtimes)
- long int CMPFBiCG_sp_iLU0(CMPFVector ans, CMPFRSMatrix a, CMPFVector b, mpf_t reps, mpf_t aeps, long int maxtimes, CMPFRSMatrix ilu, MPFVector norm2_res_history)
- long int CMPFCGS_sp_iLU0(CMPFVector ans, CMPFRSMatrix a, CMPFVector b, mpf_t reps, mpf_t aeps, long int maxtimes, CMPFRSMatrix ilu, MPFVector norm2_res_history)
- long int CMPFBiCGSTAB_sp_iLU0(CMPFVector ans, CMPFRSMatrix a, CMPFVector b, mpf_t reps, mpf_t aeps, long int maxtimes, CMPFRSMatrix ilu, MPFVector norm2_res_history)
- long int CMPFGPBiCG_sp_iLU0(CMPFVector ans, CMPFRSMatrix a, CMPFVector b, mpf_t reps, mpf_t aeps, long int maxtimes, CMPFRSMatrix ilu, MPFVector norm2_res_history)
- long int CMPFBiCG_sp_d(CMPFVector ans, CDRSMatrix a, CMPFVector b, mpf_t reps, mpf_t aeps, long int maxtimes)
- long int CMPFCGS_sp_d(CMPFVector ans, CDRSMatrix a, CMPFVector b, mpf_t reps, mpf_t aeps, long int maxtimes)
- long int CMPFBiCGSTAB_sp_d(CMPFVector ans, CDRSMatrix a, CMPFVector b, mpf_t reps, mpf_t aeps, long int maxtimes)
- long int CMPFGPBiCG_sp_d(CMPFVector ans, CDRSMatrix a, CMPFVector b, mpf_t reps, mpf_t aeps, long int maxtimes)
- long int CMPFBiCG_sp_d_iLU0(CMPFVector ans, CDRSMatrix a, CMPFVector b, mpf_t reps, mpf_t aeps, long int maxtimes, CDRSMatrix ilu, MPFVector norm2_res_history)
- long int CMPFCGS_sp_d_iLU0(CMPFVector ans, CDRSMatrix a, CMPFVector b, mpf_t reps, mpf_t aeps, long int maxtimes, CDRSMatrix ilu, MPFVector norm2_res_history)
- long int CMPFBiCGSTAB_sp_d_iLU0(CMPFVector ans, CDRSMatrix a, CMPFVector b, mpf_t reps, mpf_t aeps, long int maxtimes, CDRSMatrix ilu, MPFVector norm2_res_history)
- long int CMPFGPBiCG_sp_d_iLU0(CMPFVector ans, CDRSMatrix a, CMPFVector b, mpf_t reps, mpf_t aeps, long int maxtimes, CDRSMatrix ilu, MPFVector norm2_res_history)
- long int CMPFBiCG(CMPFVector ans, CMPFMatrix a, CMPFVector b, mpf_t reps, mpf_t aeps, long int maxtimes)
- long int CMPFCGS(CMPFVector ans, CMPFMatrix a, CMPFVector b, mpf_t reps, mpf_t aeps, long int maxtimes)
- long int CMPFBiCGSTAB(CMPFVector ans, CMPFMatrix a, CMPFVector b, mpf_t reps, mpf_t aeps, long int maxtimes)
- long int CMPFGPBiCG(CMPFVector ans, CMPFMatrix a, CMPFVector b, mpf_t reps, mpf_t aeps, long int maxtimes)

Bibliography

[1] D.H. Bailey. QD. https://www.davidhbailey.com/dhbsoftware/.

[2] T. Granlaud and GMP development team. The GNU Multiple Precision arithmetic library. https://gmplib.org/.

[3] M. Jolders, J-. M. Muller, V. Popescu, and W.Tucker. Campary: Cuda mutiple precision arithmetic library and applications. *5th ICMS*, 2016.

[4] Tomonori Kouya. BNCpack. https://na-inet.jp/na/bnc/.

[5] LAPACK. http://www.netlib.org/lapack/.

[6] MPLAPACK/MPBLAS. Multiple precision arithmetic LAPACK and BLAS. https://github.com/nakatamaho/mplapack.

[7] Daichi Mukunoki, Katsuhisa Ozaki, Takeshi Ogita, and Toshiyuki Imamura. Accurate matrix multiplication on binary128 format accelerated by ozaki scheme. In *50th International Conference on Parallel Processing*, ICPP 2021, New York, NY, USA, 2021. Association for Computing Machinery.

[8] MPFR Project. The MPFR library. https://www.mpfr.org/.

[9] S. M. Rump, T. Ogita, and S. Oishi. Accurate floating-point summation part I: Faithful rounding. *SIAM Journal on Scientific Computing*, Vol. 31, No. 1, pp. 189–224, 2008.

[10] S. M. Rump, T. Ogita, and S. Oishi. Accurate floating-point summation part II: Sign, k-fold faithful and rounding to nearest. *SIAM Journal on Scientific Computing*, Vol. 31, No. 2, pp. 1269–1302, 2008.

[11] Taiga Utsugiri and Tomonori Kouya. Acceleration of multiple precision matrix multiplication using ozaki scheme, 2023.

Index

___m256, 13
___m256d, 14–16
___m256d *, 14, 16
___m256d a, 14
___m256d*, 16
__bncavx2__dadd, 14
__bncavx2__ddiv, 14
__bncavx2__ddotp, 14
__bncavx2__dfma, 14
__bncavx2__dmul, 14
__bncavx2__dneg, 14
__bncavx2__dquick__two__diff, 14
__bncavx2__dquick__two__sum, 14
__bncavx2__dsub, 14
__bncavx2__dtwo__diff, 14
__bncavx2__dtwo__prod, 14
__bncavx2__dtwo__sum, 14
__bncavx2__fabs, 14
__bncavx2__fabsf, 13
__bncavx2__ffma, 13
__bncavx2__fmul, 13
__bncavx2__fneg, 13
__bncavx2__get__dd__m256d__i, 14
__bncavx2__get__qd__m256d__i, 15
__bncavx2__get__td__m256d__i, 15
__bncavx2__merge, 15
__bncavx2__rdd__abs, 14
__bncavx2__rdd__absmax256d, 14
__bncavx2__rdd__abssum256d, 14
__bncavx2__rdd__add, 14
__bncavx2__rdd__div, 14
__bncavx2__rdd__mul, 14
__bncavx2__rdd__norm256d, 14
__bncavx2__rdd__set0, 14
__bncavx2__rdd__sub, 14
__bncavx2__rdd__sum256d, 14
__bncavx2__renorm, 16
__bncavx2__renorm4, 16
__bncavx2__rqd__absmax256d, 15
__bncavx2__rqd__abssum256d, 15
__bncavx2__rqd__add, 16
__bncavx2__rqd__div, 16
__bncavx2__rqd__mul, 16
__bncavx2__rqd__mul__d, 16
__bncavx2__rqd__norm256d, 16
__bncavx2__rqd__sub, 16
__bncavx2__rqd__sum256d, 15
__bncavx2__rtd__abs, 15
__bncavx2__rtd__absmax256d, 15
__bncavx2__rtd__abssum256d, 15
__bncavx2__rtd__add, 15
__bncavx2__rtd__addq, 15, 16
__bncavx2__rtd__addt, 15
__bncavx2__rtd__div, 15
__bncavx2__rtd__divq, 15, 16
__bncavx2__rtd__divt, 15
__bncavx2__rtd__divtq, 15
__bncavx2__rtd__mul, 15
__bncavx2__rtd__mul__d, 15
__bncavx2__rtd__mul__dd, 15
__bncavx2__rtd__mulq, 15, 16
__bncavx2__rtd__neg, 15
__bncavx2__rtd__norm256d, 15
__bncavx2__rtd__sub, 15
__bncavx2__rtd__subq, 15
__bncavx2__rtd__sum256d, 15
__bncavx2__set0__dd, 14
__bncavx2__set0__qd, 15
__bncavx2__set0__td, 15
__bncavx2__three__sum, 16
__bncavx2__three__sum2, 16
__bncavx2__to__td, 15
__bncavx2__vec__sum, 15
__bncavx2__vseb, 15
__bncomp__mul__ddmatrix__block, 29
__bncomp__mul__ddmatrix__simple, 29
__bncomp__mul__ddmatrix__strassen, 29
__bncomp__mul__mpfdmatrix__simple, 29
__bncomp__mul__mpfmatrix__block, 29
__bncomp__mul__mpfmatrix__strassen, 29
__bncomp__mul__qdmatrix__block, 29
__bncomp__mul__qdmatrix__simple, 29
__bncomp__mul__qdmatrix__strassen, 29
__bncomp__mul__tdmatrix__block, 29
__bncomp__mul__tdmatrix__simple, 29
__bncomp__mul__tdmatrix__strassen, 29

absmax__col__ddmatrix, 31
absmax__col__mpfmatrix, 31
absmax__col__qdmatrix, 31
absmax__col__tdmatrix, 31
absmax__ddvector, 30
absmax__mpfvector, 30
absmax__qdvector, 30
absmax__row__ddmatrix, 30
absmax__row__mpfmatrix, 30
absmax__row__qdmatrix, 30
absmax__row__tdmatrix, 30
absmax__tdvector, 30
add__cmul__ddvector, 23
add__cmul__mpfvector, 23
add__cmul__qdvector, 23
add__cmul__tdvector, 23
add__ddmatrix, 25
add__ddmatrix__dmat, 30
add__ddvector, 22
add__ddvector__dvec, 30
add__mpfmatrix, 25
add__mpfmatrix__dmat, 30
add__mpfvector, 22
add__mpfvector__dvec, 30
add__qdmatrix, 25
add__qdmatrix__dmat, 30
add__qdvector, 22
add__qdvector__dvec, 30
add__tdmatrix, 25

add_tdmatrix_dmat, 30
add_tdvector, 22
add_tdvector_dvec, 30
add2_ddvector, 22
add2_mpfvector, 22
add2_qdvector, 22
add2_tdvector, 22

bnc_print_env_all, 16
byte_double_sqmat, 16

c_d_add, 8
c_d_div, 8
c_d_mul, 8
c_d_sqr, 9
c_d_sub, 8
c_dd_abs, 9
c_dd_add, 8
c_dd_add_d_dd, 8
c_dd_add_dd_d, 8
c_dd_add_sloppy, 8
c_dd_ceil, 9
c_dd_comp, 8
c_dd_comp_d_dd, 8
c_dd_comp_dd_d, 8
c_dd_copy, 9
c_dd_copy_d, 9
c_dd_div, 8
c_dd_div_d_dd, 9
c_dd_div_dd_d, 8
c_dd_floor, 9
c_dd_mul, 8
c_dd_mul_d_dd, 8
c_dd_mul_dd_d, 8
c_dd_neg, 8
c_dd_pi, 8
c_dd_set, 8
c_dd_set_dd_d, 8
c_dd_set0, 8
c_dd_setnan, 8
c_dd_sloppy_div, 8
c_dd_sqr, 9
c_dd_sqr_d, 9
c_dd_sqrt, 9
c_dd_sub, 8
c_dd_sub_d_dd, 8
c_dd_sub_dd_d, 8
c_dd_sub_sloppy, 8
c_qd_abs, 10
c_qd_add, 9
c_qd_add_qd_dd, 9
c_qd_add_sloppy, 9
c_qd_ceil, 10
c_qd_comp, 10
c_qd_comp_d_qd, 10
c_qd_comp_qd_d, 10
c_qd_copy, 9
c_qd_copy_d, 9
c_qd_copy_dd, 9
c_qd_copy_td, 10
c_qd_div_accurate, 10
c_qd_div_d_qd, 10
c_qd_div_qd_d, 10
c_qd_div_qd_dd, 10
c_qd_div_sloppy, 10
c_qd_floor, 10
c_qd_mul, 9
c_qd_mul_d_qd, 9
c_qd_mul_dd_qd, 9
c_qd_mul_pwr2, 10
c_qd_mul_qd_d, 9
c_qd_mul_qd_dd, 9
c_qd_mul_sloppy, 9
c_qd_neg, 9
c_qd_neg_d, 9
c_qd_neg_dd, 9
c_qd_nint, 10
c_qd_selfadd, 9
c_qd_selfadd_d, 9
c_qd_selfadd_dd, 9
c_qd_selfdiv, 10
c_qd_selfdiv_d, 10
c_qd_selfdiv_dd, 10
c_qd_selfmul, 9
c_qd_selfmul_d, 9
c_qd_selfmul_dd, 9
c_qd_selfsub, 9
c_qd_selfsub_d, 9
c_qd_selfsub_dd, 9
c_qd_set, 10
c_qd_set0double *qdval, 10
c_qd_sqr, 9
c_qd_sqrt, 10
c_qd_sub, 9
c_qd_sub_d_qd, 9
c_qd_sub_dd_qd, 9
c_qd_sub_qd_d, 9
c_qd_sub_qd_dd, 9
c_td_2mtw_dd_td, 11
c_td_abs, 11
c_td_add, 10
c_td_add_td_d, 10
c_td_addq, 9
c_td_comp, 11
c_td_comp_d_td, 11
c_td_comp_td_d, 11
c_td_copy, 10
c_td_copy_d, 10
c_td_copy_dd, 10
c_td_copy_qd, 10
c_td_div, 11
c_td_div_td_d, 11
c_td_divq, 11
c_td_divt, 11
c_td_divtq, 11
c_td_mul_accurate, 10
c_td_mul_d_td, 11
c_td_mul_dd_td_accurate, 10
c_td_mul_dd_td_sloppy, 10
c_td_mul_sloppy, 10
c_td_mul_td_d, 11
c_td_neg, 10
c_td_reci, 11
c_td_sqr, 11
c_td_sqrt, 11
c_td_sqrt_d, 11
c_td_sub, 10
c_td_sub_d_td, 10
c_td_sub_td_d, 10
c_td_subq, 10
c_to_td, 10
CDDMatrix, 18

CDDVector, 18
CMPFMatrix, 19
CMPFVector, 18
cmul_ddmatrix, 25
cmul_ddvector, 22
cmul_mpfmatrix, 25
cmul_mpfvector, 22
cmul_qdvector, 22
cmul_tdmatrix, 25
cmul_tdvector, 22
cmul2_ddvector, 22, 23
const __m256d, 15
const char *, 22, 24, 26, 27
const double, 8, 9, 11
const double *, 8–11
CQDMatrix, 18
CQDVector, 18
CTDMatrix, 18
CTDVector, 18

DD_FALSE, 7
DD_HI, 7
dd_is_negative, 8
dd_is_one, 8
dd_is_positive, 8
dd_is_zero, 8
DD_ISINF, 7
DD_ISNAN, 7
DD_ISNEGATIVE, 7
DD_ISONE, 7
DD_ISZERO, 7
DD_LOW, 7
DD_NAN, 8
DD_TRUE, 7
ddfloat, 22, 24
DDLUdecomp, 32
DDLUdecomp_oz, 33
DDLUdecomp_ozPM, 33
DDLUdecomp_strassen, 33
DDLUdecomp_strassenPM, 33
DDLUdecompC, 32
DDLUdecompP, 32
DDMatrix, 18, 23–33
DDSIZE, 7
DDVector, 18, 21–23, 25–27, 29–32
DFMA, 7
diag_ddmatrix, 27
diag_mpfmatrix, 27
diag_qdmatrix, 27
diag_tdmatrix, 27
DLUdecomp, 32
DLUdecomp_oz, 33
DLUdecomp_ozPM, 33
DLUdecomp_strassen, 33
DLUdecomp_strassenPM, 33
DLUdecompC, 32
DLUdecompP, 32
DMatrix, 18, 26, 28–33
DMatrix org_mat, 30
DMatrix ret_high_mat, 30
double, 8–12, 14–18, 21–31
double *, 8–11, 17, 18, 21, 23, 29
dquadratic_eq, 17
DVector, 17, 25, 29–32

extract_dvector, 30

fget_real_sec, 17
fget_real_secv, 17
fget_sec, 17
fget_secv, 17
FILE *, 11, 26
float, 13
frank_ddmatrix, 27
frank_mpfmatrix, 27
frank_qdmatrix, 27
frank_tdmatrix, 27
fread_ddmatrix, 26
fread_ddmatrix_fname, 26
fread_ddvector, 26
fread_ddvector_fname, 26
fread_mpfmatrix, 26
fread_mpfmatrix_fname, 26
fread_mpfvector, 26
fread_mpfvector_fname, 26
fread_qdmatrix, 26
fread_qdmatrix_fname, 26
fread_qdvector, 26
fread_qdvector_fname, 26
fread_tdmatrix, 26
fread_tdmatrix_fname, 26
fread_tdvector, 26
fread_tdvector_fname, 26
free_ddmatrix, 24
free_ddvector, 22
free_mpfmatrix, 24
free_mpfvector, 22
free_qdmatrix, 24
free_qdvector, 22
free_tdmatrix, 24
free_tdvector, 22
fwrite_ddmatrix, 26
fwrite_ddmatrix_fname, 26
fwrite_ddvector, 26
fwrite_ddvector_fname, 26
fwrite_mpfmatrix, 26
fwrite_mpfmatrix_fname, 26
fwrite_mpfvector, 26
fwrite_mpfvector_fname, 26
fwrite_qdmatrix, 26
fwrite_qdmatrix_fname, 26
fwrite_qdvector, 26
fwrite_qdvector_fname, 26
fwrite_tdmatrix, 26
fwrite_tdmatrix_fname, 26
fwrite_tdvector, 26
fwrite_tdvector_fname, 26

get_bnc_default_prec, 16
get_bnc_default_rounding_mode, 16
GET_DDMATRIX_IJ, 23
get_ddmatrix_ij, 23
get_ddmatrix_ij_ddfloat, 23
GET_DDVECTOR_I, 21
get_ddvector_i, 21
get_ddvector_i_ddfloat, 21
GET_MPFMATRIX_IJ, 23
get_mpfmatrix_ij, 23
GET_MPFVECTOR_I, 21
get_mpfvector_i, 21
GET_QDMATRIX_IJ, 23
get_qdmatrix_ij, 23
get_qdmatrix_ij_qdfloat, 23

GET_QDVECTOR_I, 21
get_qdvector_i, 21
get_qdvector_i_qdfloat, 21
get_real_sec, 17
get_real_secv, 17
get_sec, 17
get_secv, 17
GET_TDMATRIX_IJ, 23
get_tdmatrix_ij, 23
get_tdmatrix_ij_tdfloat, 23
GET_TDVECTOR_I, 21
get_tdvector_i, 21
get_tdvector_i_tdfloat, 21

hilbert_ddmatrix, 27
hilbert_mpfmatrix, 27
hilbert_qdmatrix, 27
hilbert_tdmatrix, 27

im_rand_ddmatrix, 28
im_rand_mpfmatrix, 28
im_rand_qdmatrix, 28
im_rand_tdmatrix, 28
in, 21
init_ddmatrix, 24
init_ddvector, 21
init_mpfmatrix, 24
init_mpfvector, 21
init_qdmatrix, 24
init_qdvector, 21
init_tdmatrix, 24
init_tdvector, 21
init2_mpfmatrix, 24
init2_mpfvector, 22
int, 10, 11, 13–19, 22, 24–26, 29–31, 33
int *, 8, 10, 11
int num_div, 31
int_sym_rand_ddmatrix, 27
int_sym_rand_mpfmatrix, 27
int_sym_rand_qdmatrix, 27
int_sym_rand_tdmatrix, 27
int_unsym_rand_ddmatrix, 27
int_unsym_rand_mpfmatrix, 27
int_unsym_rand_qdmatrix, 27
int_unsym_rand_tdmatrix, 27
inv_ddmatrix, 25
inv_ddmatrix_strassen_even, 29
inv_mpfmatrix, 25
inv_qdmatrix, 25
inv_qdmatrix_strassen_even, 29
inv_tdmatrix, 25
inv_tdmatrix_strassen_even, 29
ip_ddvector, 23
ip_mpfvector, 23
ip_qdvector, 23
ip_tdvector, 23

long int, 21–24, 26–33
long int *, 30, 31
long int dim, 27
lotkin_ddmatrix, 27
lotkin_mpfmatrix, 27
lotkin_qdmatrix, 27
lotkin_tdmatrix, 27

matmul_gflops, 16
merge, 10
MFPMatrix, 32
mp_rnd_t, 16
mpf_atan, 17
mpf_exp, 16
mpf_fma, 16
mpf_log, 16
mpf_log10, 16
mpf_nrand, 17
mpf_sin, 17
mpf_srand, 17
mpf_t, 16–19, 22–25, 28–31
mpf_tan, 17
mpf_urand, 17
MPFLUdecomp, 32
MPFLUdecomp_oz, 33
MPFLUdecomp_ozPM, 33
MPFLUdecomp_strassen, 33
MPFLUdecomp_strassenPM, 33
MPFLUdecompC, 32
MPFLUdecompP, 32
MPFMatrix, 18, 23–33
MPFMatrix b, 28
mpfquadratic_eq, 17
MPFVector, 18, 22, 23, 25–27, 29–32
mul_ddmatrix, 24, 28
mul_ddmatrix_block, 28
mul_ddmatrix_ddvec, 25
mul_ddmatrix_ddvec_oz, 31
mul_ddmatrix_oz, 31
mul_ddmatrix_simple, 28
mul_ddmatrix_strassen, 28
mul_ddmatrix_strassen_even, 28
mul_ddmatrix_winograd_even, 28
mul_ddmatrixt_ddvec, 25
mul_dmatrix_block, 28
mul_dmatrix_strassen, 28
mul_dmatrix_strassen_even, 28
mul_dmatrix_winograd_even, 28
mul_mpfmatrix, 24
mul_mpfmatrix_block, 28
mul_mpfmatrix_dvec_oz, 31
mul_mpfmatrix_mpfvec, 25
mul_mpfmatrix_oz, 31
mul_mpfmatrix_simple, 28
mul_mpfmatrix_strassen, 28
mul_mpfmatrix_strassen_even, 28
mul_mpfmatrix_winograd_even, 29
mul_mpfmatrixt_mpfvec, 25
mul_qdmatrix, 24, 28
mul_qdmatrix_block, 28
mul_qdmatrix_oz, 31
mul_qdmatrix_qdvec, 25
mul_qdmatrix_qdvec_oz, 31
mul_qdmatrix_simple, 28
mul_qdmatrix_strassen, 28
mul_qdmatrix_strassen_even, 28
mul_qdmatrix_winograd_even, 29
mul_qdmatrixt_qdvec, 25
mul_tdmatrix, 24, 28
mul_tdmatrix_block, 28
mul_tdmatrix_oz, 31
mul_tdmatrix_simple, 28
mul_tdmatrix_strassen, 28
mul_tdmatrix_strassen_even, 28
mul_tdmatrix_tdvec, 25

mul_tdmatrix_tdvec_oz, 31
mul_tdmatrix_winograd_even, 29
mul_tdmatrixt_tdvec, 25
mylog2, 16

neg_ddvector, 23
neg_mpfvector, 23
neg_qdvector, 23
neg_tdvector, 23
nint, 10
norm1_ddmatrix, 24
norm1_ddvector, 23
norm1_mpfmatrix, 25
norm1_mpfvector, 23
norm1_qdmatrix, 25
norm1_qdvector, 23
norm1_tdmatrix, 24
norm1_tdvector, 23
norm2_ddvector, 23
norm2_mpfvector, 23
norm2_qdvector, 23
norm2_tdvector, 23
normf_ddmatrix, 24
normf_mpfmatrix, 24
normf_qdmatrix, 24
normf_tdmatrix, 24
normi_ddmatrix, 24
normi_ddvector, 23
normi_mpfmatrix, 24
normi_mpfvector, 23
normi_qdmatrix, 24
normi_qdvector, 23
normi_tdmatrix, 24
normi_tdvector, 23

ONE_M_2DBL_EPS, 11
ONE_P_2DBL_EPS, 11

pascal_ddmatrix, 28
pascal_mpfmatrix, 28
pascal_qdmatrix, 28
pascal_tdmatrix, 28
print_ddmatrix, 24
print_ddvector, 22
print_mpfmatrix, 24
print_mpfvector, 22
print_normf_ddmatrix, 24
print_qdmatrix, 24
print_qdvector, 22
print_tdmatrix, 24
print_tdvector, 22

QD_FALSE, 7
QD_FMA, 7
QD_FMS, 7
QD_ISINF, 7
QD_ISNAN, 7
QD_ISNEGATIVE, 7
QD_ISONE, 7
QD_ISZERO, 7
QD_NAN, 8
QD_TRUE, 7
qdfloat, 22, 24
qdfloat *, 15
QDLUdecomp, 32
QDLUdecomp_oz, 33
QDLUdecomp_ozPM, 33
QDLUdecomp_strassen, 33
QDLUdecomp_strassenPM, 33
QDLUdecompC, 32
QDLUdecompP, 32
QDMatrix, 18, 23–33
QDMatrixmat, 31
QDSIZE, 7
QDVector, 18, 21–23, 25–27, 29–32
quick_renorm, 9
quick_two_diff, 8
quick_two_sum, 8

rdd_abs, 12
rdd_add, 12
rdd_add_d, 12
rdd_add_ui, 12
rdd_cmp, 11
rdd_cmp_d, 11
rdd_cmp_ui, 12
rdd_div, 12
rdd_div_d, 12
rdd_div_ui, 12
rdd_fma, 11
rdd_get_d, 12
rdd_mul, 12
rdd_mul_d, 12
rdd_mul_ui, 12
rdd_neg, 12
rdd_nrand, 17
rdd_out_str_base, 11
rdd_pow, 11
rdd_set, 12
rdd_set_d, 12
rdd_set_ui, 12
rdd_set0, 12
rdd_sqrt, 12
rdd_sqrt_d, 11, 12
rdd_sqrt_ui, 12
rdd_srand, 17
rdd_sub, 12
rdd_sub_d, 12
rdd_sub_ui, 12
rdd_ui_div, 12
rdd_ui_sub, 12
rdd_urand, 17
read_test_linear_eq_dd, 27
read_test_linear_eq_mpf, 27
read_test_linear_eq_qd, 27
read_test_linear_eq_td, 27
relerr_ddvector, 26
relerr_ddvector_mpfvec, 25
relerr_element_ddvector, 26
relerr_element_ddvector_mpf, 25
relerr3_cddmatrix, 18
relerr3_cddvector, 18
relerr3_cmpfmatrix, 19
relerr3_cmpfvector, 18
relerr3_cqdmatrix, 18
relerr3_cqdvector, 18
relerr3_ctdmatrix, 18
relerr3_ctdvector, 18
relerr3_ddmatrix, 18, 29
relerr3_ddvector, 18, 29
relerr3_dmatrix, 18, 29
relerr3_dvector, 17, 29

relerr3_mpfmatrix, 18, 30
relerr3_mpfvector, 18, 29
relerr3_qdmatrix, 18, 30
relerr3_qdvector, 18, 29
relerr3_tdmatrix, 18, 30
relerr3_tdvector, 18, 29
relerr3_tdvector_qdvec, 18
renorm, 9
renorm4, 9
row_swap_ddmatrix, 26
rqd_abs, 13
rqd_add, 13
rqd_add_d, 13
rqd_add_ui, 13
rqd_cmp, 11
rqd_cmp_d, 11
rqd_cmp_ui, 13
rqd_div, 13
rqd_div_d, 13
rqd_div_ui, 13
rqd_fma, 11
rqd_get_d, 13
rqd_mul, 13
rqd_mul_d, 13
rqd_mul_ui, 13
rqd_neg, 13
rqd_nrand, 17
rqd_out_str_base, 11
rqd_pow, 11
rqd_set, 13
rqd_set_d, 13
rqd_set_ui, 13
rqd_set0, 13
rqd_sqrt, 13
rqd_sqrt_d, 11, 13
rqd_sqrt_ui, 13
rqd_srand, 17
rqd_sub, 13
rqd_sub_d, 13
rqd_sub_ui, 13
rqd_ui_div, 13
rqd_ui_sub, 13
rqd_urand, 17
rtd_abs, 13
rtd_add, 12
rtd_add_d, 13
rtd_add_ui, 13
rtd_addq, 12
rtd_addt, 12
rtd_cmp, 11
rtd_cmp_d, 12
rtd_cmp_ui, 13
rtd_div, 12
rtd_div_d, 13
rtd_div_ui, 13
rtd_divq, 12
rtd_divt, 12
rtd_divtq, 12
rtd_fma, 12
rtd_get_d, 12
rtd_mul, 12
rtd_mul_d, 13
rtd_mul_ui, 13
rtd_neg, 13
rtd_nrand, 17
rtd_out_str_base, 11
rtd_pow, 12
rtd_set, 13
rtd_set_d, 12
rtd_set_ui, 12
rtd_set0, 12
rtd_sqrt, 12
rtd_sqrt_d, 12
rtd_sqrt_ui, 12
rtd_srand, 17
rtd_sub, 12
rtd_sub_d, 13
rtd_sub_ui, 13
rtd_subq, 12
rtd_subt, 12
rtd_ui_div, 13
rtd_ui_sub, 13
rtd_urand, 17

set_bnc_default_prec, 16
set_bnc_default_prec_decimal, 16
set_bnc_rounding_mode, 16
set_ddfloat_ddmat, 24
set_ddfloat_ddvec, 22
set_ddmatrix_ddfloat, 24
SET_DDMATRIX_IJ, 23
set_ddmatrix_ij, 23
SET_DDMATRIX_IJ_D, 24
set_ddmatrix_ij_d, 24
SET_DDMATRIX_IJ_UI, 24
set_ddmatrix_ij_ui, 24
set_ddvector_ddfloat, 22
SET_DDVECTOR_I, 21
set_ddvector_i, 21
SET_DDVECTOR_I_D, 21
set_ddvector_i_d, 21
set_ddvector_i_str, 22
SET_MPFMATRIX_IJ, 23
set_mpfmatrix_ij, 23
set_mpfvector_i_str, 22
set_qdfloat_qdmat, 24
set_qdfloat_qdvec, 22
SET_QDMATRIX_IJ, 23
set_qdmatrix_ij, 23
set_qdmatrix_qdfloat, 24
SET_QDVECTOR_I, 21
set_qdvector_i, 21
SET_QDVECTOR_I_D, 21
set_qdvector_i_d, 21
set_qdvector_i_str, 22
set_qdvector_qdfloat, 22
set_tdfloat_tdmat, 24
set_tdfloat_tdvec, 22
SET_TDMATRIX_IJ, 23
set_tdmatrix_ij, 23
set_tdmatrix_tdfloat, 24
SET_TDVECTOR_I, 21
set_tdvector_i, 21
SET_TDVECTOR_I_D, 21
set_tdvector_i_d, 21
set_tdvector_i_str, 22
set_tdvector_tdfloat, 22
SET0_DD, 11
set0_dd, 12
set0_ddmatrix, 24
SET0_DDMATRIX_IJ, 24
set0_ddmatrix_ij, 24

set0_ddvector, 22
SET0_DDVECTOR_I, 21
set0_ddvector_i, 21
set0_mpfmatrix, 24
set0_mpfvector, 22
SET0_QD, 11
set0_qd, 13
set0_qdmatrix, 24
set0_qdvector, 22
SET0_QDVECTOR_I, 21
set0_qdvector_i, 21
SET0_TD, 11
set0_td, 12
set0_tdmatrix, 24
set0_tdvector, 22
SET0_TDVECTOR_I, 21
set0_tdvector_i, 21
setI_ddmatrix, 25
setI_mpfmatrix, 25
setI_qdmatrix, 25
setI_tdmatrix, 25
SFMA, 7
SolveDDLS, 32
SolveDDLSC, 32
SolveDDLSP, 32
SolveDLS, 32
SolveDLSC, 32
SolveDLSP, 32
SolveMPFLS, 32
SolveMPFLSC, 32
SolveMPFLSP, 32
SolveQDLS, 32
SolveQDLSC, 32
SolveQDLSP, 32
SolveTDLS, 32
SolveTDLSC, 32
SolveTDLSP, 32
split, 8
split_ddmatrix_dmat, 30
split_ddmatrix_t_dmat, 31
split_ddvector_dvec, 30
split_dmatrix, 30
split_dmatrix_t, 30
split_mpfmatrix_dmat, 31
split_mpfmatrix_t_dmat, 31
split_qdmatrix_dmat, 31
split_qdmatrix_t_dmat, 31
split_qdvector_dvec, 31
split_tdmatrix_dmat, 30
split_tdmatrix_t_dmat, 31
split_tdvector_dvec, 30
sub_ddmatrix, 25
sub_ddmatrix_dmat, 30
sub_ddvector, 22
sub_ddvector_dvec, 30
sub_mpfmatrix, 25
sub_mpfmatrix_dmat, 30
sub_mpfvector, 22
sub_mpfvector_dvec, 30
sub_qdmatrix, 25
sub_qdmatrix_dmat, 30
sub_qdvector, 22
sub_qdvector_dvec, 30
sub_tdmatrix, 25
sub_tdmatrix_dmat, 30
sub_tdvector, 22
sub_tdvector_dvec, 30
sub2_ddvector, 22
sub2_mpfvector, 22
sub2_qdvector, 22
sub2_tdvector, 22
subst_ddmatrix, 25
subst_ddmatrix_dmat, 26
subst_ddmatrix_mpfmat, 25
subst_ddvector_dvec, 25
subst_ddvector_mpfvec, 25
subst_dmatrix_ddmat, 26
subst_dmatrix_qdmat, 31
subst_dvector_ddvec, 25
subst_dvector_qdvec, 30
subst_mpfmatrix, 25
subst_mpfmatrix_ddmat, 25
subst_mpfvector_ddvec, 25
subst_qdmatrix, 25
subst_qdmatrix_dmat, 31
subst_tdmatrix, 25

TD_FALSE, 7
TD_ISINF, 7
TD_ISNAN, 7
TD_ISNEGATIVE, 7
TD_ISONE, 7
TD_ISZERO, 7
TD_NAN, 8
TD_TRUE, 7
tdfloat, 22, 24
tdfloat *, 15
TDLUdecomp, 32
TDLUdecomp_oz, 33
TDLUdecomp_ozPM, 33
TDLUdecomp_strassen, 33
TDLUdecomp_strassenPM, 33
TDLUdecompC, 32
TDLUdecompP, 32
TDMatrix, 18, 23–33
TDSIZE, 7
TDVector, 18, 21–23, 25–27, 29–32
three_sum, 9
three_sum2, 9
toeplitz_ddmatrix, 27
toeplitz_mpfmatrix, 28
toeplitz_qdmatrix, 28
toeplitz_tdmatrix, 28
transpose_ddmatrix, 25
transpose_mpfmatrix, 25
transpose_qdmatrix, 25
transpose_tdmatrix, 25
tridiag_ddmatrix, 27
tridiag_mpfmatrix, 27
tridiag_qdmatrix, 27
tridiag_tdmatrix, 27
two_diff, 8
two_prod, 8
two_sqr, 8
two_sum, 8

unsigned long, 16, 17, 22, 24, 28
USE_QD_DIV_ACCURATE, 10
USE_TD_MUL_ACCURATE, 10
USE_TD_MUL_DD_TD_ACCURATE, 10

vec_sum, 10

void, 15, 17
vseb, 10